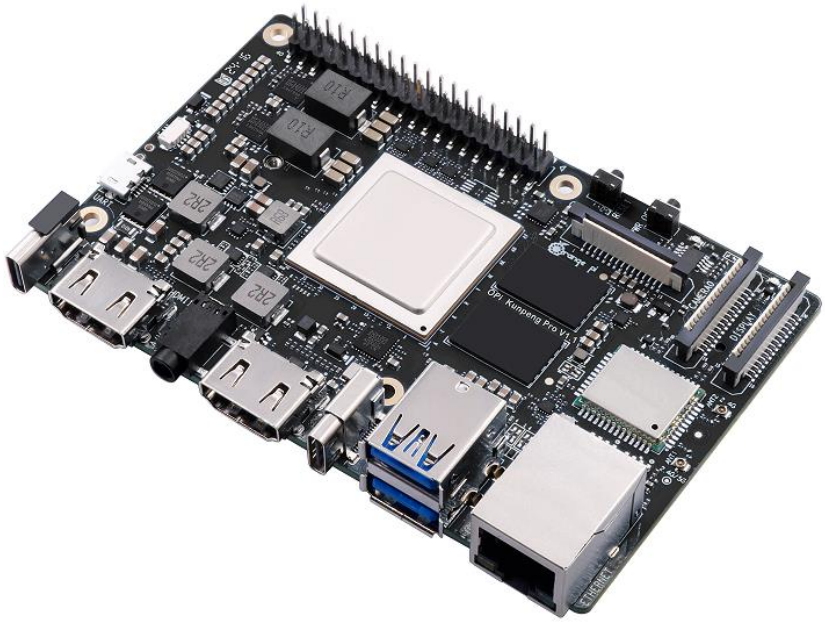
Orange Pi Kunpeng Pro
用户手册



目录

1. 开发板参数介绍 1
1.1. 开发板简介 1
1.2. v1.1 版本开发板的硬件规格（不带 FPGA） 1
1.3. v1.2 版本开发板的硬件规格（带 FPGA） 2
1.4. 开发板的顶层视图和底层视图 4
1.5. v1.1 版本开发板的接口详情图 5
1.6. v1.2 版本开发板的接口详情图 6
2. 开发板使用介绍 7
2.1. 准备需要的配件 7
2.2. 下载开发板的镜像和相关的资料 11
2.3. 控制启动设备的两个拨码开关的使用说明 12
2.4. 烧写 Linux 镜像到 TF 卡中的方法 12
2.4.1. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法 12
2.4.2. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法 16
2.5. 烧写 Linux 镜像到 eMMC 中的方法 20
2.6. 烧写 Linux 镜像到 NVMe SSD 中的方法 25
2.7. 烧写 Linux 镜像到 SATA SSD 中的方法 31
2.8. 启动开发板的步骤 38
2.9. 调试串口的使用方法 39
2.9.1. 通过 Micro USB 接口来使用调试串口的连接说明 39
2.9.2. 通过 40 pin 接口中的 uart0 来使用调试串口的连接说明 40
2.9.3. Ubuntu 平台调试串口的使用方法 41
2.9.4. Windows 平台调试串口的使用方法 44
2.10. WIFI 蓝牙天线使用注意事项 47
2.11. v1.2 版本开发板搭配 OPi MSOC 的使用说明 48

3. openEuler 桌面系统使用说明 50
3.1. 已支持的 openEuler 镜像类型和内核版本 50
3.2. openEuler 系统功能适配情况 50
3.3. Linux 系统登录说明 51
3.3.1. 登录 Linux 系统桌面的方法 51
3.3.2. Linux 系统默认登录账号和密码 52
3.4. 板载 LED 灯测试说明 53
3.5. 网络连接测试 53
3.5.1. 以太网口测试 53
3.5.2. WIFI 连接测试 55
3.6. SSH 远程登录开发板 61
3.6.1. Ubuntu 下 SSH 远程登录开发板 61
3.6.2. Windows 下 SSH 远程登录开发板 62
3.7. 蓝牙使用方法 64
3.8. USB 接口测试 66
3.8.1. Type-C USB 接口使用注意事项 66
3.8.2. 连接 USB 鼠标或键盘测试 67
3.8.3. USB 音频测试 67
3.9. 40 Pin 接口引脚功能说明 69
3.10. 40 pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试 71
3.10.1. 40 pin GPIO 口的测试方法 71
3.10.2. 40 pin SPI 回环测试 74
3.10.3. 40 pin I2C 测试 76
3.10.4. 40 pin UART 测试 77
3.10.5. 40 pin PWM 测试 79
3.10.6. 40 pin CAN 的测试方法 81
3.11. wiringOP 的安装使用方法 88
3.11.1. 安装 wiringOP 的方法 88
3.11.2. 使用 wiringOP 控制 40pin GPIO 的方法 89
3.12. wiringOP 硬件 PWM 的使用方法 91

3.12.1. 使用 wiringOP 的 gpio 命令设置 PWM 的方法 92
3.12.2. PWM 测试程序的使用方法 96
3.13. 上传文件到开发板 Linux 系统中的方法 97
3.13.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法 97
3.13.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法 101
3.14. 散热风扇的使用方法 105
3.15. 设置 Swap 内存的方法 106
3.16. 关机和重启开发板的方法 107
4. Linux 内核源码包的使用说明 109
4.1. 编译主机系统的需求 109
4.2. 安装交叉编译工具链和依赖包 110
4.3. 下载解压 Linux 内核源码包 112
4.4. 编译并生效内核 Image 文件的方法 113
4.5. 编译并生效内核 DTB 文件的方法 115
5. 附录 117
5.1. 用户手册更新历史 117
5.2. 镜像更新历史 117

1. 开发板参数介绍

1.1. 开发板简介

Orange Pi Kunpeng Pro 开发板是香橙派联合华为精心打造的高性能开发板，其搭载了鲲鹏处理器，提供了 8GB 和 16GB 两种内存版本。Kunpeng Pro 开发板结合了鲲鹏全栈根技术，全面使能高校计算机系统教学和原生开发。同时支持 FPGA+ARM，从体系结构、数字逻辑设计、操作系统和编译，再到嵌入式开发，可以基于同一套体系结构和一套开发板实现贯穿打通。

1.2. v1.1 版本开发板的硬件规格（不带 FPGA）

Orange Kunpeng Pro 开发板硬件规格	
处理器	4 核 64 位 Arm 处理器
内存	• 类型：LPDDR4X • 容量：8GB 或 16GB
存储	• 板载 32MB 的 SPI Flash • Micro SD 卡插槽 • eMMC 插座：可外接 eMMC 模块 • M.2 M-Key 接口：可接 2280 规格的 NVMe SSD 或 SATA SSD
以太网	• 支持 10/100/1000Mbps • 板载 PHY 芯片：RTL8211F
Wi-Fi+蓝牙	• 支持 2.4G 和 5G 双频 WIFI • BT4.2 • 模组：欧智通 6221BUUC
USB	• 2 个 USB3.0 Host 接口 • 1 个 Type-C 接口（只支持 USB3.0，不支持 USB2.0）
摄像头	2 个 MIPI CSI 2 Lane 接口
显示	• 2 个 HDMI 接口 • 1 个 MIPI DSI 2 Lane 接口
音频	• 1 个 3.5mm 耳机孔，支持音频输入输出

	• 2 个 HDMI 音频输出
40 pin 扩展口	用于扩展 UART、I2C、SPI、PWM 和 GPIO 等接口
按键	1 个复位键，1 个关机键，1 个升级按键
拨码开关	2 个拨码开关：用于控制 SD 卡、eMMC 和 SSD 启动选项
电源	支持 Type-C 供电，20V PD-65W 适配器
LED 灯	1 个电源指示灯和 1 个软件可控指示灯
风扇接口	4pin，0.8mm 间距，用于接 12V 风扇，支持 PWM 控制
电池接口	2pin，2.54mm 间距，用于接 3 串电池，支持快充
调试串口	Micro USB 接口的调试串口
支持的操作系统	openEuler 22.03
外观规格介绍	
产品尺寸	107*68mm
重量	82g
rangePi™是深圳市迅龙软件有限公司的注册商标	

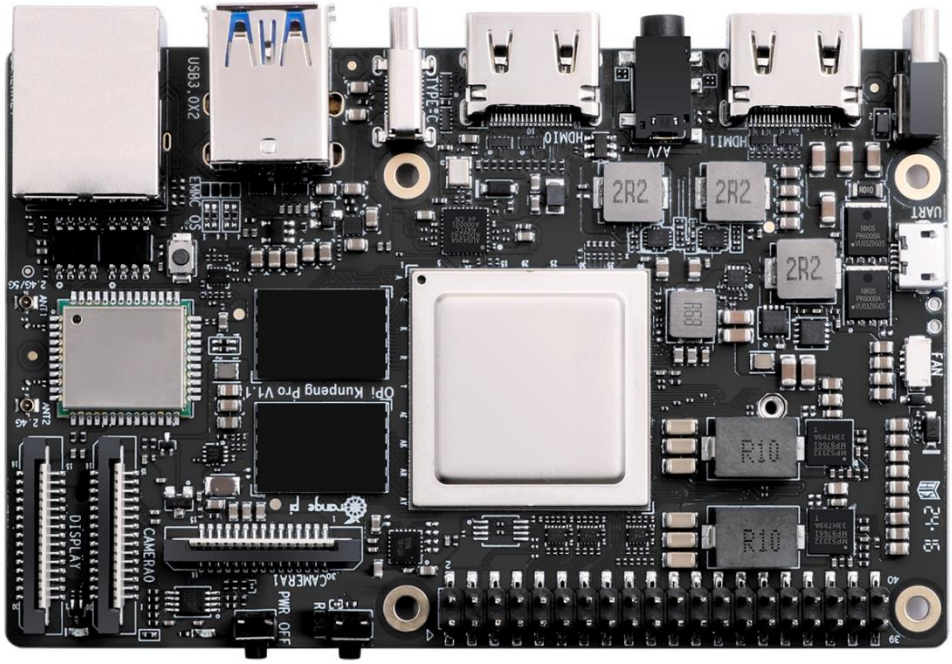
1.3. v1.2 版本开发板的硬件规格（带 FPGA）

Orange Kunpeng Pro 开发板硬件规格	
处理器	4 核 64 位 Arm 处理器
内存	• 类型：LPDDR4X • 容量：8GB 或 16GB
存储	• 板载 32MB 的 SPI Flash • Micro SD 卡插槽 • eMMC 插座：可外接 eMMC 模块 • M.2 M-Key 接口：可接 2280 规格的 NVMe SSD 或 SATA SSD
以太网	• 支持 10/100/1000Mbps • 板载 PHY 芯片：RTL8211F
Wi-Fi+蓝牙	• 支持 2.4G 和 5G 双频 WIFI • BT4.2

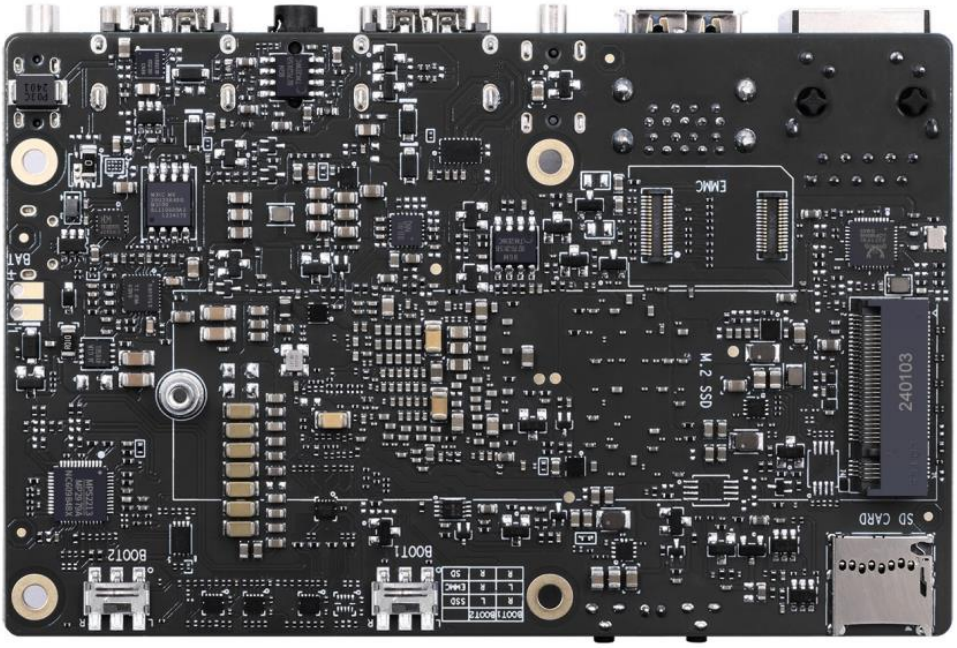
	• 模组：欧智通 6221BUUC
USB	• 1 个 USB3.0 Host 接口 • 1 个 USB2.0 Host 接口 • 1 个 Type-C 接口（只支持 USB3.0，不支持 USB2.0）
摄像头	2 个 MIPI CSI 2 Lane 接口
显示	• 2 个 HDMI 接口 • 1 个 MIPI DSI 2 Lane 接口
音频	• 1 个 3.5mm 耳机孔，支持音频输入输出 • 2 个 HDMI 音频输出
40 pin 扩展口	用于扩展 UART、I2C、SPI、PWM 和 GPIO 等接口
按键	1 个复位键，1 个关机键，1 个升级按键
拨码开关	2 个拨码开关：用于控制 SD 卡、eMMC 和 SSD 启动选项
电源	支持 Type-C 供电，20V PD-65W 适配器
LED 灯	1 个电源指示灯和 1 个软件可控指示灯
风扇接口	4pin，0.8mm 间距，用于接 12V 风扇，支持 PWM 控制
电池接口	2pin，2.54mm 间距，用于接 3 串电池，支持快充
调试串口	Micro USB 接口的调试串口
OPi MSOC	M.2 接口中包含有专用的 USB 转 JTAG 和 UART 接口
支持的操作系统	openEuler 22.03
外观规格介绍	
产品尺寸	107*68mm
重量	82g
rangePi™是深圳市迅龙软件有限公司的注册商标	

1.4. 开发板的顶层视图和底层视图

顶层视图：

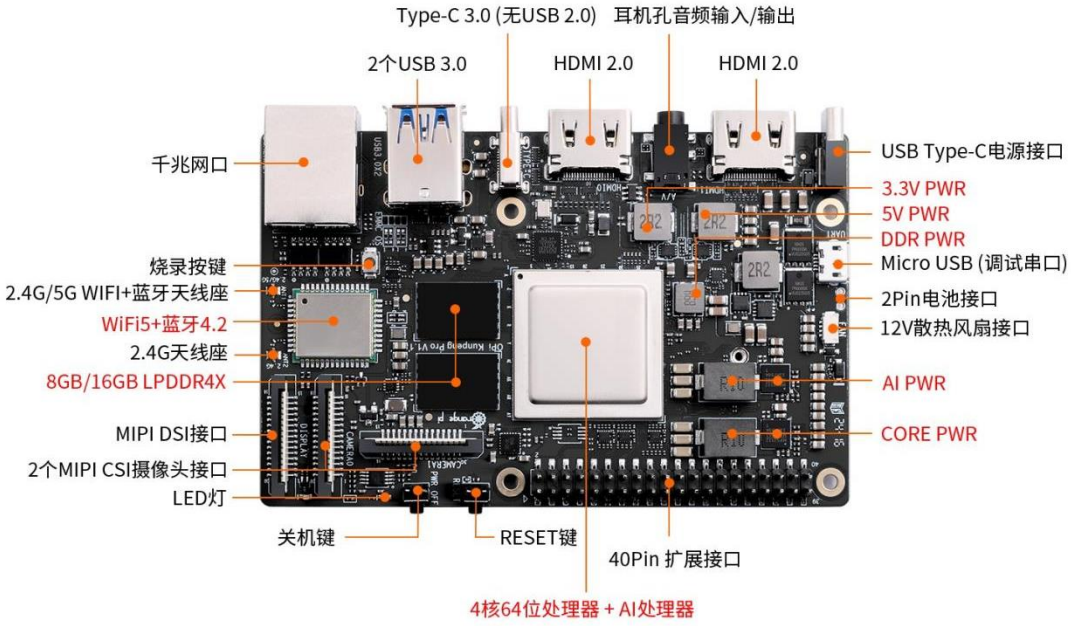


底层视图：

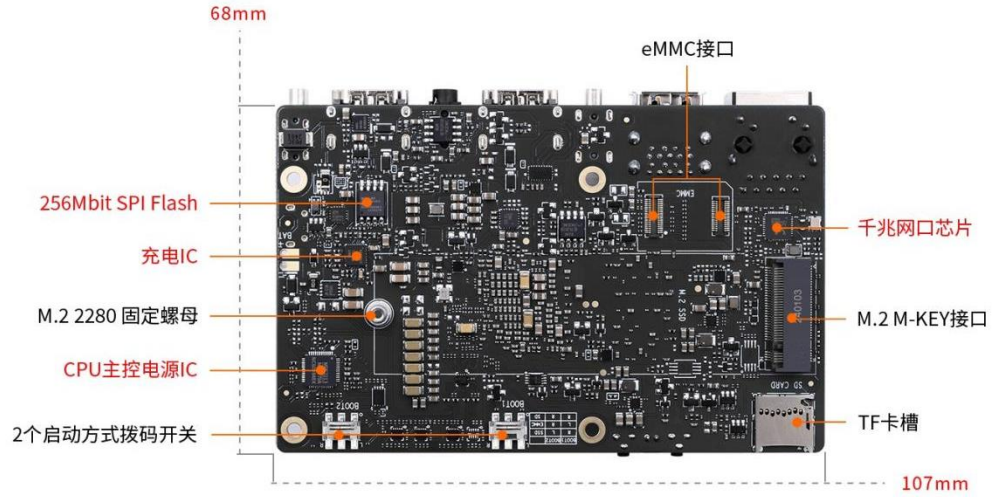


1.5. v1.1 版本开发板的接口详情图

顶层视图

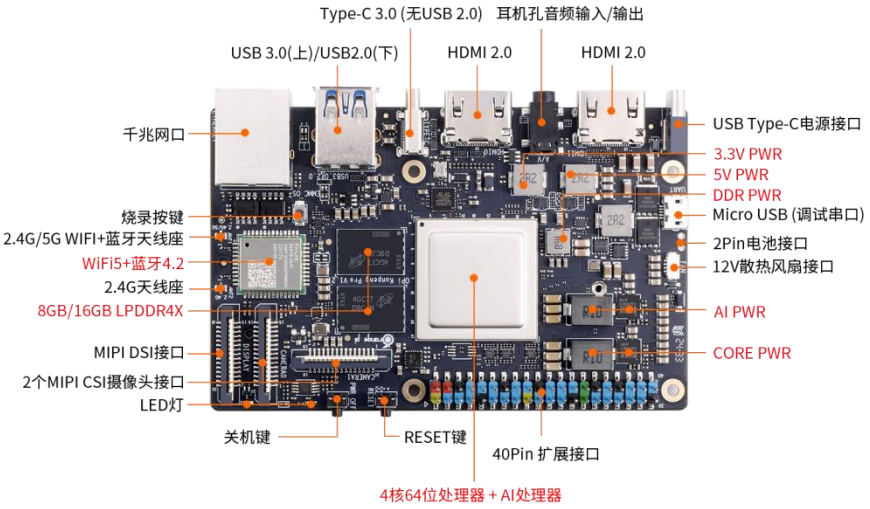


底层视图

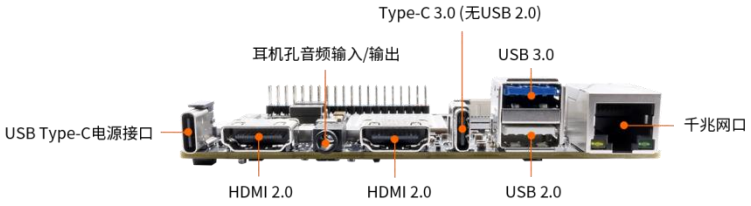


1.6. v1.2 版本开发板的接口详情图

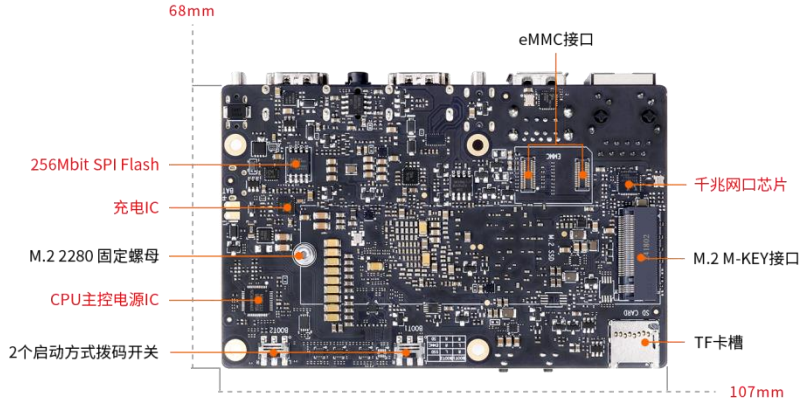
顶层视图



侧面视图



底层视图



2. 开发板使用介绍

2.1. 准备需要的配件

1) TF 卡，最小 32GB 容量的 **class10** 级或以上的高速闪迪卡。强烈推荐使用 64GB 或以上容量的 TF 卡。



2) TF 卡读卡器，用于读写 TF 卡。



3) HDMI 转 HDMI 连接线，用于将开发板连接到 HDMI 显示器或者电视进行显示。



4) Type-C转USB3.0 转接线，用于Type-C接口连接USB3.0 的存储设备。



5) 电源，Type-C 接口的 20V PD-65W 适配器。



6) M.2 M-Key 2280 规格PCIe NVMe SSD。

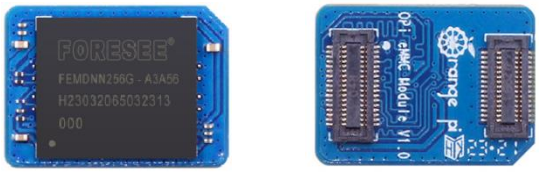
注意，NVMe SSD目前测试了樊想、金士顿和三星的，只有三星的NVMe SSD能稳定运行Linux系统。非三星品牌的NVMe SSD需要等后续软件更新才能正常使用。



7) M.2 M-Key 2280 规格的SATA SSD。



8) eMMC模块



9) USB接口的鼠标和键盘。



10) USB摄像头。



11) 金属配套外壳。



12) 百兆或者千兆网线，用于将开发板连接到因特网。



13) 12V 的散热风扇。

14) 树莓派 IMX219 型号的摄像头。



15) 树莓派 5 寸 MIPI LCD 显示屏。



16) OPi MSOC FPGA 开发板。

注意，OPi MSOC只能搭配v1.2 版本的Kunpeng Pro开发板使用。



17) Micro USB 接口的数据线，用于串口调试。



18) 安装有 Ubuntu 22.04 和 Windows 操作系统的 X64 电脑。

1	**Ubuntu22.04 PC**	**可选，用于编译 Linux 源码**
2	**Windows PC**	**用于烧录 openEuler 镜像**

2.2. 下载开发板的镜像和相关的资料

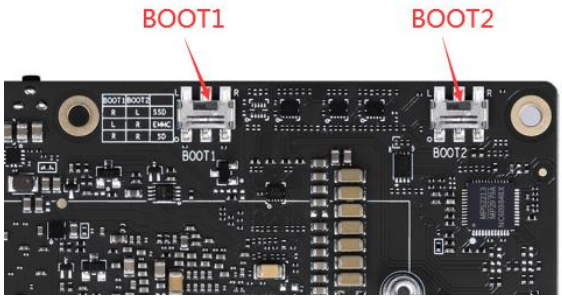
开发板资料下载页面的链接如下所示：

http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-kunpeng.html



2.3. 控制启动设备的两个拨码开关的使用说明

开发板支持从 TF 卡、eMMC 和 SSD（支持 NVMe SSD 和 SATA SSD）启动。具体从哪个设备启动是由开发板背面的两个拨码（BOOT1 和 BOOT2）开关来控制的。



BOOT1 和 BOOT2 两个拨码开关都支持左右两种设置状态，所以总共有 4 种设置状态，开发板目前只使用了其中的三种。不同的设置状态对应的启动设备如下表所示：

拨码开关 BOOT1	拨码开关 BOOT2	对应的启动设备
左	左	未使用
右	左	SATA SSD 和 NVMe SSD
左	右	eMMC
右	右	TF 卡

注意，SATA SSD和NVMe SSD的启动方式对应的拨码开关的设置状态是一样的。这两种启动方式是通过M2_TYPE引脚的电平来自动区分的。

另外请注意，切换拨码开关后必须重新拔插电源上下电才能让新的启动设备选项生效。通过开发板的复位按键来复位系统是不会让拨码开关新设置的配置生效的。

2.4. 烧写 Linux 镜像到 TF 卡中的方法

2.4.1. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从开发板的资料下载页面下载的

openEuler镜像。

1) 首先准备一张 32GB 或更大容量的 TF 卡（**推荐使用 64GB 或以上容量的 TF 卡**），TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡。

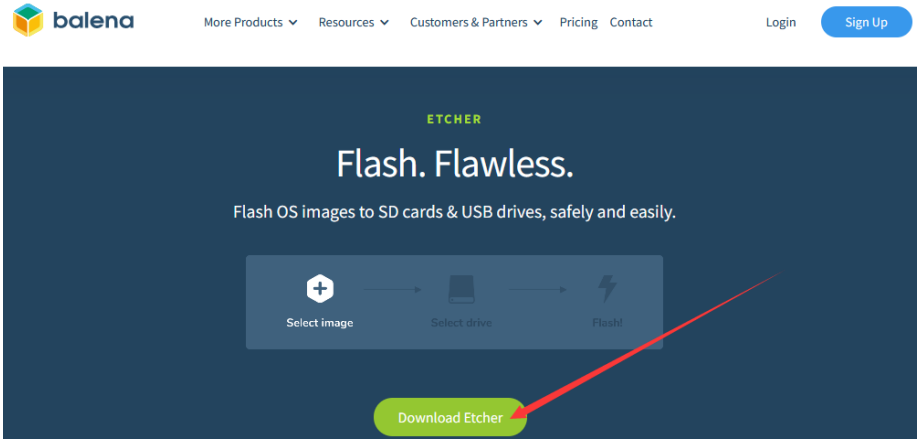
2) 然后把 TF 卡插入读卡器，再把读卡器插入电脑。

3) 从开发板的资料下载页面下载想要烧录的 Linux 镜像压缩包。

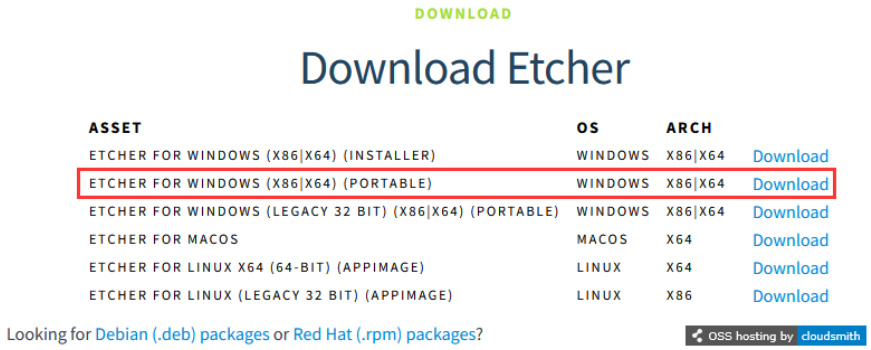
4) 然后下载用于烧录 Linux 镜像的软件——**balenaEtcher**，下载地址为：

https://www.balena.io/etcher/

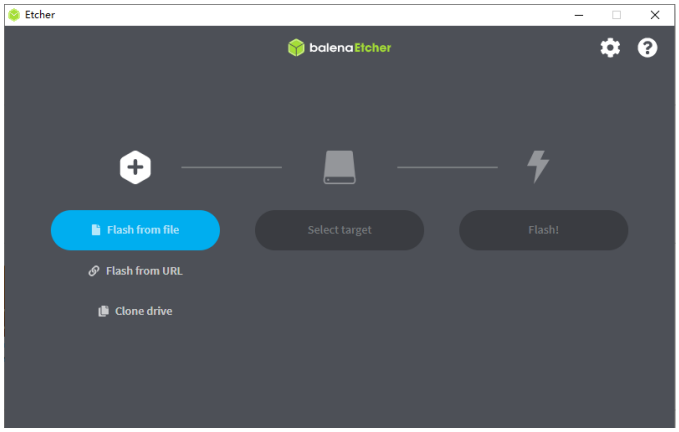
5) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载的地方。



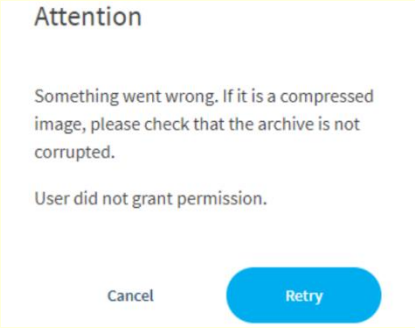
6) 然后可以选择下载 Portable 版本的 balenaEtcher，Portable 版本无需安装，双击打开就可以使用。



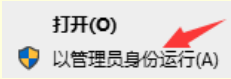
7) 打开后的 balenaEtcher 界面如下图所示：



打开 balenaEtcher 时如果提示下面的错误：

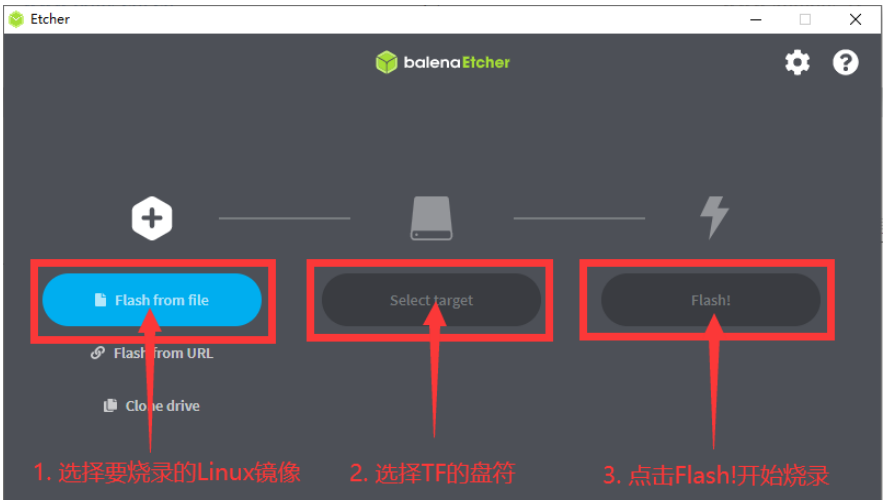


请选择 balenaEtcher 后点击右键，然后选择以管理员身份运行。

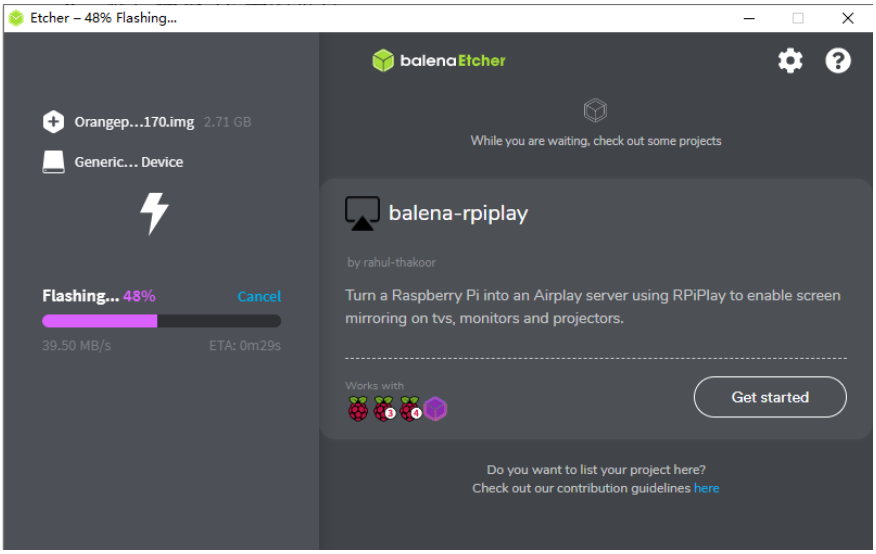


8) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示：

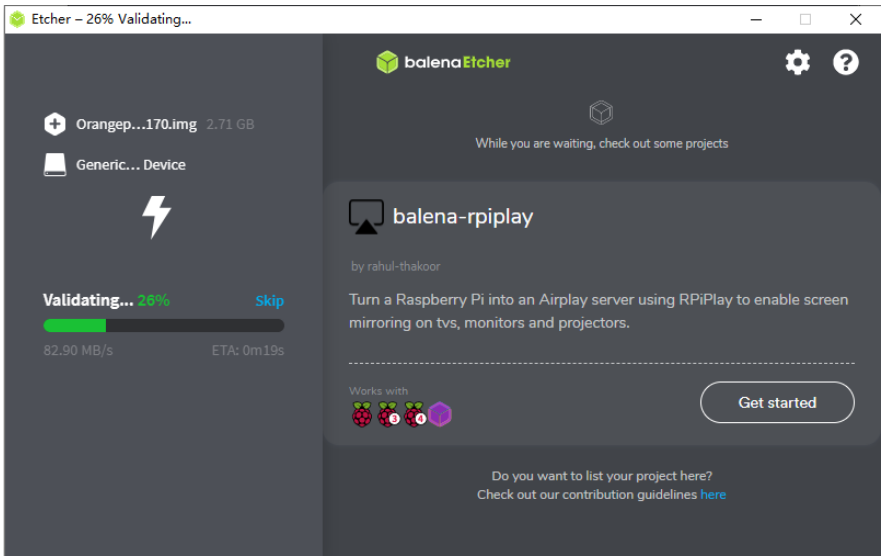
a. 首先选择要烧录的 Linux 镜像文件的路径。
b. 然后选择 TF 卡的盘符。
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中。



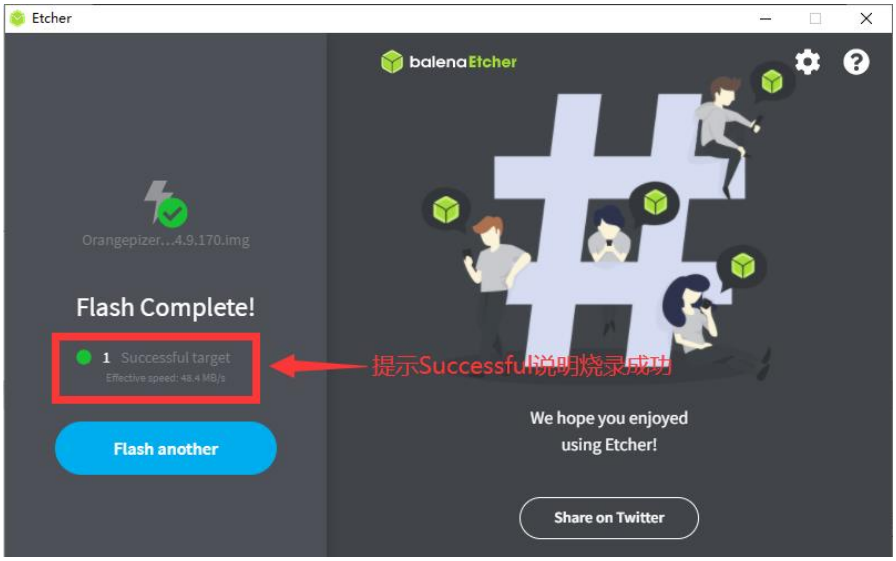
9) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中。



10) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验。



11) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了。



注意，启动系统前请确保拨码开关拨到了TF卡启动的位置了。拨码开关的使用说明请参考控制启动设备的两个拨码开关的使用说明一小节的说明。

2.4.2. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法

注意，这里说的Linux镜像具体指的是从开发板的资料下载页面下载的Ubuntu或者openEuler镜像。

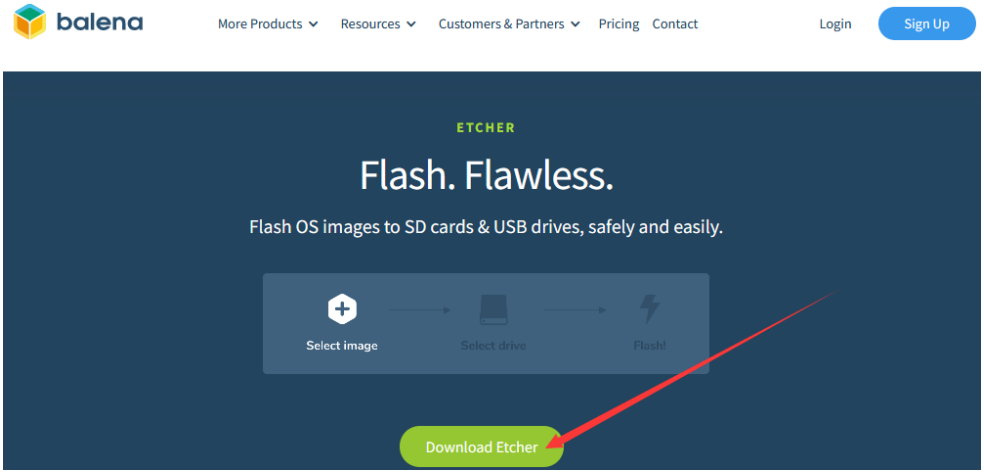
1) 首先准备一张 32GB 或更大容量的 TF 卡（**推荐使用 64GB 或以上容量的 TF 卡**），TF 卡的传输速度必须为 **class10** 级或 **class10** 级以上，建议使用闪迪等品牌的 TF 卡。

2) 然后把 TF 卡插入读卡器，再把读卡器插入电脑。

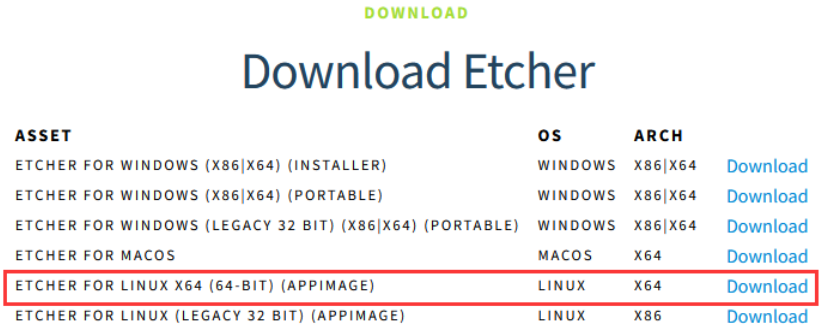
3) 下载 balenaEtcher 软件，下载地址为：

https://www.balena.io/etcher/

4) 进入 balenaEtcher 下载页面后，点击绿色的下载按钮会跳到软件下载的地方。

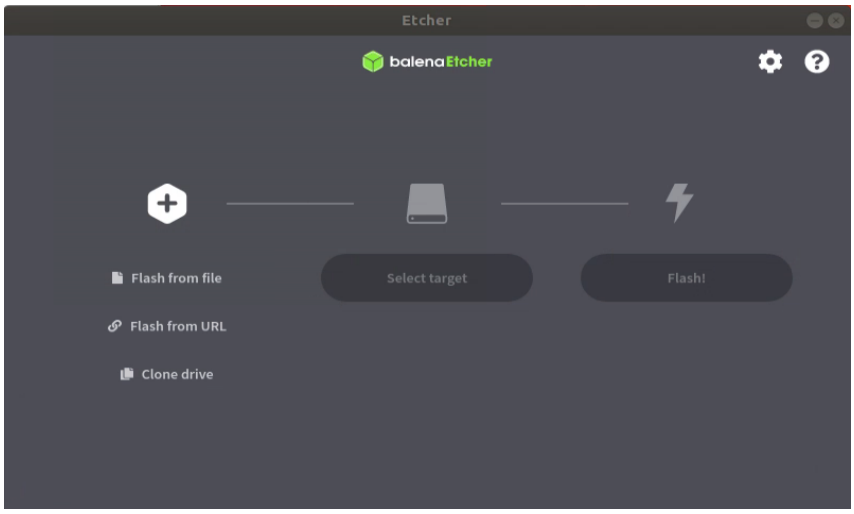


5) 然后选择下载 Linux 版本的软件即可。



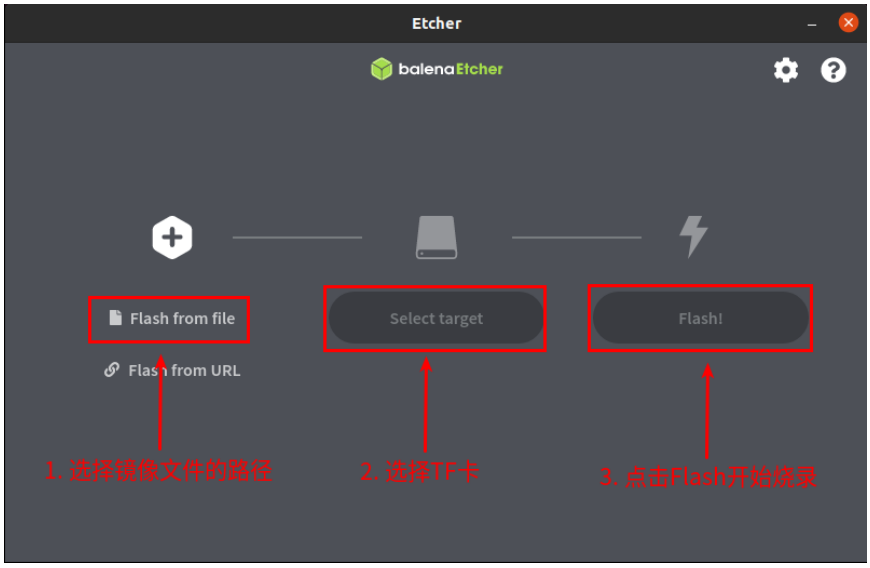
6) 从开发板的资料下载页面下载想要烧录的 Linux 镜像文件压缩包。

7) 然后在 Ubuntu PC 的图形界面双击 **balenaEtcher-x.x.x-x64.AppImage** 即可打开 balenaEtcher，balenaEtcher 打开后的界面显示如下图所示：

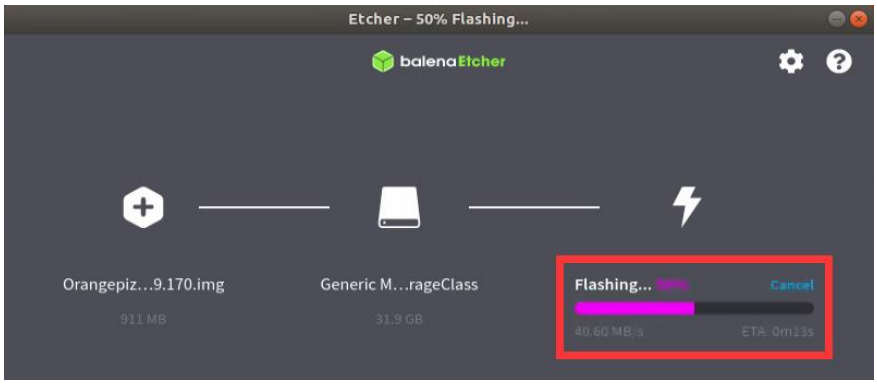


8) 使用 balenaEtcher 烧录 Linux 镜像的具体步骤如下所示：

a. 首先选择要烧录的 Linux 镜像文件的路径。
b. 然后选择 TF 卡的盘符。
c. 最后点击 Flash 就会开始烧录 Linux 镜像到 TF 卡中。



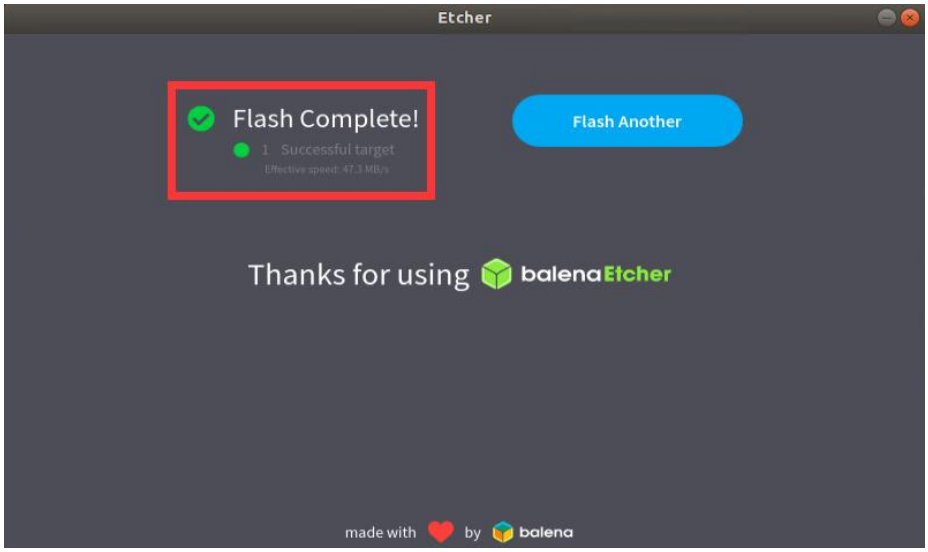
9) balenaEtcher 烧录 Linux 镜像的过程显示的界面如下图所示，另外进度条显示紫色表示正在烧录 Linux 镜像到 TF 卡中。



10) Linux 镜像烧录完后，balenaEtcher 默认还会对烧录到 TF 卡中的镜像进行校验，确保烧录过程没有出问题。如下图所示，显示绿色的进度条就表示镜像已经烧录完成，balenaEtcher 正在对烧录完成的镜像进行校验。



11) 成功烧录完成后 balenaEtcher 的显示界面如下图所示，如果显示绿色的指示图标说明镜像烧录成功，此时就可以退出 balenaEtcher，然后拔出 TF 卡插入到开发板的 TF 卡槽中使用了。

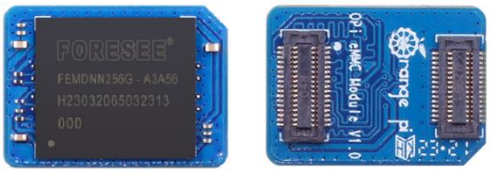


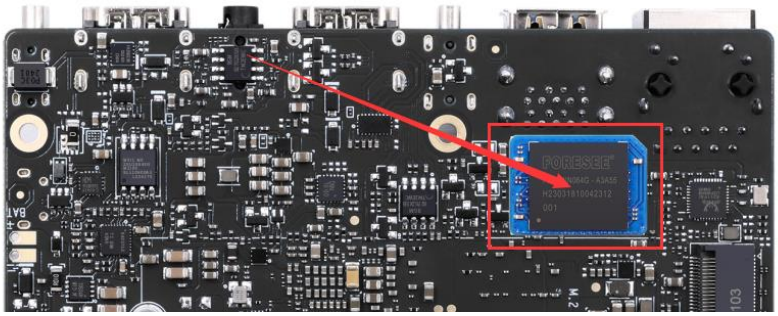
注意，启动系统前请确保拨码开关拨到了TF卡启动的位置了。拨码开关的使用说明请参考控制启动设备的两个拨码开关的使用说明一小节的说明。

2.5. 烧写 Linux 镜像到 eMMC 中的方法

注意，这里说的Linux镜像具体指的是从开发板的资料下载页面下载的openEuler镜像。

1) 开发板预留了 eMMC 模块的扩展接口，烧录系统到 eMMC 前，需要先购买一个与开发板 eMMC 接口相匹配的 eMMC 模块。然后将 eMMC 模块安装到开发板上。配套的 eMMC 模块和插入开发板的方法如下所示：





2) 烧录 Linux 镜像到 eMMC 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux 镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法**两小节的说明。

3) 启动进入 TF 卡的 Linux 系统后，请先确认下 eMMC 模块已经被开发板的 Linux 系统正常识别了。如果 eMMC 模块正常识别了的话，在 root 用户下使用 **fdisk -l** 命令就能看到 eMMC 模块的容量信息。

```
[root@openEuler ~]# fdisk -l
......
Disk /dev/mmcblk0: 28.91 GiB, 31037849600 bytes, 60620800 sectors
......
```

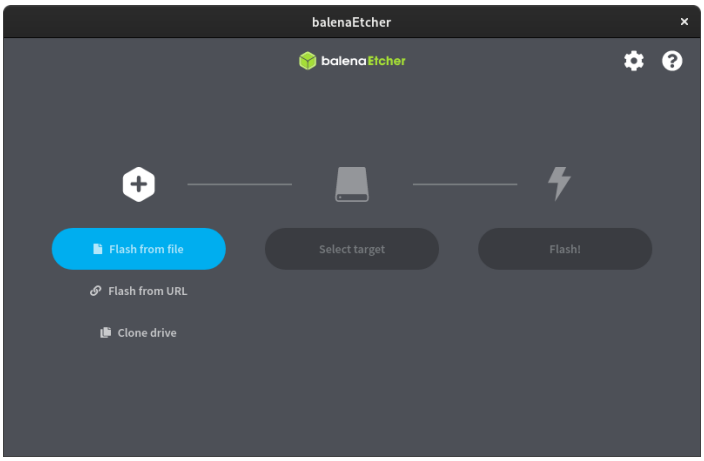
4) 然后将要烧录的 Linux 镜像文件压缩包上传到 TF 卡的 Linux 系统中。

注意，使用xz格式压缩的Linux镜像压缩包不需要解压，balenaEtcher烧录时会自动解压。

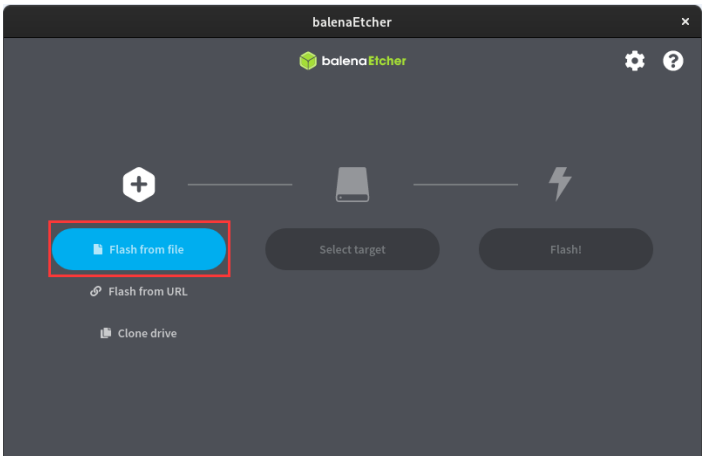
5) 然后就可以开始使用 balenaEtcher 软件烧录镜像到 eMMC 中了。Linux 系统中已经预装了 balenaEtcher，打开方法如下所示：



6) balenaEtcher 打开后的界面如下所示：

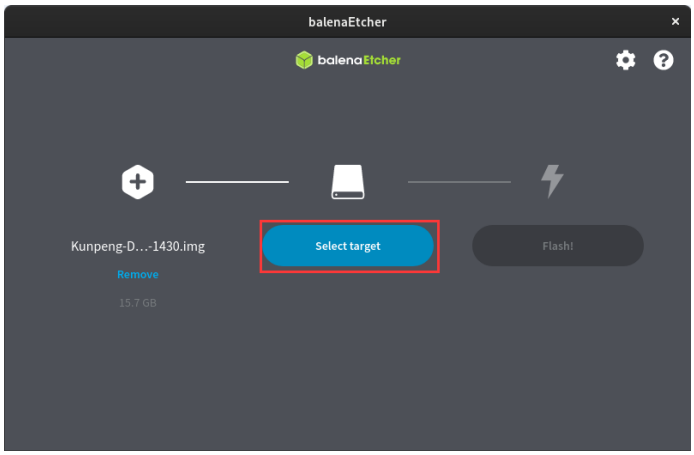


7) 然后点击 **Flash from file** 选择前面上传的想要烧录的镜像文件。

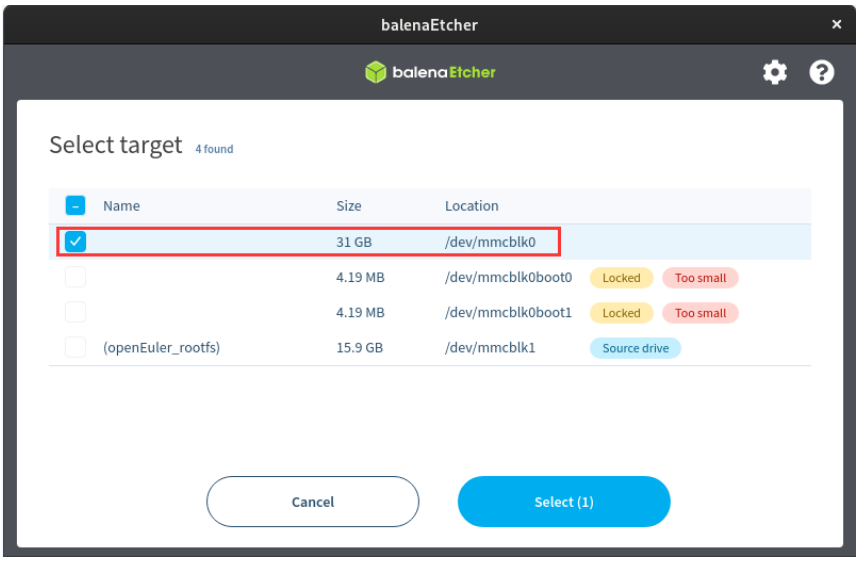


如果打开Linux镜像文件时提示没有权限，请使用sudo chmod 777 镜像文件名这条命令来给镜像文件添加权限。

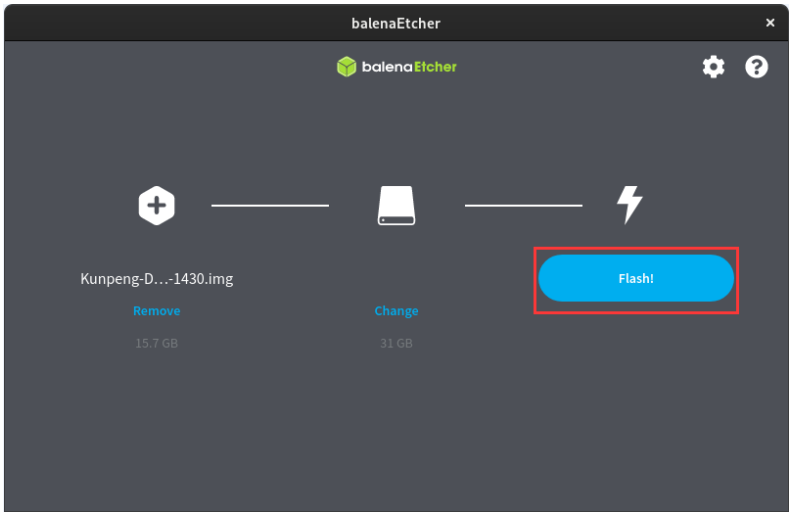
8) 然后点击 **Select target**。



9) 然后选择 eMMC 对应的**/dev/mmcblk0** 选项，再点击 **Select** 即可。



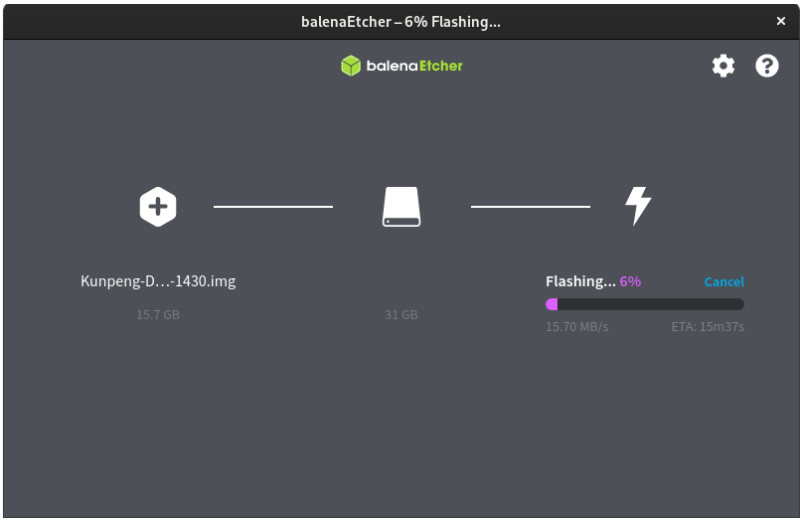
10) 然后点击 **Flash!**开始烧录。



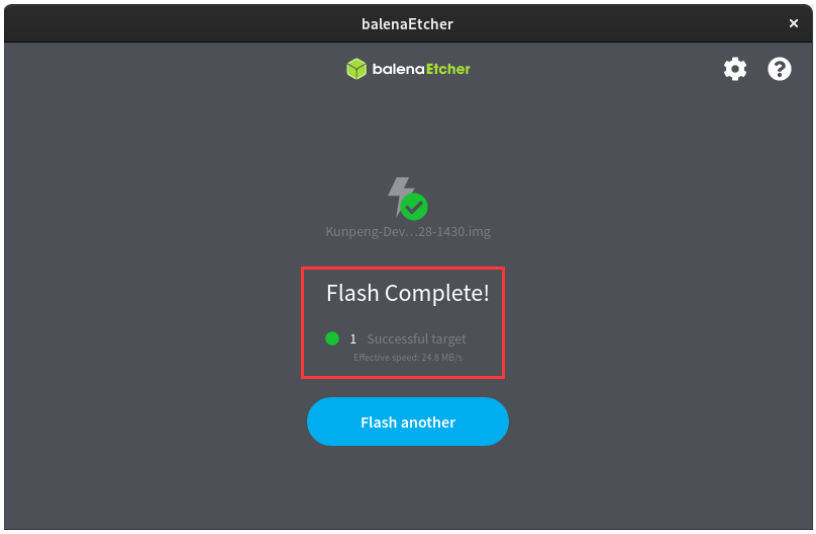
11) 然后输入 Linux 系统的密码：**openEuler**。



12) 然后就会真正开始烧录 Linux 镜像到 eMMC 中了。



13) Linux 镜像烧录完后的显示如下所示：



14) 此时就可以关闭掉 Linux 系统，然后拔出 TF 卡，并断开 Type-C 电源。再将两个拨码开关拨到 eMMC 启动对应的位置，然后重新插入 Type-C 电源就可以启动 eMMC 中的 Linux 系统了。

注意，启动系统前请确保拨码开关拨到eMMC启动的位置了。拨码开关的使用说明请参考控制启动设备的两个拨码开关的使用说明一小节的说明。

2.6. 烧写 Linux 镜像到 NVMe SSD 中的方法

注意，这里说的Linux镜像具体指的是从开发板的资料下载页面下载的Ubuntu

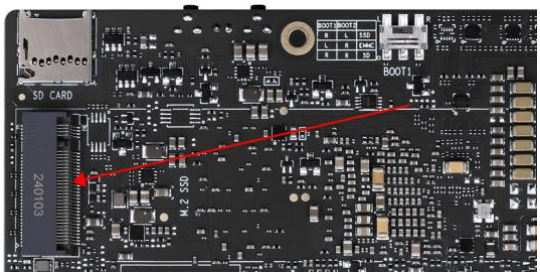
或者openEuler镜像。

注意，NVMe SSD目前测试了樊想、金士顿和三星的，只有三星的NVMe SSD能稳定运行Linux系统。非三星品牌的NVMe SSD需要等后续软件更新才能正常使用。

1) 首先需要准备一个 2280 规格 NVMe SSD，开发板 M.2 插槽支持的 PCIe 规格为 PCIe3.0x4。PCIe3.0 和 PCIe4.0 的 NVMe SSD 都是可以用的，只是 PCIe4.0 SSD 速度最高只有 PCIe3.0x4 的速度。2242 等其他规格的 SSD 也都是可以使用的，只是没法固定。



2) 然后把 NVMe SSD 插入开发板的 M.2 接口中，并固定好。



3) 烧录 Linux 镜像到 NVMe SSD 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux 镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法**两小节的说明。

4) 启动进入 TF 卡的 Linux 系统后，请先确认下 NVMe SSD 已经被开发板的 Linux 系统正常识别了。如果 NVMe SSD 正常识别了的话，使用 **sudo fdisk -l** 命令就能看到 **nvme** 相关的信息。

```
[openEuler@openEuler ~]$ sudo fdisk -l | grep "nvme0n1"
Disk /dev/nvme0n1: 238.47 GiB, 256060514304 bytes, 500118192 sectors
.....
```

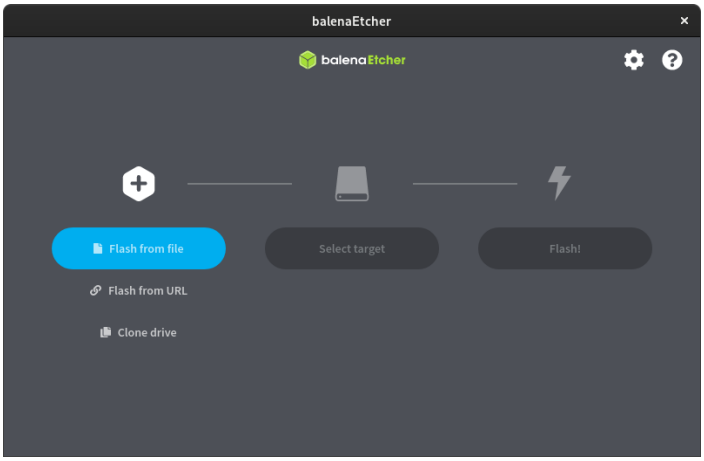
5) 然后将要烧录的 Linux 镜像文件压缩包上传到 TF 卡的 Linux 系统中。

注意，使用xz格式压缩的Linux镜像压缩包不需要解压，balenaEtcher烧录时会自动解压。

6) 然后就可以开始使用 balenaEtcher 软件烧录镜像到 SSD 中了。Linux 系统中已经预装了 balenaEtcher，打开方法如下所示：

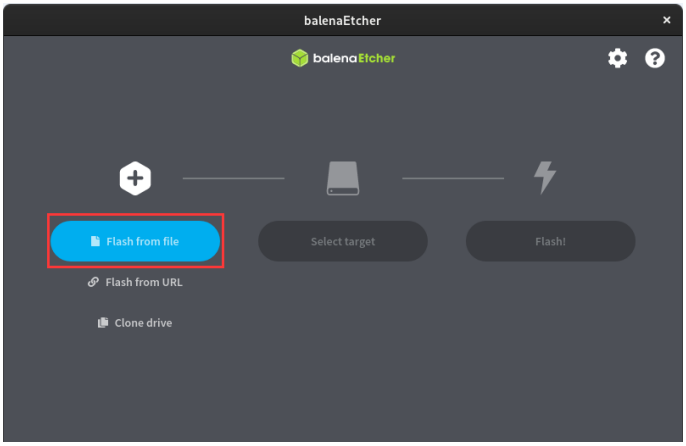


7) balenaEtcher 打开后的界面如下所示：

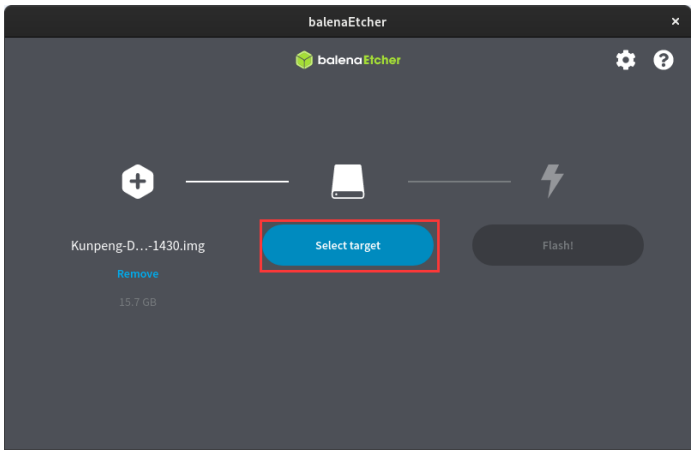


如果打开Linux镜像文件时提示没有权限，请使用sudo chmod 777 镜像文件名这条命令来给镜像文件添加权限。

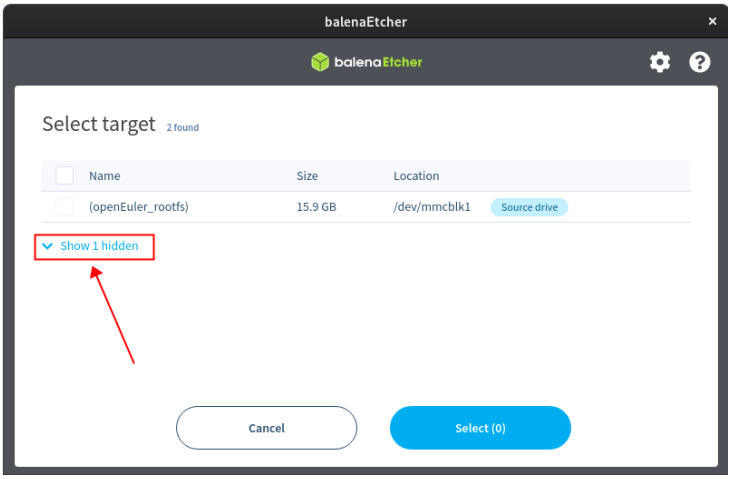
8) 然后点击 **Flash from file** 选择前面上传的想要烧录的镜像文件。



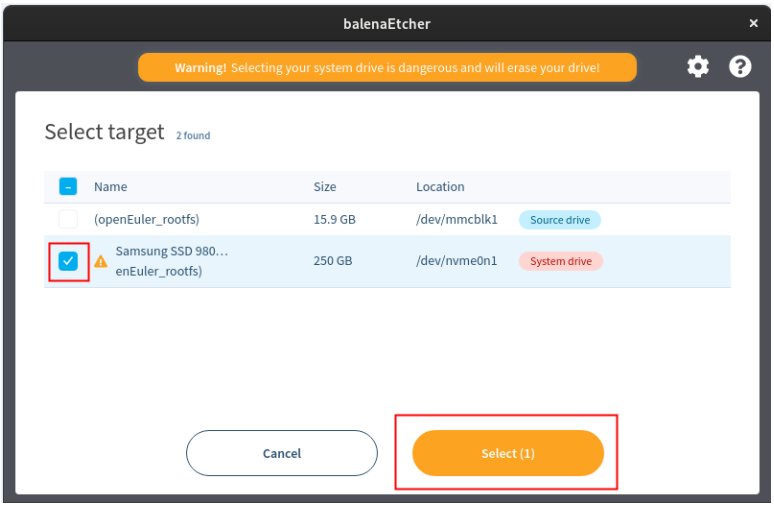
9) 然后点击 **Select target**。



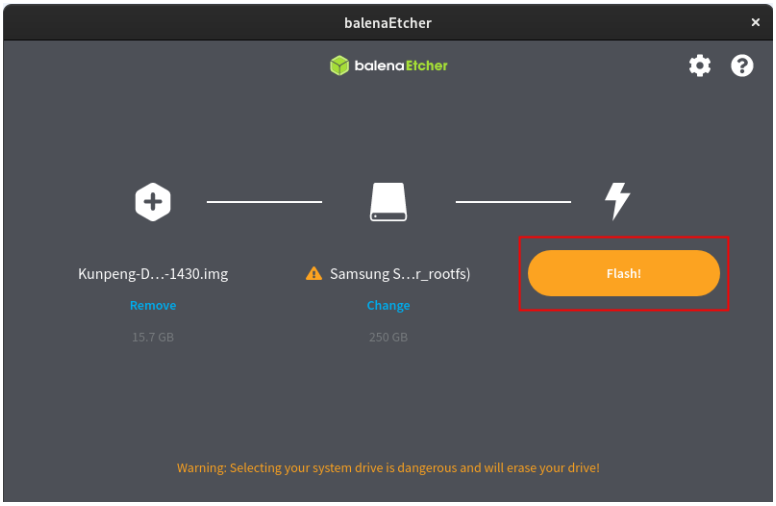
10) 然后点击 **Show 1 hidden**。



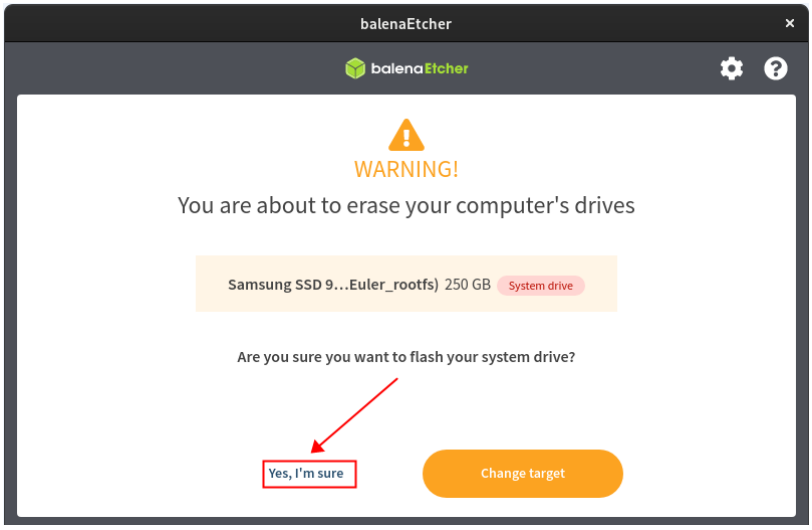
11) 然后选择 SSD 对应的选项，再点击 **Select** 即可。



12) 然后点击 **Flash!**开始烧录。



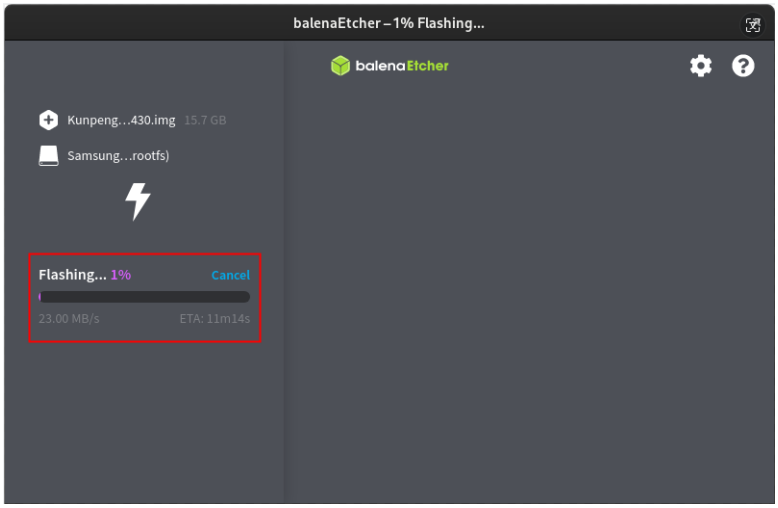
13) 然后选择 **Yes, I'm sure**。



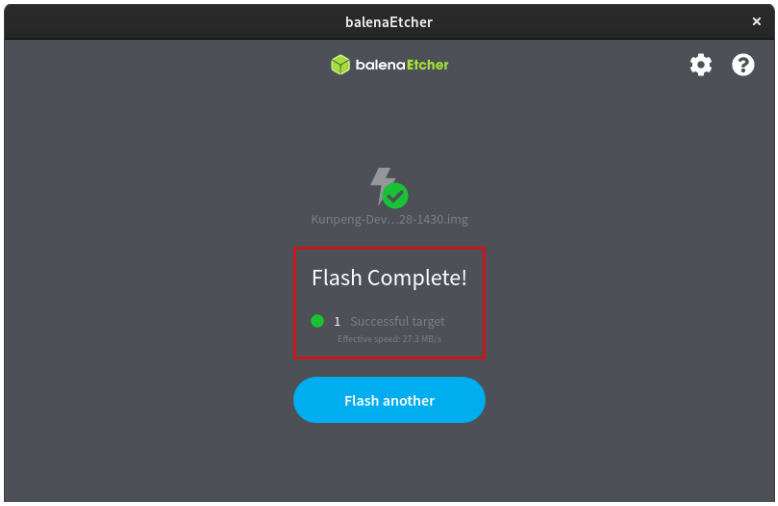
14) 然后输入 Linux 系统的密码：**openEuler**。



15) 然后就会真正开始烧录 Linux 镜像到 SSD 中了。



16) Linux 镜像烧录完后的显示如下所示：



17) 此时就可以关闭掉 Linux 系统，然后拔出 TF 卡，并断开 Type-C 电源。再将两个拨码开关拨到 SSD 启动对应的位置，然后重新插入 Type-C 电源就可以启动 SSD 中的 Linux 系统了。

注意，启动系统前请确保拨码开关拨到了SSD启动的位置了。拨码开关的使用说明请参考控制启动设备的两个拨码开关的使用说明一小节的说明。

2.7. 烧写 Linux 镜像到 SATA SSD 中的方法

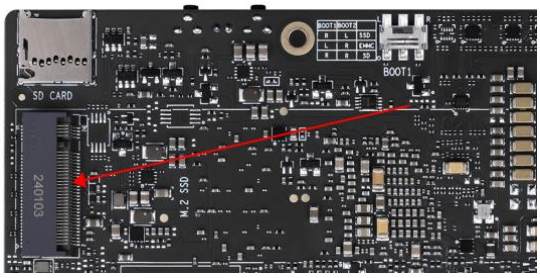
注意，这里说的Linux镜像具体指的是从开发板的资料下载页面下载的Ubuntu

或者openEuler镜像。

1) 首先需要准备一个 M.2 2280 规格的 SATA SSD。2242 等其他规格的 SSD 也都是可以使用的，只是没法固定。



2) 然后把 SATA SSD 插入开发板的 M.2 接口中，并固定好。



3) 烧录 Linux 镜像到 SATA SSD 中需要借助 TF 卡来完成，所以首先需要将 Linux 镜像烧录到 TF 卡上，然后使用 TF 卡启动开发板进入 Linux 系统。烧录 Linux 镜像到 TF 卡的方法请见**基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法**和**基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法**两小节的说明。

4) 启动进入 TF 卡的 Linux 系统后，首先需要更新下 SATA 对应的 dt.img 文件。步骤如下所示：

a. 首先进入**/opt/opi_test/sata** 文件夹。

```
[openEuler@openEuler ~]$ cd /opt/opi_test/sata
```

b. 然后运行下 **update.sh** 脚本来更新 SATA 对应的 dt.img。

```
[openEuler@openEuler sata]$ sudo ./update.sh
```

c. 运行完 **update.sh** 脚本后会自动重启 Linux 系统让配置生效。

d. 一切顺利的话，重新进入 TF 卡的 Linux 系统后就能识别到 SATA SSD 了。

```
[openEuler@openEuler ~]$ sudo fdisk -l | grep "/dev/sd"
Disk /dev/sda: 238.47 GiB, 256060514304 bytes, 500118192 sectors
.....
```

开发板的M.2 接口既支持NVMe SSD也支持SATA SSD，只能二选一。Linux系统

中dt.img的配置默认打开的是PCIe的配置，如果M.2 接口接的是SATA SSD，需要手动切换下SATA对应的dt.img才能正常使用。

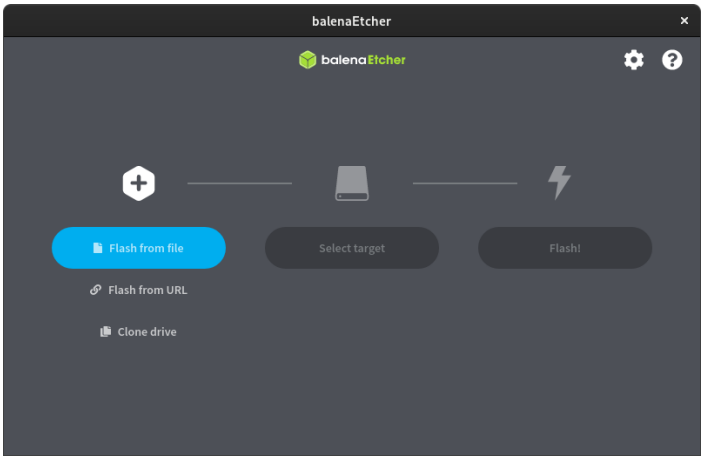
5) 然后将要烧录的 Linux 镜像文件压缩包上传到 TF 卡的 Linux 系统中。

注意，使用xz格式压缩的Linux镜像压缩包不需要解压，balenaEtcher烧录时会自动解压。

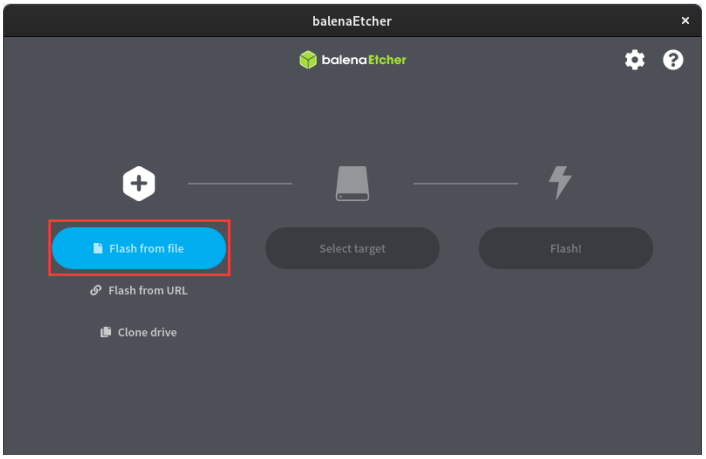
6) 然后就可以开始使用 balenaEtcher 软件烧录镜像到 SSD 中了。Linux 系统中已经预装了 balenaEtcher，打开方法如下所示：



7) balenaEtcher 打开后的界面如下所示：

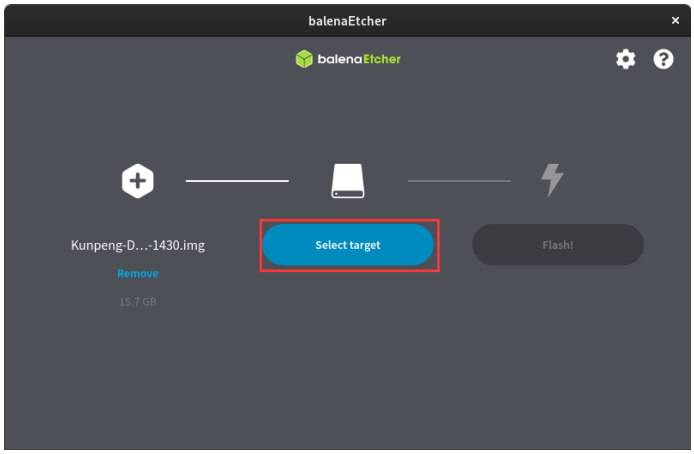


8) 然后点击 **Flash from file** 选择前面上传的想要烧录的镜像文件。

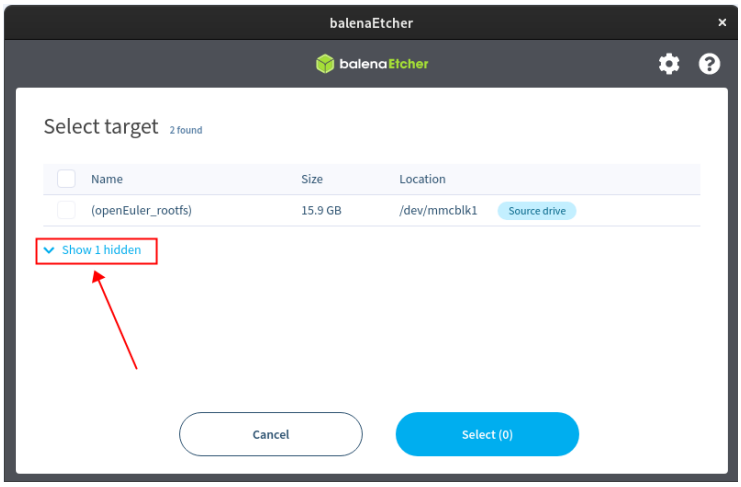


如果打开Linux镜像文件时提示没有权限，请使用sudo chmod 777 镜像文件名这条命令来给镜像文件添加权限。

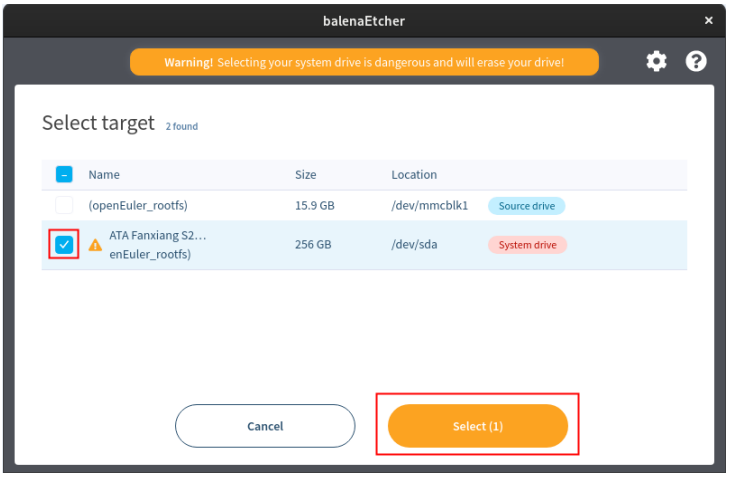
9) 然后点击 **Select target**。



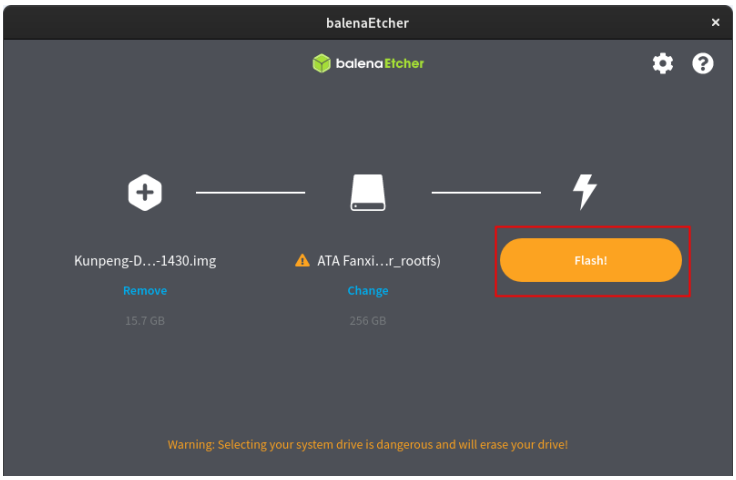
10) 然后点击 **Show 1 hidden**。



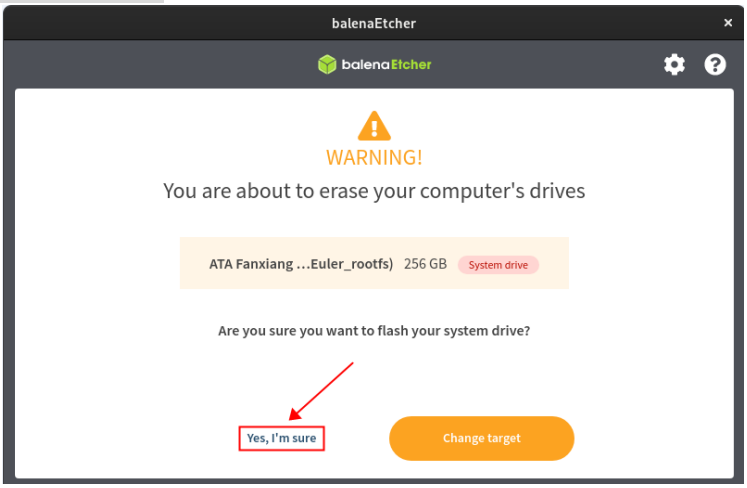
11) 然后选择 SSD 对应的选项，再点击 **Select** 即可。



12) 然后点击 **Flash!** 开始烧录。



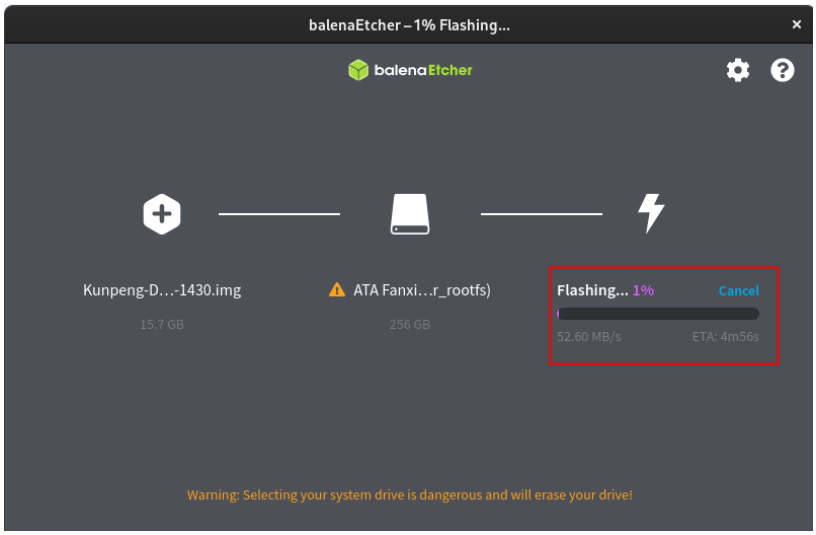
13) 然后选择 **Yes, I'm sure**。



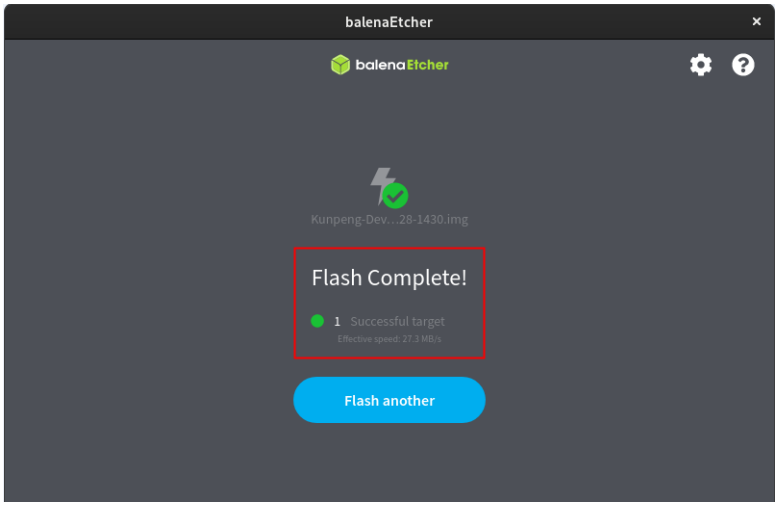
14) 然后输入 Linux 系统的密码：**openEuler**。



15) 然后就会真正开始烧录 Linux 镜像到 SSD 中了。



16) Linux 镜像烧录完后的显示如下所示：



17) 烧录完成后，还需要将 SATA 版本的 dt.img 烧录到 SATA SSD 中，因为提供的镜像默认打开的都是 PCIe 的配置。具体命令如下所示：

```
sudo dd if=/opt/opi_test/dt_img/dt_drm_sata.img of=/dev/sda count=4096 seek=114688 bs=512
```

注意，上面的命令中，“of=”参数后面的/dev/sda为SSD对应的设备节点名。请根据实际情况进行修改。

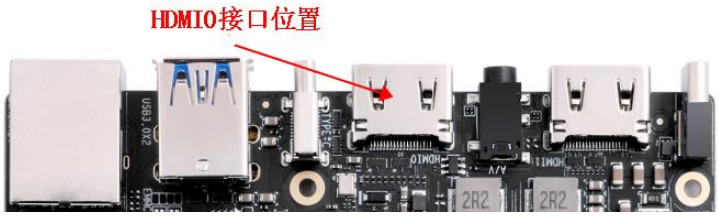
18) 此时就可以关闭掉 Linux 系统，然后拔出 TF 卡，并断开 Type-C 电源。再将两个拨码开关拨到 SSD 启动对应的位置，然后重新插入 Type-C 电源就可以启动 SSD 中的 Linux 系统了。

注意，启动系统前请确保拨码开关拨到了SSD启动的位置了。拨码开关的使用说明请参考控制启动设备的两个拨码开关的使用说明一小节的说明。

2.8. 启动开发板的步骤

1) 将烧录好镜像的 TF 卡或者 eMMC 模块或者 SSD 插入开发板对应的插槽中。

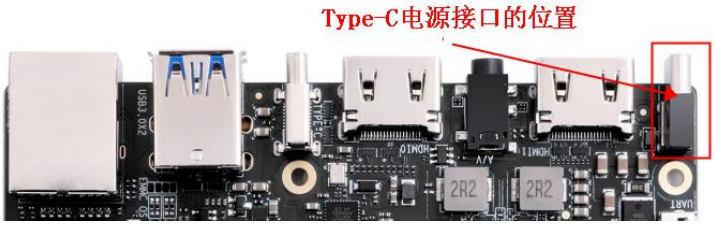
2) 开发板有两个 HDMI 接口（**目前只有 HDMI0 支持显示 Linux 系统的桌面，HDMI1 显示 Linux 系统桌面的功能还需等软件更新**），如果想显示 Linux 系统的桌面，可以将开发板的 HDMI0 接口连接到 HDMI 显示器。



3) 开发板有 USB 接口，可以接上 USB 鼠标和键盘，来控制开发板。

4) 开发板有千兆以太网口，可以插入网线用来上网。

5) 然后需要连接一个 20V PD-65W 的 Type C 接口的电源，电源接口的位置如下图所示：



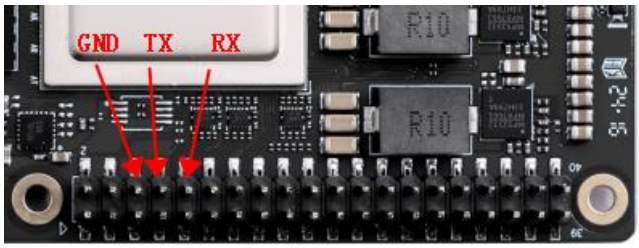
6) 然后打开电源适配器的开关，如果一切正常，等待一段时间后，HDMI 显示器就能看到 Linux 系统的登录界面了。

7) 如果想通过调试串口查看系统的输出信息，请使用串口线将开发板连接到电脑，串口的连接方法请参看调试串口的使用方法一节。

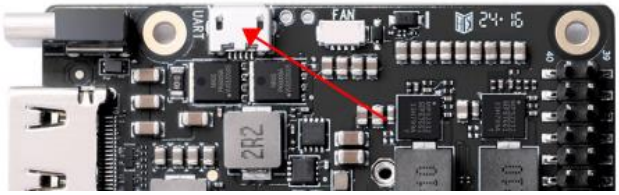
2.9. 调试串口的使用方法

开发板默认使用 uart0 做为调试串口。需要注意的是，uart0 的 tx 和 rx 引脚同时接到了两个地方，所以有两种使用调试串口的方法：

1) uart0 的 tx 和 rx 引脚接到了 40 pin 扩展接口中的 8 号和 10 号引脚，此种方式需要准备一个 **3.3v** 的 USB 转 TTL 模块和相应的杜邦线，然后才能正常使用开发板的调试串口功能。



2) uart0 的 tx 和 rx 引脚又接到了开发板的 CH343P 芯片上，再通过 CH343P 芯片引出到 Micro USB 接口上。此种方式只需要一根 Micro USB 接口的数据线将开发板连接到电脑的 USB 接口就可以开始使用开发板的调试串口功能了，无需购买 USB 转 TTL 模块。这种方法是推荐的方法。



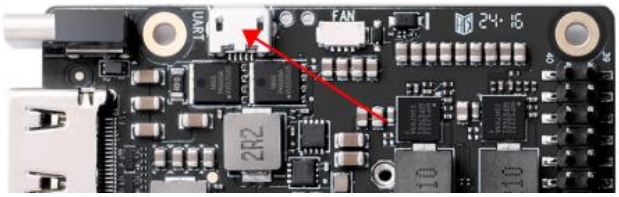
另外请注意，上面的两种方法只能二选一，请不要同时使用。

2.9.1. 通过 Micro USB 接口来使用调试串口的连接说明

1) 首先需要准备一根 Micro USB 接口的数据线



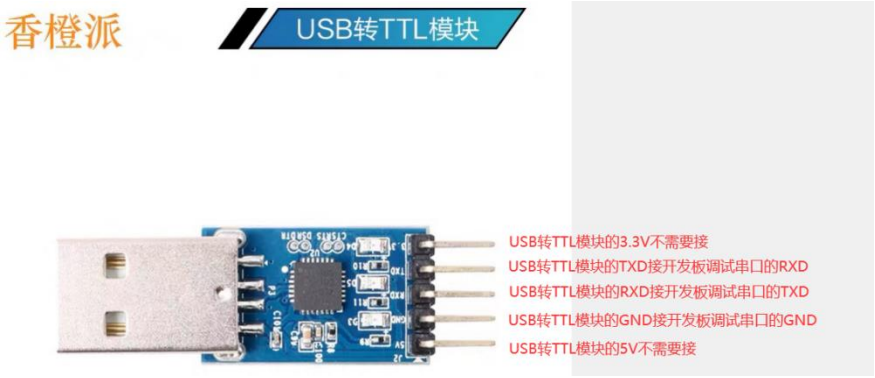
2) 然后将 Micro USB 接口一端插入开发板的 Micro USB 接口中。



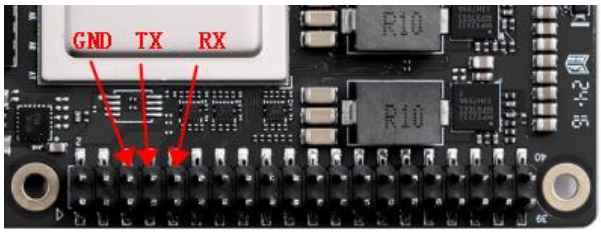
3) 再将数据线的另一端插入电脑的 USB 接口中即可。

2.9.2. 通过 40 pin 接口中的 uart0 来使用调试串口的连接说明

1) 首先需要准备一个 **3.3v** 的 USB 转 TTL 模块，然后将 USB 转 TTL 模块的 USB 接口一端插入到电脑的 USB 接口中。



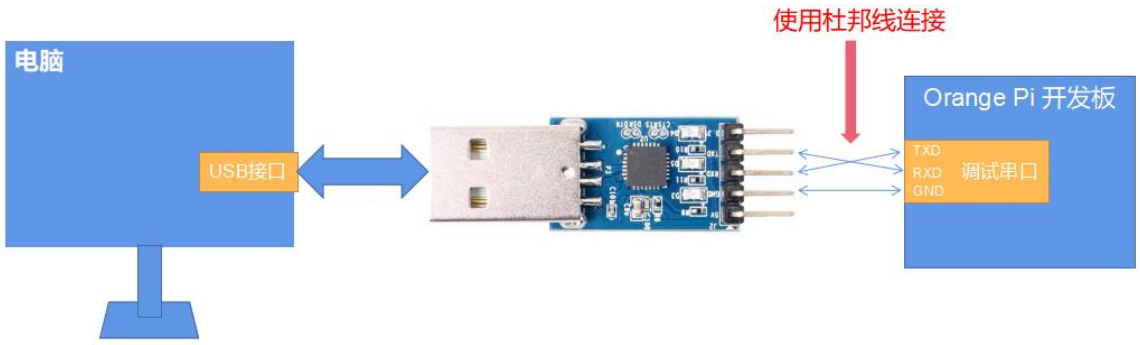
2) 开发板的调试串口 GND、TX 和 RX 引脚的对应关系如下图所示：



3) USB 转 TTL 模块 GND、TX 和 RX 引脚需要通过杜邦线连接到开发板的调试串口上。

a. USB 转 TTL 模块的 GND 接到开发板的 GND 上。
b. USB 转 TTL 模块的 **RX 接到开发板的 TX 上**。
c. USB 转 TTL 模块的 **TX 接到开发板的 RX 上**。

4) USB 转 TTL 模块连接电脑和开发板的示意图如下所示：



USB转TTL模块连接电脑和 Orange Pi 开发板的示意图

串口的 TX 和 RX 是需要交叉连接的，如果不想仔细区分 TX 和 RX 的顺序，可以把串口的 TX 和 RX 先随便接上，如果测试串口没有输出再交换下 TX 和 RX 的顺序，这样就总有一种顺序是对的。

2.9.3. Ubuntu 平台调试串口的使用方法

Linux 下可以使用的串口调试软件有很多，如 putty、minicom 等，下面演示下 putty 的使用方法。

1) 首先请按照**通过 40 pin 接口中的 uart0 来使用调试串口的连接说明**或**通过 Micro USB 接口来使用调试串口的连接说明**一小节的说明（两种方法请根据自己的情况二选一）将开发板和电脑连接起来，如果串口模块识别正常的话，在 Ubuntu PC 的 **/dev** 下就可以看到对应的设备名，请记住这个设备名，后面设置串口软件时会用到。

a. 如果使用 40 pin 接口中的 uart0 显示的设备名一般为**/dev/ttyUSB0**。

```
test@test:~$ ls /dev/ttyUSB*
/dev/ttyUSB0
```

b. 如果使用 Micro USB 接口显示的设备名一般为**/dev/ttyACM0**。

```
test@test:~$ ls /dev/ttyACM*
/dev/ttyACM0
```

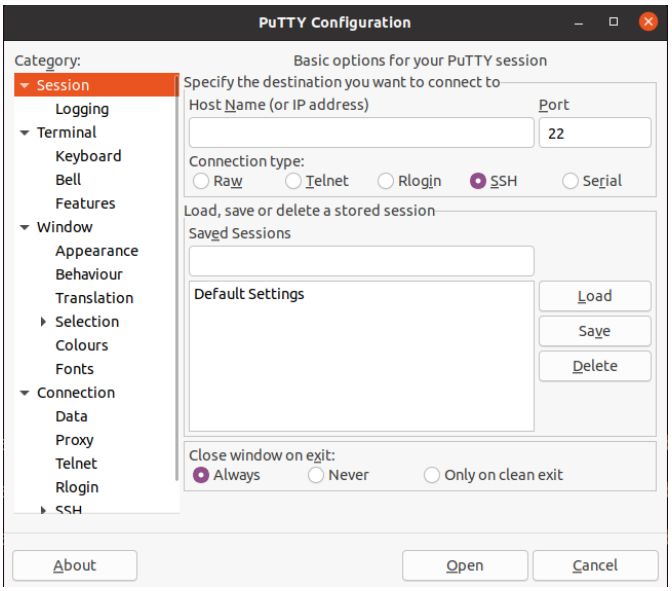
2) 然后使用下面的命令在 Ubuntu PC 上安装下 putty。

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install -y putty
```

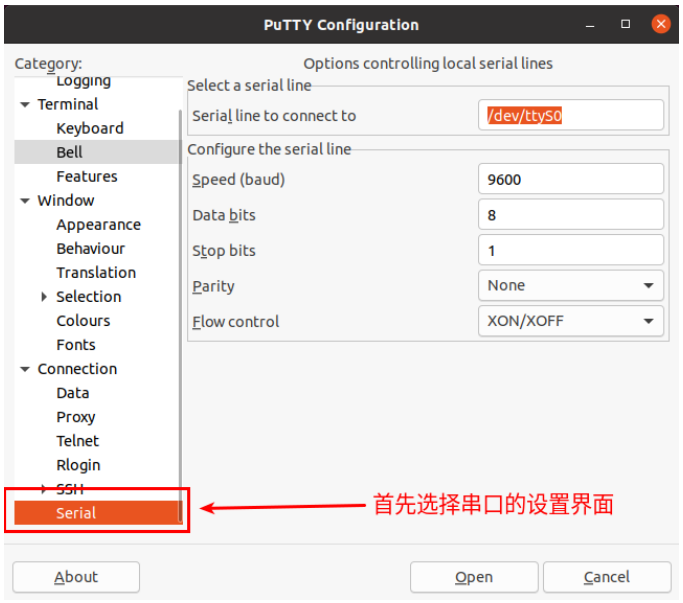
3) 然后运行 putty，**记得加 sudo 权限**。

```
test@test:~$ sudo putty
```

4) 执行 putty 命令后会弹出下面的界面。

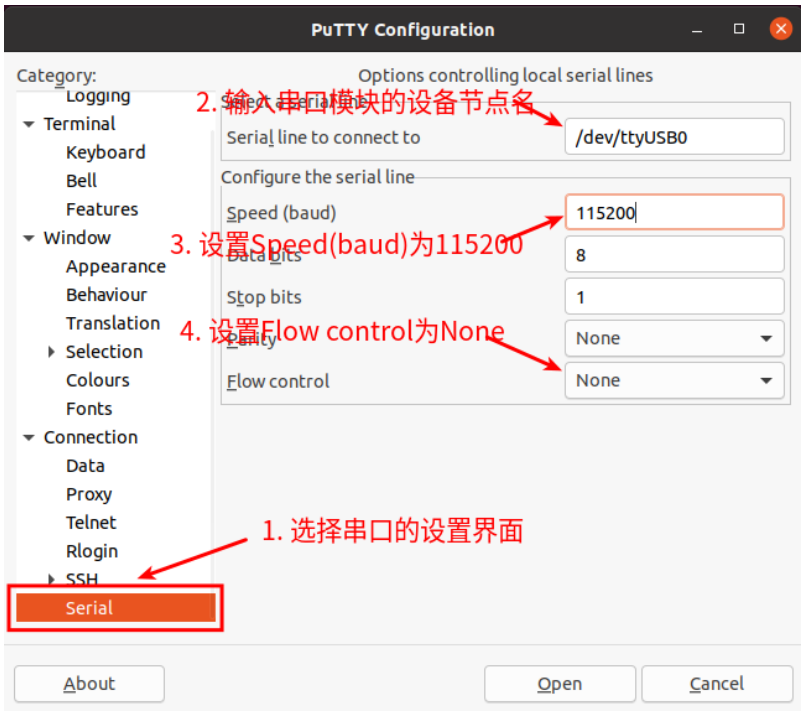


5) 首先选择串口的设置界面。



6) 然后设置串口的参数。

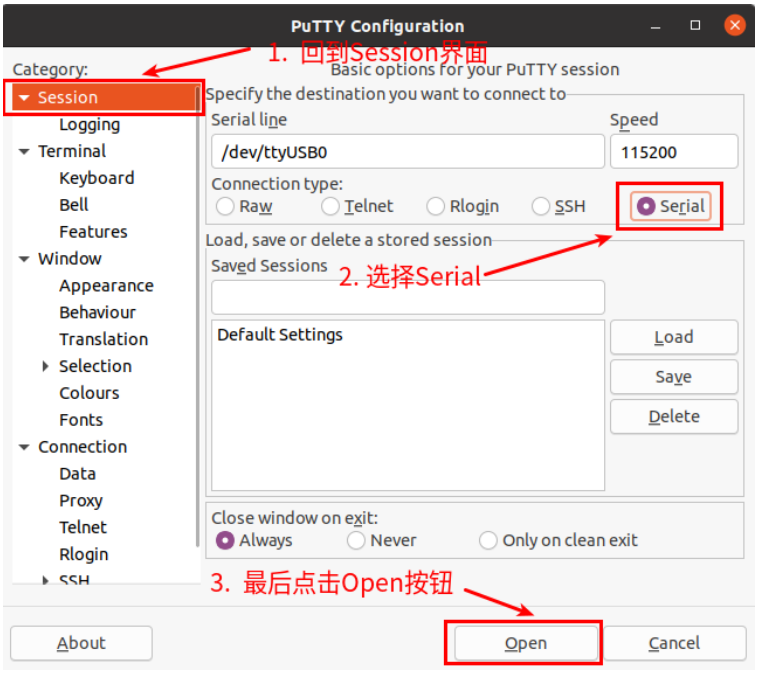
a. 设置 **Serial line to connect to** 为**/dev/ttyUSB0** 或者**/dev/ttyACM0**（请根据实际情况进行修改）。
b. 设置 **Speed(baud)**为 **115200**（串口的波特率）。
c. 设置 **Flow control** 为 **None**。



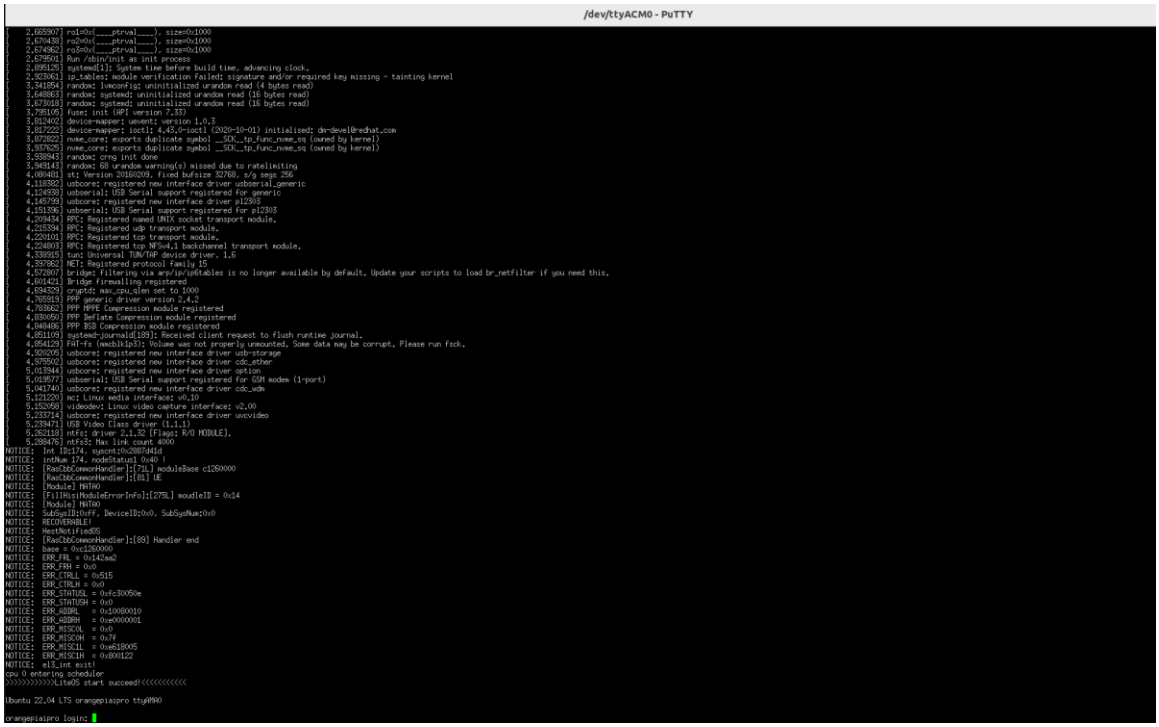
7) 在串口的设置界面设置完后，再回到 Session 界面。

a. 首先选择 **Connection type** 为 **Serial**。

b. 然后点击 **Open** 按钮连接串口。



8) 然后启动开发板，就能从打开的串口终端中看到系统输出的 Log 信息了。



9) 当看到登录界面时，就可以使用下面的账号和密码来登录 Linux 系统了。

账号	密码
root	**openEuler**
openEuler	**openEuler**

2.9.4. Windows 平台调试串口的使用方法

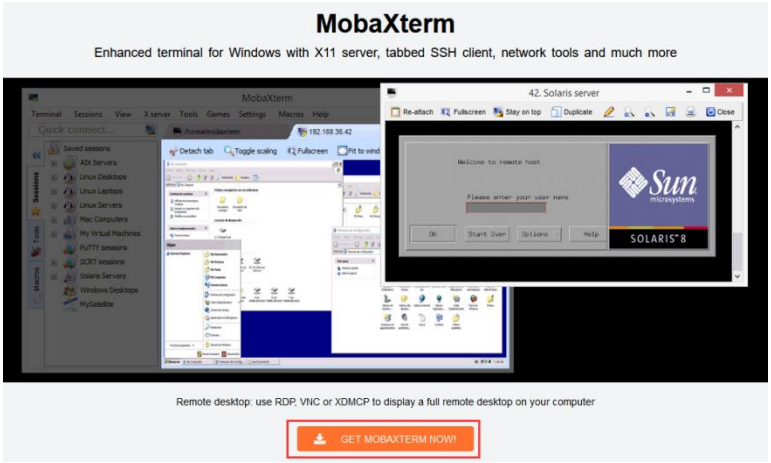
Windows 下可以使用的串口调试软件有很多，如 SecureCRT、MobaXterm 等，下面演示 MobaXterm 的使用方法，这款软件有免费版本，无需购买序列号即可使用。

1) 首先下载 MobaXterm。

a. 下载 MobaXterm 网址如下：

https://mobaxterm.mobatek.net/

b. 进入 MobaXterm 下载网页后点击 **GET XOBATERM NOW!**。



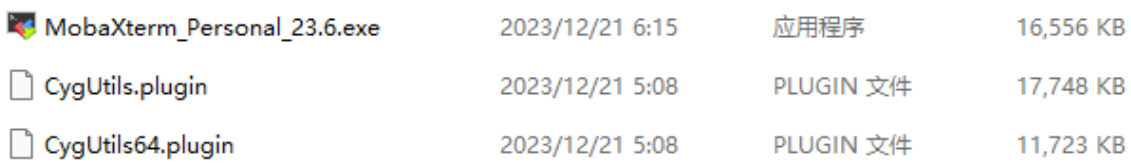
c. 然后选择下载 Home 版本。



d. 然后选择 Portable 便携式版本，下载完后无需安装，直接打开就可以使用。



2) 下载完后使用解压缩软件解压下载的压缩包，即可得到 MobaXterm 的可执软件，然后双击打开。



3) 打开软件后，设置串口连接的步骤如下：

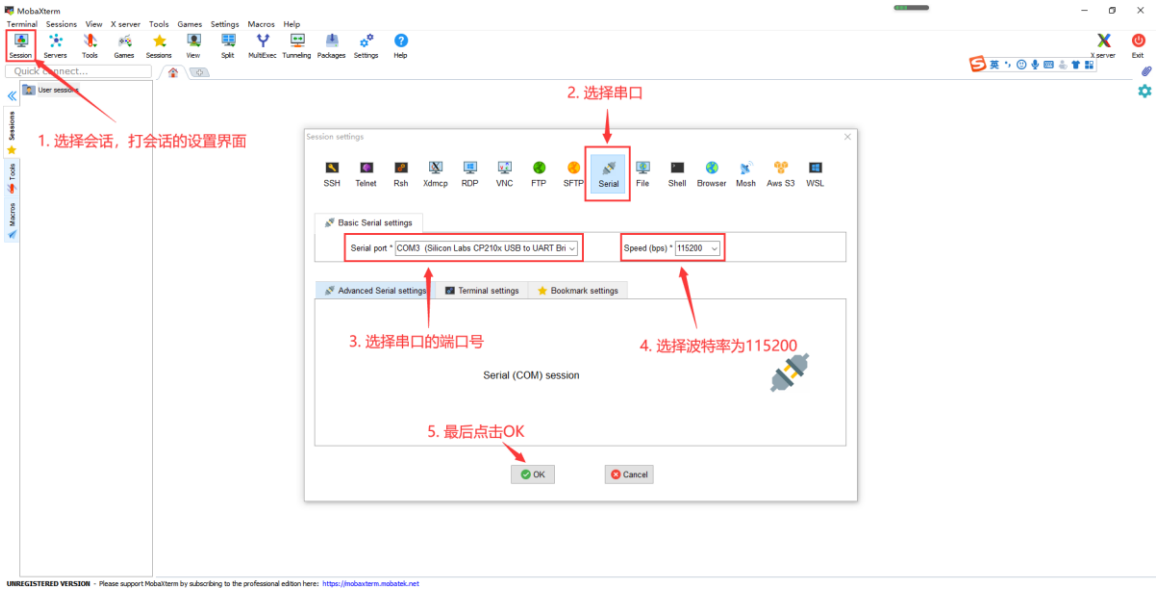
a. 打开会话的设置界面。

b. 选择串口类型。

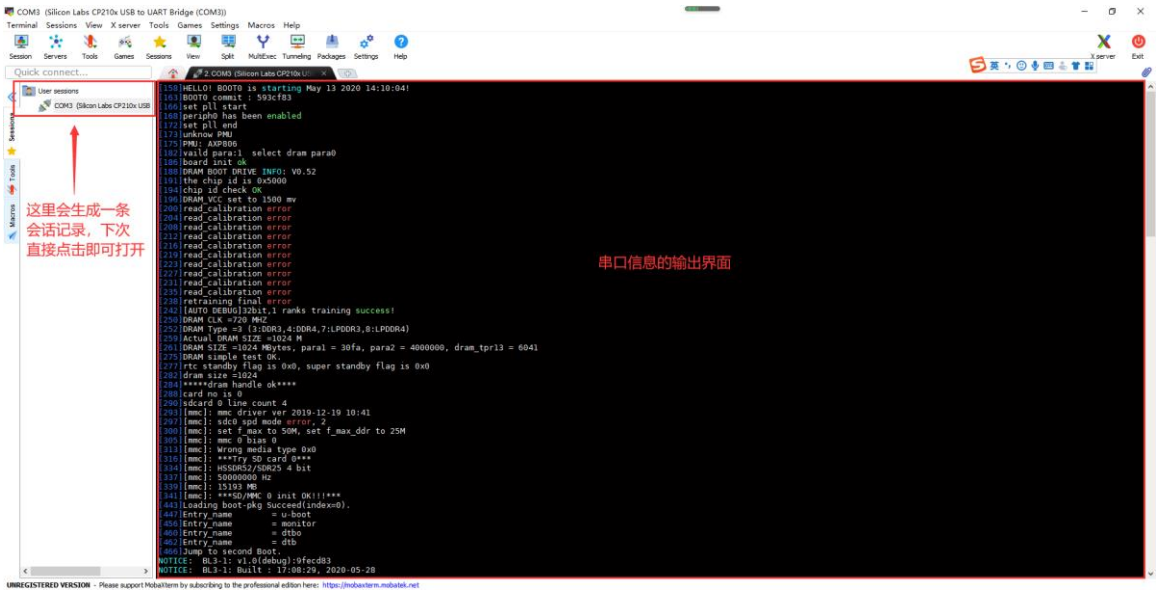
c. 选择串口的端口号（根据实际的情况选择对应的端口号），如果看不到端口号，请使用 **360 驱动大师**扫描安装 USB 转 TTL 串口芯片的驱动。

d. 选择串口的波特率为 **115200**。

e. 最后点击“**OK**”按钮完成设置。



4) 点击“**OK**”按钮后会进入下面的界面，此时启动开发板就能看到串口的输出信息了。

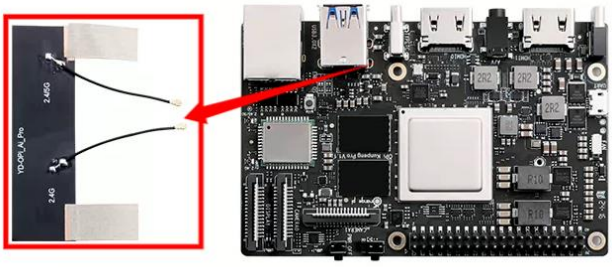


5) 当看到登录界面时，就可以使用下面的账号和密码来登录 Linux 系统了。

账号	密码
root	**openEuler**
openEuler	**openEuler**

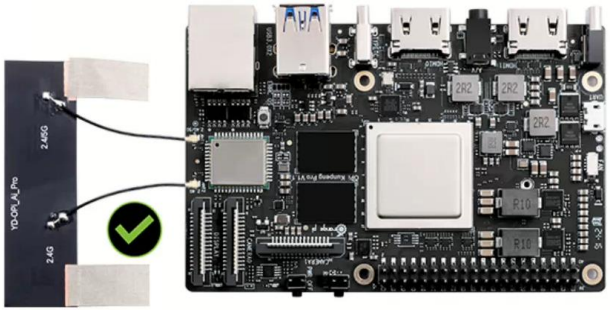
2.10. WIFI 蓝牙天线使用注意事项

开发板的 WIFI 蓝牙天线如下图左边所示：

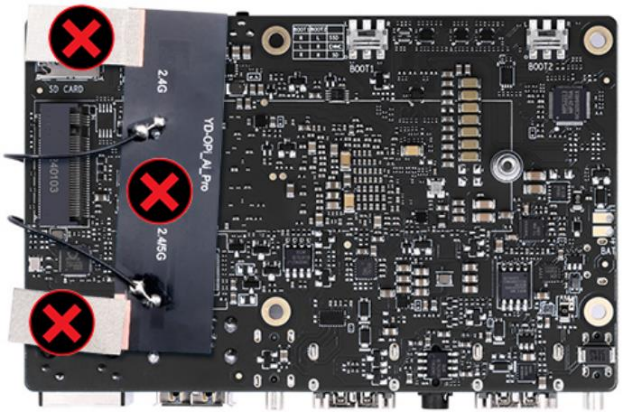


天线*1 开发板*1

WIFI 蓝牙天线的正确安装方法如下图所示：



安装好天线后，请不要像下图所示的一样将天线贴到开发板的背面，同时天线上的导电布也不能挨着开发板，否则可能会烧坏开发板。

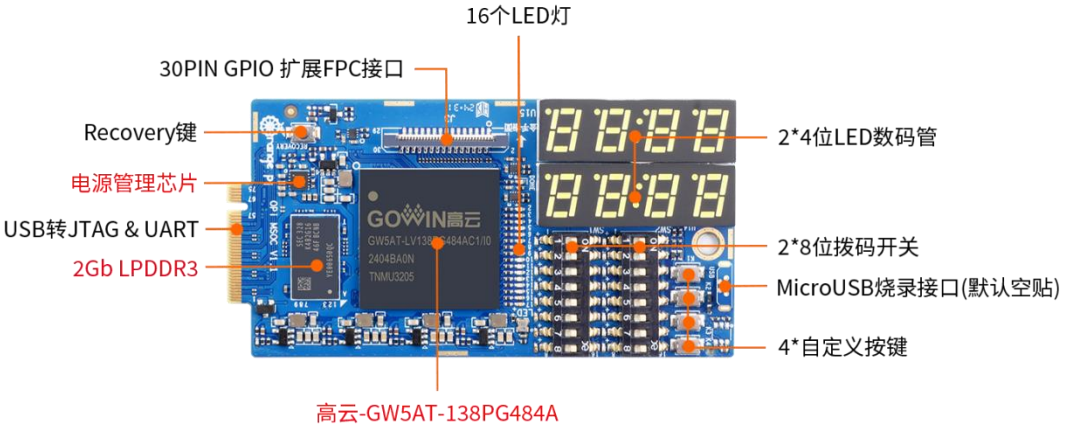


2. 11. v1.2 版本开发板搭配 OPi MSOC 的使用说明

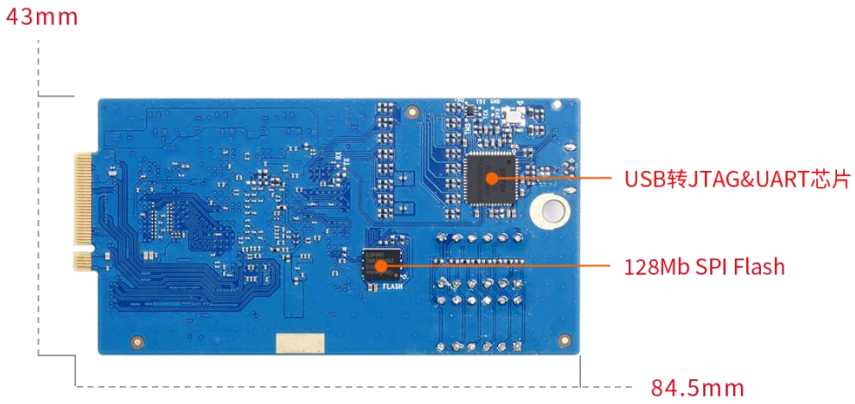
注意，Orange Pi MSOC必须搭配v1.2 版本的Kunpeng Pro开发板使用。

1) OPi MSOC 的接口详情图如下所示：

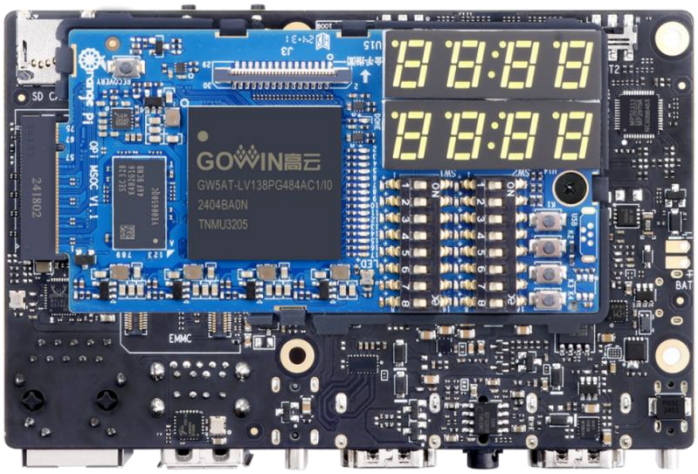
顶层视图



底层视图



2) OPi MSOC 安装在 Kunpeng Pro 开发板上的方法如下所示：



3) OPi MSOC 详细的使用说明请查看 OPi MSOC 对应的用户手册。

3. openEuler 桌面系统使用说明

3.1. 已支持的 openEuler 镜像类型和内核版本

Linux 镜像类型	内核版本	桌面版
openEuler	Linux5.10	支持

3.2. openEuler 系统功能适配情况

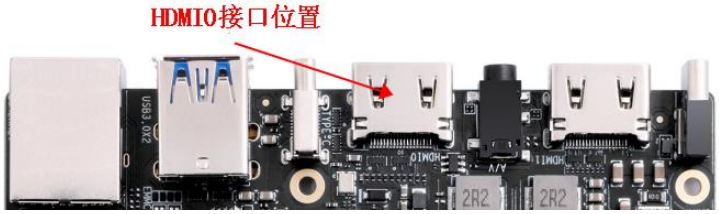
功能	是否能测试	Linux 内核驱动
HDMI0 显示	OK	OK
HDMI0 音频	NO	NO
HDMI1 显示	NO	NO
HDMI1 音频	NO	NO
耳机播放	NO	NO
耳机 MIC 录音	NO	NO
Type-C USB3.0(无 USB2.0)	OK	OK
USB3.0 Host x 2	OK	OK
千兆网口	OK	OK
千兆网口灯	OK	OK
WIFI	OK	OK
蓝牙	OK	OK
Micro USB 调试串口	OK	OK
复位按键	OK	OK
关机按键（无开机功能）	OK	OK
烧录按键	OK	OK
MIPI 摄像头 0	NO	NO
MIPI 摄像头 1	NO	NO
MIPI LCD 显示	NO	NO
电源指示灯	OK	OK
软件可控的 LED 灯	OK	OK
风扇接口	OK	OK
电池接口	OK	OK

TF 卡启动	OK	OK
TF 卡启动识别 eMMC	OK	OK
TF 卡启动识别 NVMe SSD	OK	OK
TF 卡启动识别 SATA SSD	OK	OK
eMMC 启动	OK	OK
SATA SSD 启动	OK	OK
NVMe SSD 启动	OK	OK
2 个拨码开关	OK	OK
40 pin-GPIO	OK	OK
40 pin-UART	OK	OK
40 pin-SPI	OK	OK
40 pin-I2C	OK	OK
40 pin-PWM	OK	NO

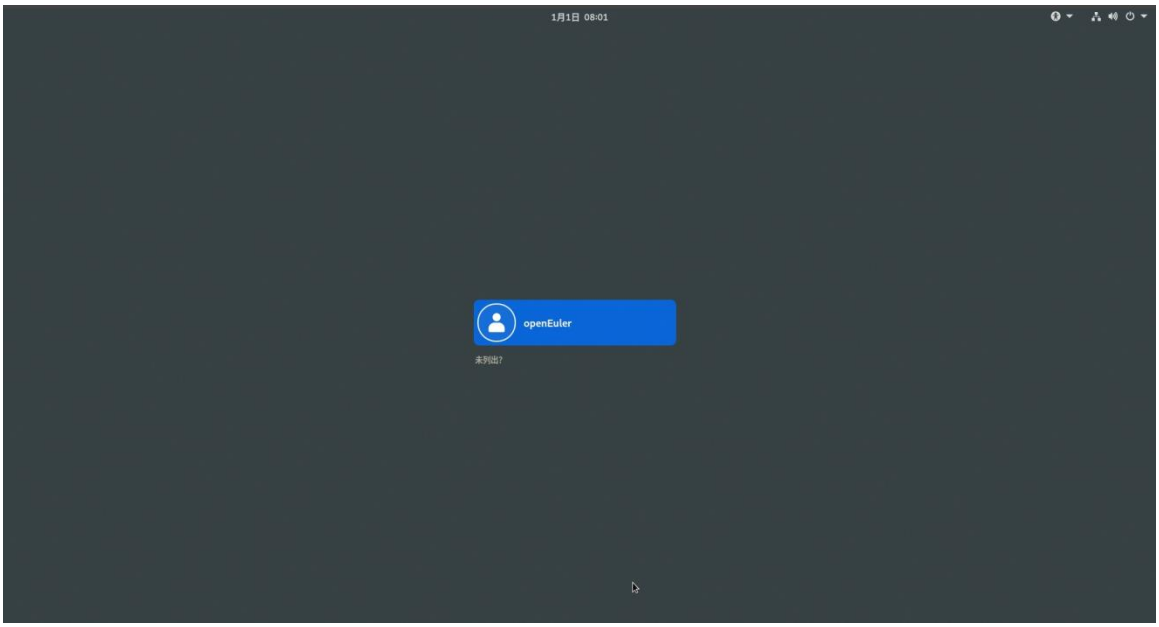
3.3. Linux 系统登录说明

3.3.1. 登录 Linux 系统桌面的方法

开发板有两个 HDMI 接口，目前只有 HDMI0 支持显示 Linux 系统的桌面，HDMI1 还需等软件更新。如果想显示 Linux 系统的桌面，请将开发板的 HDMI0 接口连接到 HDMI 显示器。



开发板上电开机后，需要等待一段时间，HDMI 显示器才会显示 Linux 系统的登录界面，登录界面如下图所示：



Linux 桌面系统的默认登录用户为 **openEuler**，登录密码为 **openEuler**。目前没有打开 root 用户登录的通道。成功登录后显示的 Linux 系统桌面如下图所示：



3.3.2. Linux 系统默认登录账号和密码

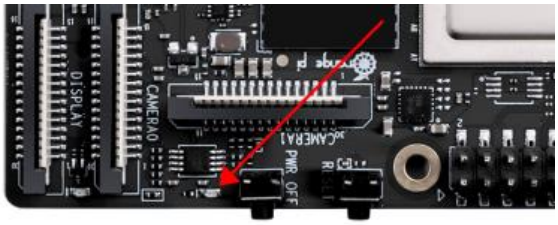
账号	密码
root	**openEuler**
openEuler	**openEuler**

当输入密码提示错误，或者 ssh 连接有问题，请注意，只要使用的是 Orange Pi 提供的 Linux 镜像，就请不要怀疑上面的密码不对，而是要找其他的原因。

3.4. 板载 LED 灯测试说明

开发板上有两个绿色的 LED 灯，作用如下所示：

1) 靠近关机按键的绿灯：此绿灯为电源指示灯，由硬件控制其亮灭，软件无法控制。只要开发板接入了 Type-C 电源并上电了，此绿灯就会点亮。



2) MIPI LCD 和 CAMERA0 之间的绿灯：此绿灯由 **GPIO4_19** 控制其亮灭，可以作为 SATA 硬盘的指示灯或者其他需要的用途。目前发布的 Linux 系统默认在 DTS 中将其点亮。当看到此灯点亮后，至少可以说明 Linux 内核已经启动了。



3.5. 网络连接测试

3.5.1. 以太网口测试

1) 最新版本的镜像默认给网口设置了静态 IP 地址。如下所示，使用 **ifconfig** 命令看到的网口的 IP 地址为 **192.168.10.8**。

```
[root@openEuler ~]# ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.10.8  netmask 255.255.255.0  broadcast 192.168.10.255
        inet6 fdcd:e671:36f4:0:c274:2bff:feef:1147  prefixlen 64  scopeid 0x0<global>
```

```
    inet6 fe80::c274:2bff:feef:1147  prefixlen 64  scopeid 0x20<link>
    ether c0:74:2b:ef:11:47  txqueuelen 1000  (Ethernet)
    RX packets 1061  bytes 225815 (220.5 KiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 12  bytes 892 (892.0 B)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

2) 默认的静态 IP 地址配置如下所示：

```
[root@openEuler ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE=eth0
BOOTPROTO=static
ONBOOT=yes
IPADDR=192.168.10.8
NETMASK=255.255.255.0
```

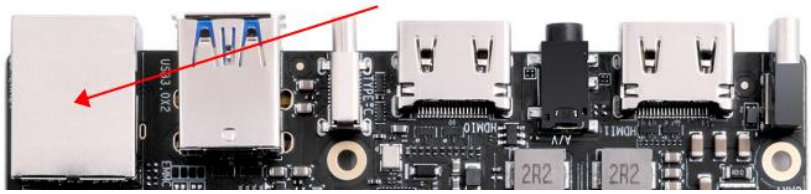
3) 如果不需要使用静态 IP 地址，需要连接路由器或者交换机来上网，可以将网口配置改成 DHCP。具体的配置如下所示：

```
[root@openEuler ~]# vim /etc/sysconfig/network-scripts/ifcfg-eth0
TYPE=Ethernet
BOOTPROTO=dhcp
DEFROUTE=yes
NAME=eth0
DEVICE=eth0
ONBOOT=yes
```

4) 然后重启系统。

```
[root@openEuler ~]# reboot
```

5) 然后将网线的一端插入开发板的以太网接口，网线的另一端接入交换机或者路由器，并确保网络是畅通的。



6) 系统重启后会通过 **DHCP** 自动给以太网口分配 IP 地址。

7) 在开发板的 Linux 系统中查看 IP 地址的命令如下所示，可以看到 IP 地址不再是默认的静态 IP 地址了。

```
[openEuler@openEuler ~]$ ip a s eth0
3: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 2c:52:af:89:11:11 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.100/24 brd 192.168.2.255 scope global dynamic noprefixroute eth0
       valid_lft 42077sec preferred_lft 42077sec
    inet6 fe80::913d:474c:4834:a1a4/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

8) 测试网络连通性的命令如下所示，如果能 ping 通百度说明开发板的网络连接正常，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行。

```
[openEuler@openEuler ~]$ ping www.baidu.com -I eth0
PING www.a.shifen.com (183.2.172.185) from 192.168.2.100 eth0: 56(84) bytes of data.
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=1 ttl=52 time=10.0 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=2 ttl=52 time=9.77 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=3 ttl=52 time=9.94 ms
64 bytes from 183.2.172.185 (183.2.172.185): icmp_seq=4 ttl=52 time=9.94 ms
^C
--- www.a.shifen.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 9.770/9.931/10.065/0.105 ms
```

3.5.2. WIFI 连接测试

请不要通过修改/etc/network/interfaces 配置文件的方式来连接 WIFI，通过这种方式连接 WIFI 网络使用会有问题。

3.5.2.1. 通过 nmcli 命令连接 WIFI 的方法

1) 先登录 Linux 系统，有下面三种方式：

a. 如果开发板连接了网线，可以通过 **ssh 远程登录 Linux 系统**。

a. 如果开发板连接好了调试串口，可以使用串口终端登录 Linux 系统。
b. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 Linux 系统。

2) 然后使用 **nmcli dev wifi** 命令扫描周围的 WIFI 热点。

```
[openEuler@openEuler ~]$ nmcli dev wifi
```

3) 然后使用 **nmcli** 命令连接扫描到的 WIFI 热点，其中：

a. **wifi_name** 需要换成想连接的 WIFI 热点的名字。
b. **wifi_passwd** 需要换成想连接的 WIFI 热点的密码。

```
[openEuler@openEuler ~]$ sudo nmcli dev wifi connect wifi_name password wifi_passwd
Device 'wlan0' successfully activated with 'cf937f88-ca1e-4411-bb50-61f402eef293'.
```

4) 通过 **ip addr show wlan0** 命令可以查看 wifi 的 IP 地址。

```
[openEuler@openEuler ~]$ ip a s wlan0
4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 54:f2:9f:7b:ba:36 brd ff:ff:ff:ff:ff:ff
    inet 10.31.2.93/16 brd 10.31.255.255 scope global dynamic noprefixroute wlan0
       valid_lft 43191sec preferred_lft 43191sec
    inet6 fe80::5297:7036:a33c:bb93/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

5) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行。

```
[openEuler@openEuler ~]$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (123.57.147.237) from 10.31.2.93 wlan0: 56(84) bytes of data.
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=1 ttl=53 time=47.1 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=2 ttl=53 time=44.3 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=3 ttl=53 time=45.0 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=4 ttl=53 time=71.0 ms
^C
--- www.orangepi.org ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 44.377/51.902/71.082/11.119 ms
```

3.5.2.2. 通过 nmtui 图形化方式连接 WIFI 的方法

1) 先登录 Linux 系统，有下面三种方式：

a. 如果开发板连接了网线，可以通过 **ssh 远程登录 Linux 系统**。

b. 如果开发板连接好了调试串口，可以使用串口终端登录 Linux 系统。

c. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 Linux 系统。

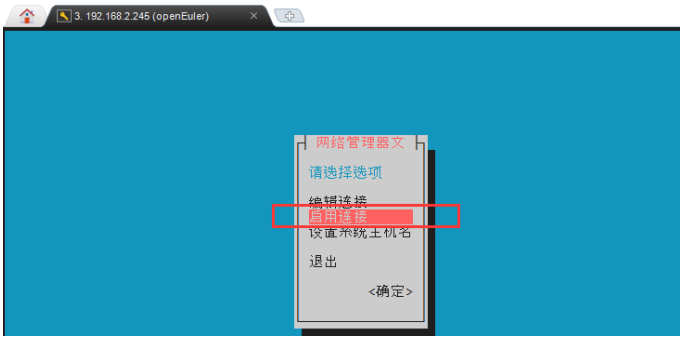
2) 然后在命令行中输入 nmtui 命令打开 wifi 连接的界面。

```
[openEuler@openEuler ~]$ sudo nmtui
```

3) 输入 nmtui 命令打开的界面如下所示：



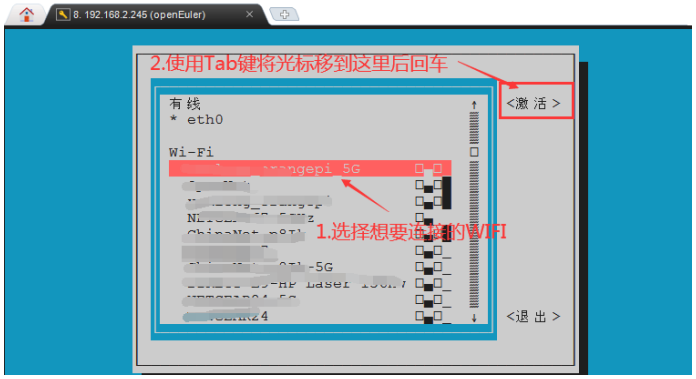
4) 选择启用连接后回车。



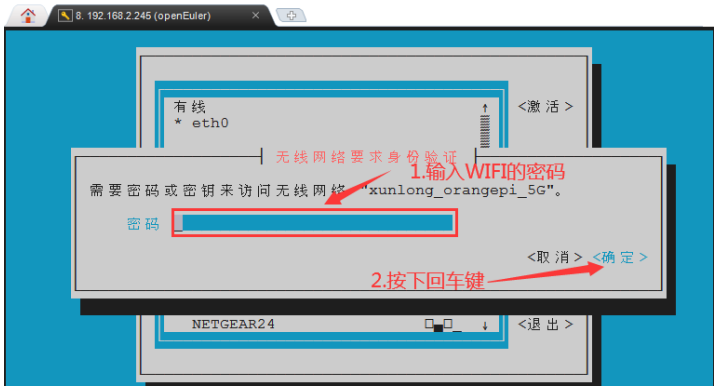
5) 然后就能看到所有搜索到的 WIFI 热点。



6) 选择想要连接的 WIFI 热点后再使用 Tab 键将光标定位到**激活**后回车。



7) 然后会弹出输入密码的对话框，在**密码**框输入对应的密码然后回车就会开始连接 WIFI。



8) WIFI 连接成功后会在已连接的 WIFI 名称前显示一个“*”。



9) 通过 **ip a s wlan0** 命令可以查看 wifi 的 IP 地址。

```
[openEuler@openEuler ~]$ ip a s wlan0
4: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP
group default qlen 1000
    link/ether 54:f2:9f:7b:ba:36 brd ff:ff:ff:ff:ff:ff
    inet 10.31.2.93/16 brd 10.31.255.255 scope global dynamic noprefixroute wlan0
       valid_lft 43003sec preferred_lft 43003sec
    inet6 fe80::5297:7036:a33c:bb93/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

10) 使用 **ping** 命令可以测试 wifi 网络的连通性，**ping** 命令可以通过 **Ctrl+C** 快捷键来中断运行。

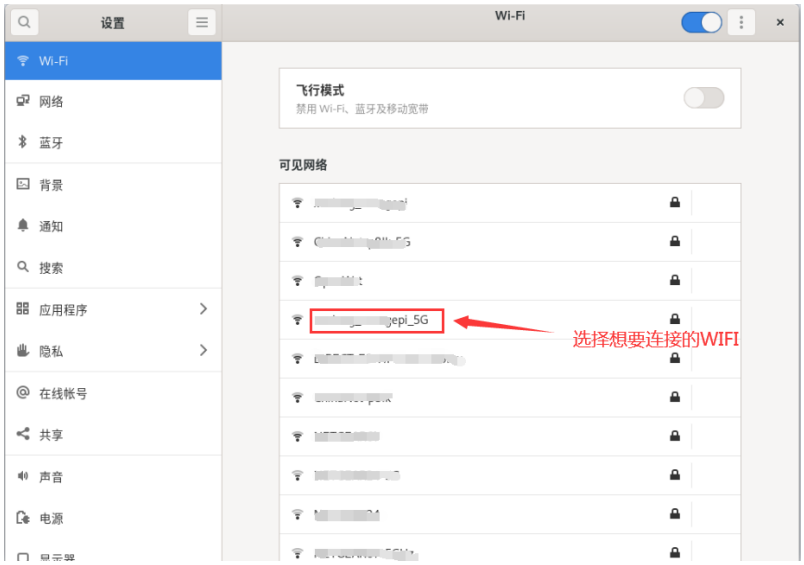
```
[openEuler@openEuler ~]$ ping www.orangepi.org -I wlan0
PING www.orangepi.org (123.57.147.237) from 10.31.2.93 wlan0: 56(84) bytes of data.
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=1 ttl=53 time=47.1 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=2 ttl=53 time=44.3 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=3 ttl=53 time=45.0 ms
64 bytes from 123.57.147.237 (123.57.147.237): icmp_seq=4 ttl=53 time=71.0 ms
^C
--- www.orangepi.org ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 44.377/51.902/71.082/11.119 ms
```

3.5.2.3. 桌面版镜像的测试方法

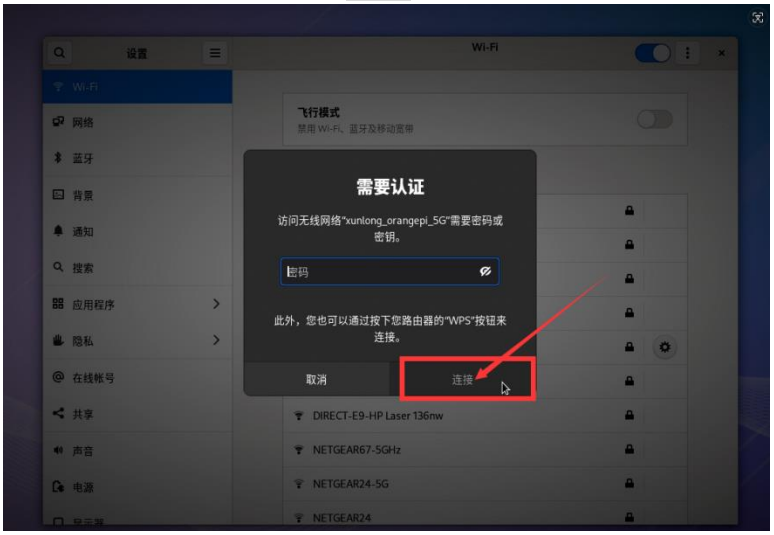
1) 首先点击桌面右上角的位置打开 WIFI 设置。



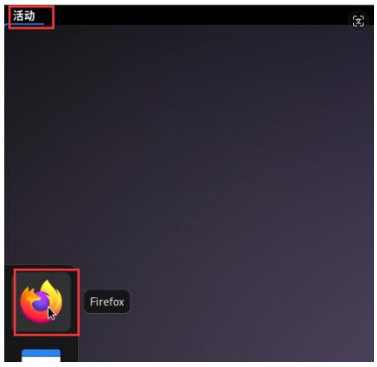
2) 在弹出的窗口中可以看到所有扫描到的 WIFI 热点，然后选择想要连接的 WIFI 热点。



3) 然后输入 WIFI 热点的密码，再点击连接就会开始连接 WIFI。



4) 连接好 WIFI 后，可以打开浏览器查看是否能上网，浏览器的入口如下图所示：



5) 打开浏览器后如果能打开其他网页说明 WIFI 连接正常。



3.6. SSH 远程登录开发板

Linux 系统默认都开启了 ssh 远程登录，并且允许 root 用户登录系统。ssh 登录前首先需要确保以太网或者 wifi 网络已连接，然后使用 ip addr 命令或者通过查看路由器的方式获取开发板的 IP 地址。

3.6.1. Ubuntu 下 SSH 远程登录开发板

1) 获取开发板的 IP 地址。

2) 然后就可以通过 ssh 命令远程登录 Linux 系统。

```
test@test:~$ ssh root@192.168.2.xxx          (需要替换为开发板的 IP 地址，不要照抄)
```

```
root@192.168.2.xx's password:          （在这里输入密码，默认密码为 openEuler）
```

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

如果提示拒绝连接，只要使用的是 Orange Pi 提供的镜像，就请不要怀疑 openEuler 这个密码是不是不对，而是要找其他原因。

3) 成功登录系统后的显示如下图所示：

```
csy@ubuntu:~$ ssh root@192.168.2.245
root@192.168.2.245's password:
Last failed login: Thu Jan  1 08:07:53 CST 1970 from 192.168.2.248 on ssh:notty
There was 1 failed login attempt since the last successful login.


Welcome to 5.10.0+

System information as of time:  1970年 01月 01日 星期四 08:07:56 CST

System load:    17.10
Processes:      288
Memory used:    6.8%
Swap used:      0.0%
Usage On:       99%
IP address:     192.168.2.245
IP address:     10.31.2.45
Users online:   3

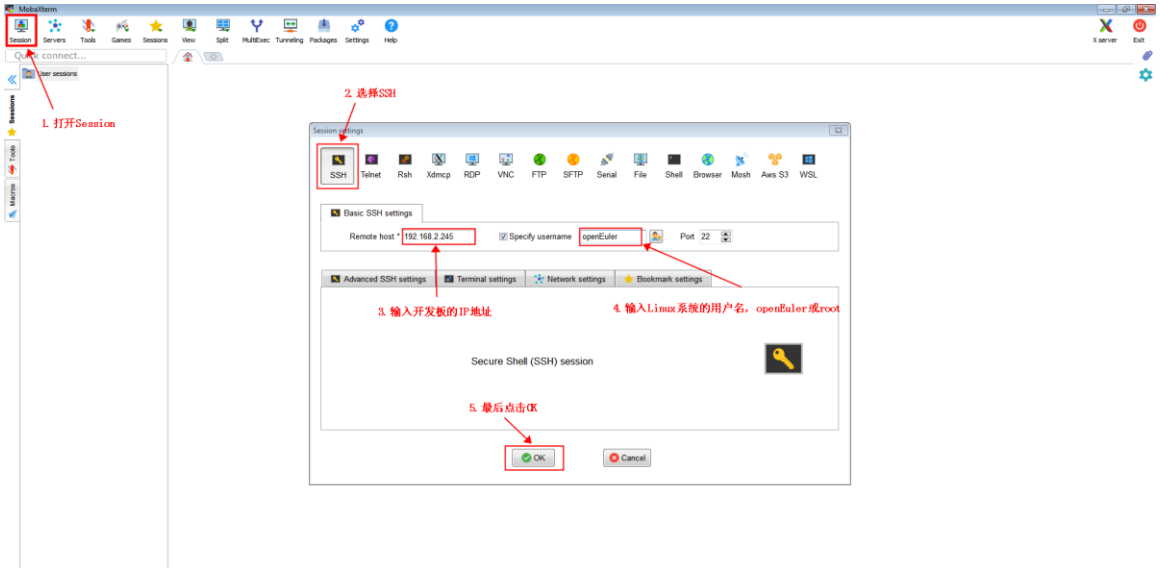

[root@openEuler ~]#
```

3.6.2. Windows 下 SSH 远程登录开发板

1) 首先获取开发板的 IP 地址。

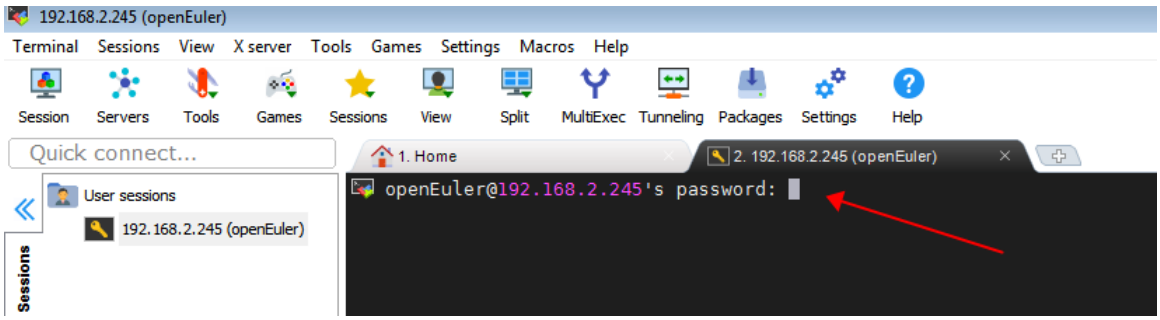
2) 在 Windows 下可以使用 MobaXterm 远程登录开发板，首先新建一个 ssh 会话。

a. 打开 **Session**。
b. 然后在 **Session Setting** 中选择 **SSH**。
c. 然后在 **Remote host** 中输入开发板的 IP 地址。
d. 然后在 **Specify username** 中输入 Linux 系统的用户名 **root** 或 **openEuler**。
e. 最后点击 **OK** 即可。

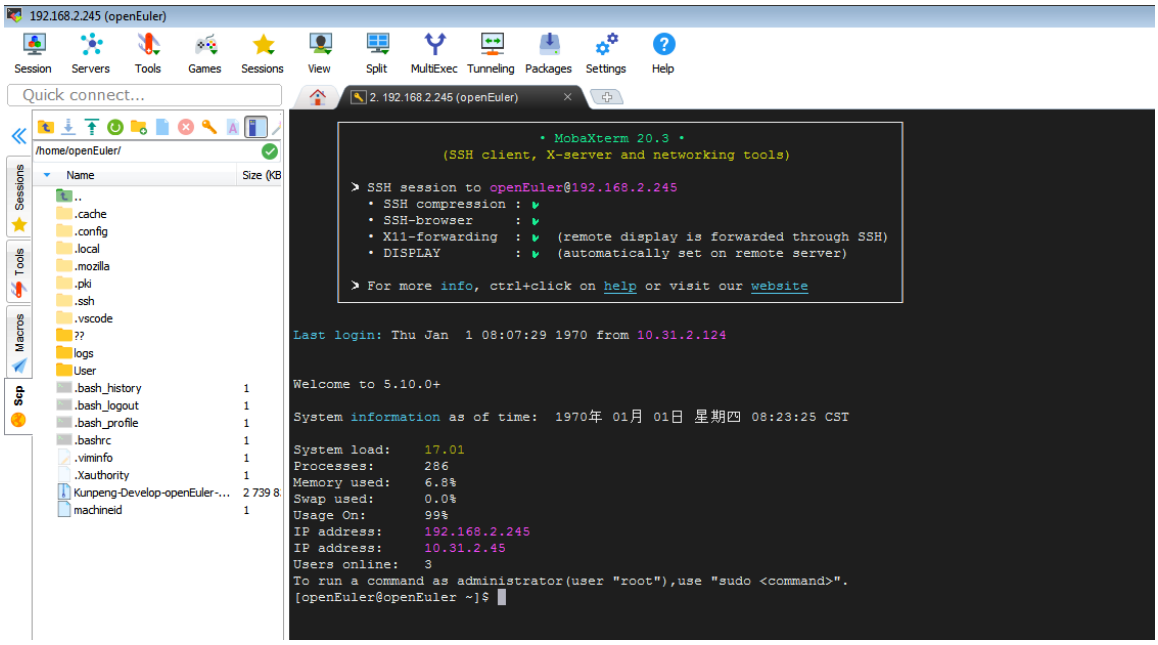


3) 然后会提示输入密码，默认 **root** 和 **openEuler** 用户的密码都为 **openEuler**。

注意，输入密码的时候，屏幕上是不会显示输入的密码的具体内容的，请不要以为是有什么故障，输入完后直接回车即可。

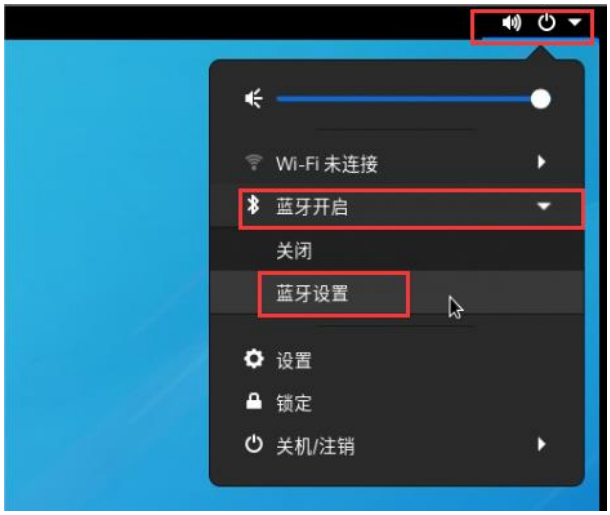


4) 成功登录系统后的显示如下图所示

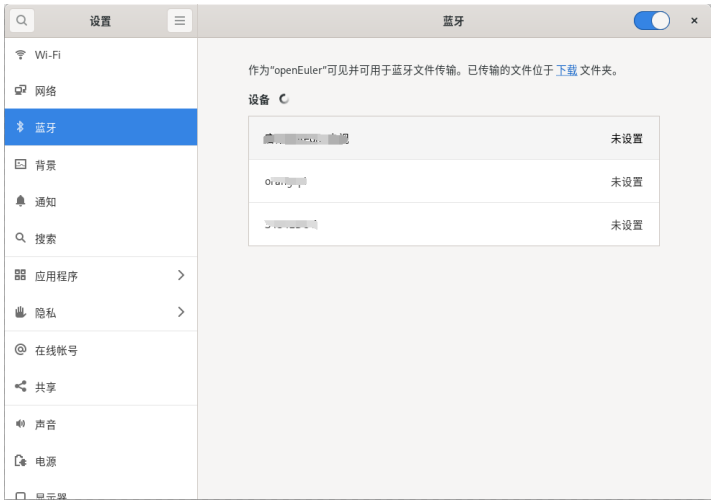


3.7. 蓝牙使用方法

1) 点击桌面右上角的位置打开蓝牙设置



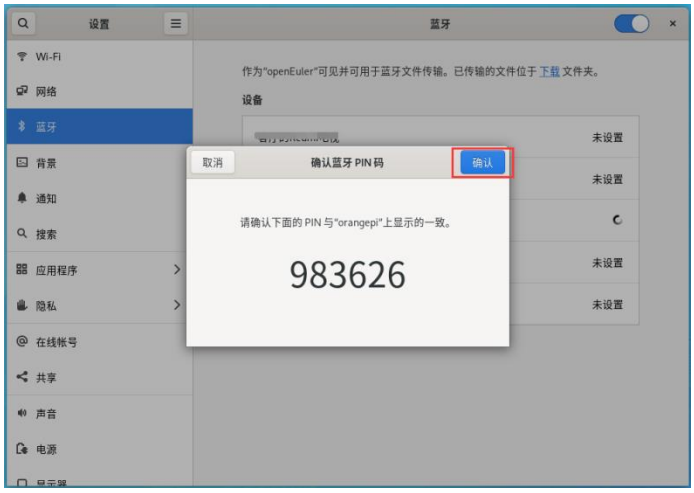
2) 然后在弹出的窗口中可以看到扫描到的蓝牙设备。



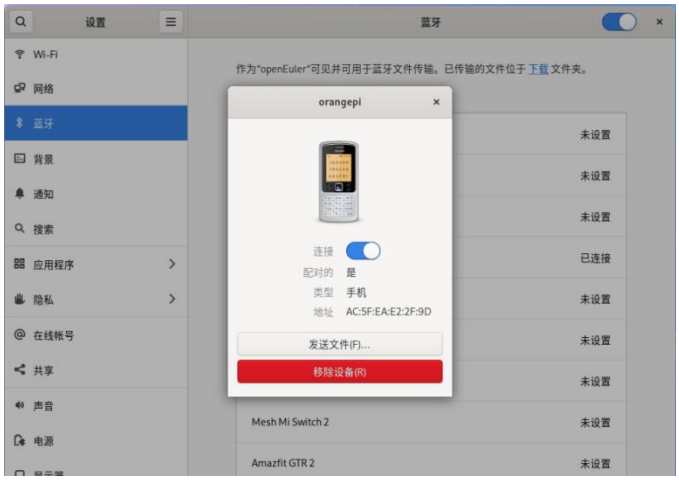
3) 然后选择想要连接的蓝牙设备，这里演示的是和 Android 手机配对。



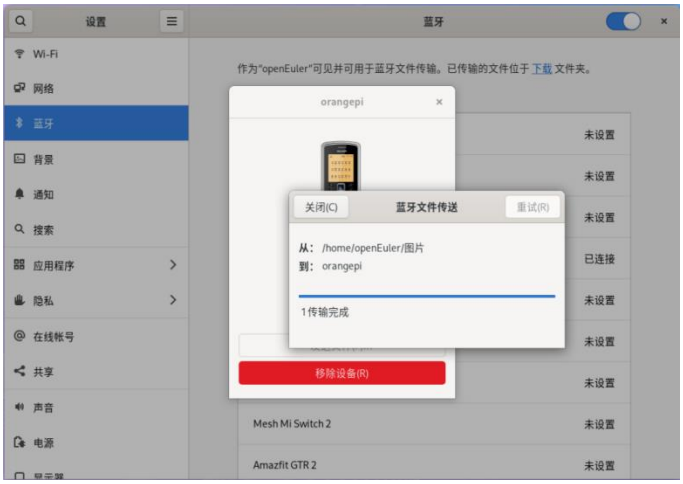
4) 配对时，桌面的右上角会弹出配对确认框，选择 **Confirm** 确认即可，此时手机上也同样需要进行确认。



5) 和手机配对完后，可以选择已配对的蓝牙设备，然后选择**发送文件**即可开始给手机发送一张图片。



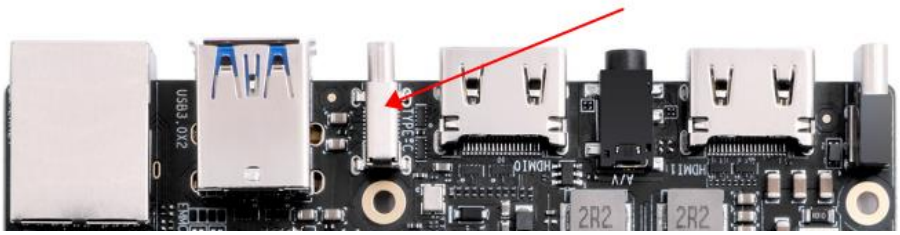
6) 发送图片的界面如下所示：



3.8. USB 接口测试

3.8.1. Type-C USB 接口使用注意事项

开发板有一个 Type-C USB 接口，其所在位置如下图所示：

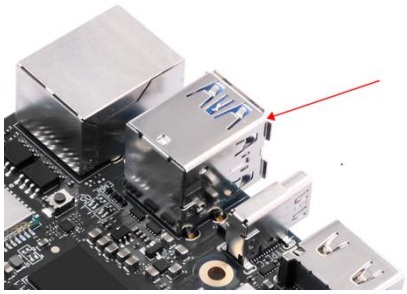


需要注意的是，Type-C USB 接口只有 USB3.0 的功能，没有 USB2.0 和 USB1.1 的功能（Soc 没有多余的 D+和 D-信号线可以用），所以无法接 USB 鼠标和键盘以及 USB2.0 的存储设备。测试此接口时，可以使用下图所示的 Type-C 转 USB3.0 转接线，来连接一个 USB3.0 的存储设备。



3.8.2. 连接 USB 鼠标或键盘测试

开发板有两个 USB HOST 接口可以接鼠标和键盘，其所在位置如下所示。如果 USB 接口不够用的话，还可以通过 USB Hub 来扩展 USB 接口的数量。



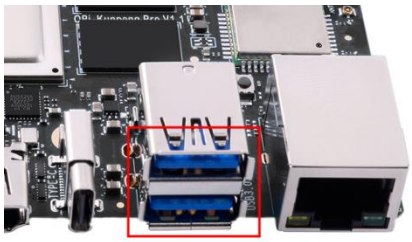
将 USB 接口的鼠标和键盘插入开发板的 USB 接口中，然后连接开发板到的 HDMI0 接口到 HDMI 显示器，等 HDMI 显示器显示 Linux 系统的桌面后就可以使用鼠标键盘来操作 Linux 系统了。

3.8.3. USB 音频测试

1) 首先需要准备一个带录音功能的 USB 接口的耳机。



2) 然后将 USB 接口的耳机插入开发的 USB 接口中。



3) 然后使用 **arecord -l** 命令查看下录音设备的编号，如下面的输出所示，其中 **card 0** 中的 **0** 表示录音设备编号为 **0**。

```
[openEuler@openEuler ~]$  arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: Audio [AB13X USB Audio], device 0: USB Audio [USB Audio]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

4) 然后进入 USB 音频测试代码的路径中。

```
[openEuler@openEuler ~]$ sudo -i
[root@openEuler ~]# cd /opt/opi_test/USBAudio
[root@openEuler USBAudio# ]# ls
Readme.md  main  main.c
```

5) 然后使用下面的命令可以使用 USB 音频设备录制一段音频。其中 **0** 表示录音设备编号，需根据实际情况填写。

```
[root@openEuler USBAudio# ]# ./main plughw:0
```

6) 录制结束后，在终端界面输入 **over** 即可退出录制。

```
[root@openEuler USBAudio# ]# ./main plughw:0
Start record!
over            #输入 over 结束录制音频。
```

7) 录音成功后，在 USBAudio 样例目录下会生成音频文件 **audio.pcm**。

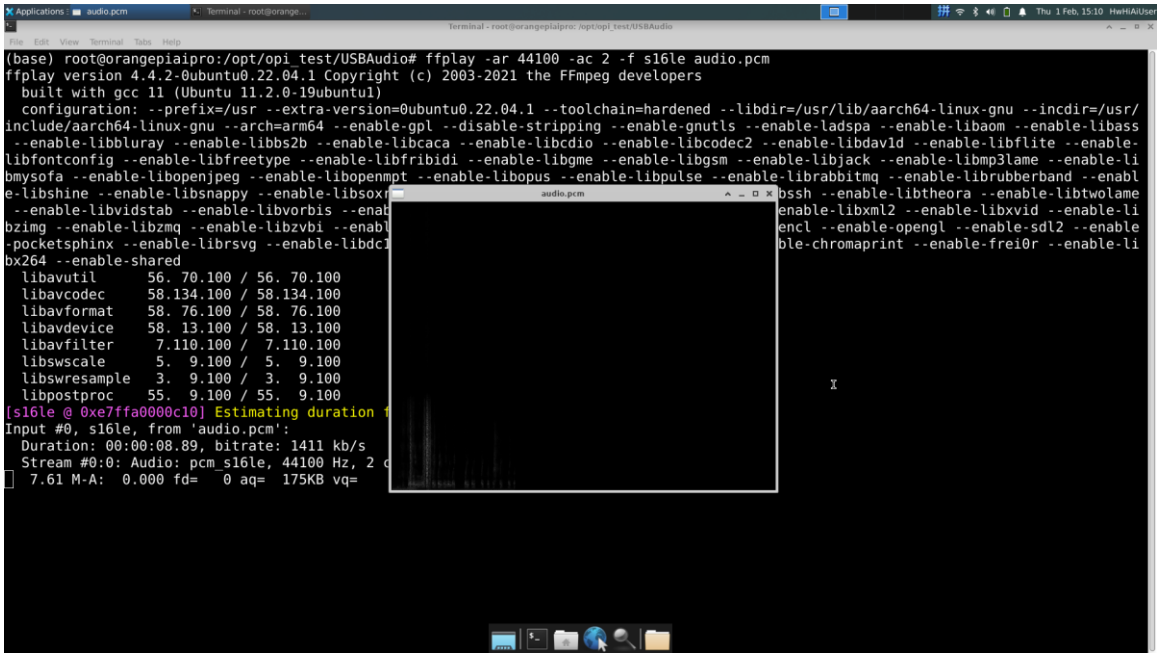
```
[root@openEuler USBAudio# ]# ls *.pcm
audio.pcm
```

8) 然后确保使用 **aplay -l** 命令能看到 USB 的播音设备。

```
[root@openEuler USBAudio# ]# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: Audio [AB13X USB Audio], device 0: USB Audio [USB Audio]
  Subdevices: 0/1
  Subdevice #0: subdevice #0
```

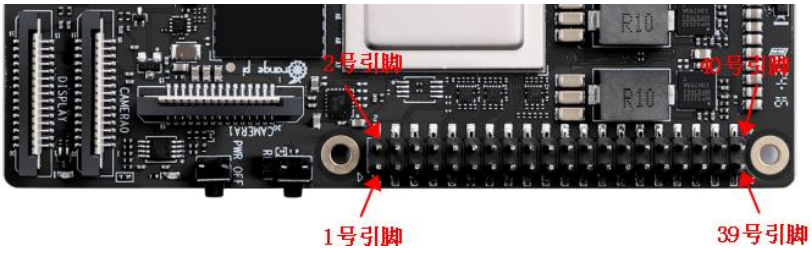
9) 然后在 Linux 系统桌面中，使用下面的命令可以将录制的音频播放到 USB 耳机。

```
[root@openEuler USBAudio# ]# ffplay -ar 44100 -ac 2 -f s16le audio.pcm
```



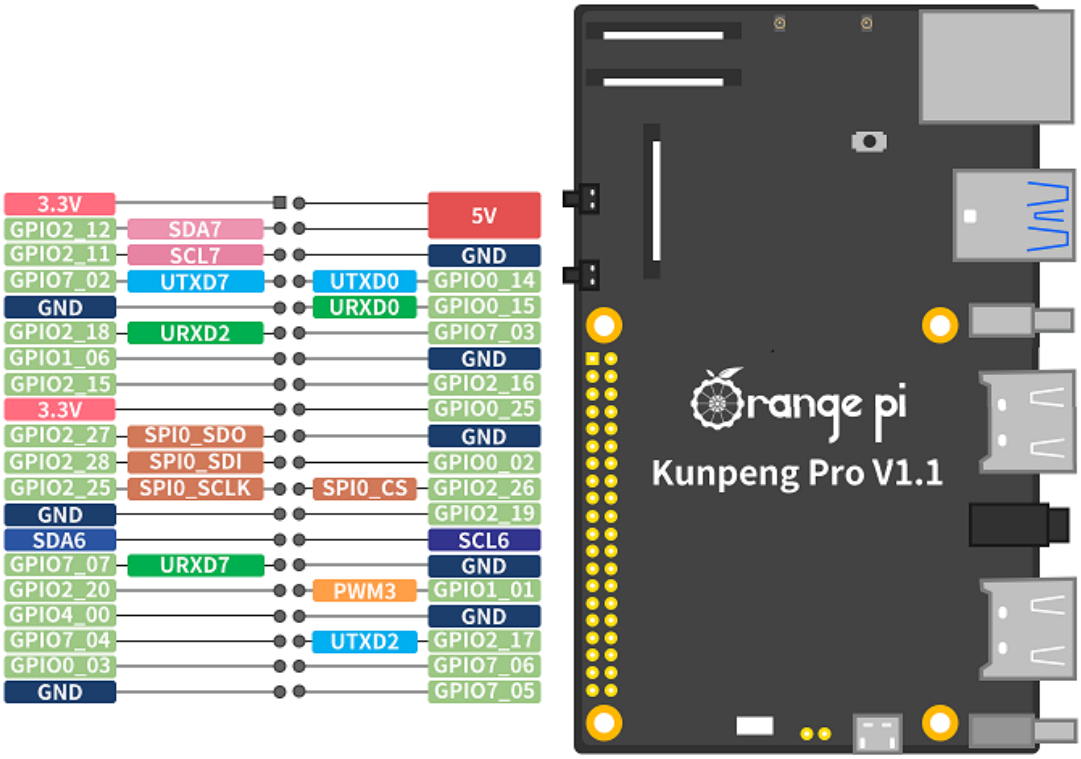
3.9. 40 Pin 接口引脚功能说明

开发板的 40 pin 接口引脚的顺序如下图所示：



开发板 40 pin 接口引脚的功能如下表所示：

GPIO序号	GPIO	功能	引脚	引脚	功能	GPIO	GPIO序号
		3.3V	1	2	5V		
76	GPIO2_12	SDA7	3	4	5V		
75	GPIO2_11	SCL7	5	6	GND		
226	GPIO7_02	UTXD7	7	8	UTXD0	GPIO0_14	14
		GND	9	10	URXD0	GPIO0_15	15
82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229



40 pin 接口使用注意事项如下所示：

1) 40 pin 接口中总共有 **26** 个 GPIO 口，但 8 号和 10 号引脚默认是用于调试串口功能的，并且这两个引脚和 Micro USB 调试串口是连接在一起的，所以这两个引脚请不要设置为 GPIO 等功能。

2) 所有的 GPIO 口的电压都是 **3.3v**。

3) 40 pin 接口中 27 号和 28 号引脚只有 I2C 的功能，没有 GPIO 等其他复用功能，另外这两个引脚的电压默认都为 **1.8v**。

3.10. 40 pin 接口 GPIO、I2C、UART、SPI 和 PWM 测试

3.10.1. 40 pin GPIO 口的测试方法

开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚默认配置为 GPIO 功能，可以直接使用，其他具有 GPIO 复用功能的引脚需要修改 DTS 配置才能正常使用 GPIO 的功能。

GPIO序号	GPIO	功能	引脚
		3.3V	1
76	GPIO2_12	SDA7	3

引脚	功能	GPIO	GPIO序号
2	5V		
4	5V		

75	GPIO2_11	SCL7	5	6	GND		
226	GPIO7_02	UTXD7	7	8	UTXD0	GPIO0_14	14
		GND	9	10	URXD0	GPIO0_15	15
82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

Linux 镜像中预装了 **gpio_operate** 工具用于设置 GPIO 管脚的输入与输出方向，也可将每个 GPIO 管脚独立的设为 0 或 1。**gpio_operate** 工具的详细使用方法如下所示：

1) **gpio_operate** 工具必须使用 **root** 帐号执行。

2) **gpio_operate -h** 命令可以获取 **gpio_operate** 工具的帮助信息：

```
[root@openEuler ~]# gpio_operate -h
Usage: gpio_operate <Command|-h> [Options...]
gpio_operate Command:
        -h                      : This command's help information.
        set_value               : Set gpio pin value.
        get_value               : Get gpio pin value.
        set_direction           : Set gpio pin direction value.
        get_direction           : Get gpio pin direction value.
```

3) **gpio_operate get_direction *gpio_group gpio_pin*** 用于查询 GPIO 管脚方向。

a. gpio_group 和 gpio_pin 参数说明如下所示：

类型	描述
gpio_group	GPIO 组号，取值为[0，8]
gpio_pin	GPIO 管脚号，取值为[0，31]

b. 比如 40 pin 中的第 31 号引脚对应的 GPIO 为 GPIO2_20，那么其 GPIO 组号为 2，GPIO 管脚号为 20，获取其方向的命令为：

```
[root@openEuler ~]# gpio_operate get_direction 2 20
Get gpio pin direction value successed, value is 0.
```

c. 输出的打印信息说明

字段	说明
direction	GPIO 管脚方向，取值为[0，1] • 0：输入方向 • 1：输出方向

4) **gpio_operate set_direction *gpio_group gpio_pin direction*** 用于设置 GPIO 管脚方向。

a. gpio_group、gpio_pin 和 direction 参数说明如下所示：

类型	描述
gpio_group	GPIO 组号，取值为[0，8]
gpio_pin	GPIO 管脚号，取值为[0，31]
direction	GPIO 管脚方向，取值为[0，1] • 0：输入方向 • 1：输出方向

b. 比如 40 pin 中的第 31 号引脚对应的 GPIO 为 GPIO2_20，那么其 GPIO 组号为 2，GPIO 管脚号为 20，设置其方向为输出的命令为：

```
[root@openEuler ~]# gpio_operate set_direction 2 20 1
Set gpio pin direction value successed.
```

5) **gpio_operate get_value *gpio_group gpio_pin*** 命令用于查询 GPIO 管脚值。

a. gpio_group 和 gpio_pin 参数说明如下所示：

类型	描述
gpio_group	GPIO 组号，取值为[0，8]
gpio_pin	GPIO 管脚号，取值为[0，31]

b. 比如 40 pin 中的第 31 号引脚对应的 GPIO 为 GPIO2_20，那么其 GPIO 组号为 2，GPIO 管脚号为 20，查询其管脚值的命令如下所示：

```
[root@openEuler ~]# gpio_operate get_value 2 20
Get gpio pin value successed, value is 0.          #这里查询到的值为 0，也就是低电平
```

6) **gpio_operate set_value *gpio_group gpio_pin value*** 命令用于设置 GPIO 管脚值为高电平或者低电平，**注意设置管脚值前，请确保已将 GPIO 管脚的方向设置为输出了**。

a. gpio_group、gpio_pin 和 value 参数说明如下所示：

类型	描述
gpio_group	GPIO 组号，取值为[0，8]
gpio_pin	GPIO 管脚号，取值为[0，31]
value	GPIO 管脚值，取值为[0，1] 当 GPIO 管脚方向为输入方向时，不允许设置 GPIO 管脚值。

b. 比如 40 pin 中的第 31 号引脚对应的 GPIO 为 GPIO2_20，那么其 GPIO 组号为 2，GPIO 管脚号为 20，设置其输出为高电平的命令为：

```
[root@openEuler ~]# gpio_operate set_value 2 20 1
```

3.10.2. 40 pin SPI 回环测试

开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 SPI 功能，并且 Linux 系统默认配置为了 SPI 功能，可以直接使用。

GPIO序号	GPIO	功能	引脚
		3.3V	1
76	GPIO2_12	SDA7	3
75	GPIO2_11	SCL7	5
226	GPIO7_02	UTXD7	7
		GND	9

引脚	功能	GPIO	GPIO序号
2	5V		
4	5V		
6	GND		
8	UTXD0	GPIO0_14	14
10	URXD0	GPIO0_15	15

82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

40 pin 接口中的 SPI 总线为 SPI0，测试前请先确保**/dev** 下存在 **spidev0.0** 设备节点。

```
[openEuler@openEuler ~]$ ls /dev/spidev0.0
/dev/spidev0.0
```

然后先不短接 SPI0 的 mosi 和 miso 两个引脚，运行 spidev_test 的输出结果如下所示，可以看到 TX 和 RX 的数据不一致。

```
[openEuler@openEuler ~]$ sudo spidev_test -v -D /dev/spidev0.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D  |......@.....................|
RX | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00  |.............................|
```

然后用杜邦线短接 SPI1 的 mosi（40 pin 接口中的第 19 号引脚）和 miso（40 pin 接口中的第 21 号引脚）两个引脚再运行 spidev_test 的输出如下，可以看到发送和接收的数据一样，说明回环测试成功。

```
[openEuler@openEuler ~]$ sudo spidev_test -v -D /dev/spidev0.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D  | ......@....▒.................▒.
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D  | ......@....▒.................▒.
```

3.10.3. 40 pin I2C 测试

开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 I2C 功能，并且 Linux 系统默认配置为了 I2C 功能，可以直接使用。

GPIO序号	GPIO	功能	引脚	引脚	功能	GPIO	GPIO序号
		3.3V	1	2	5V		
76	GPIO2_12	SDA7	3	4	5V		
75	GPIO2_11	SCL7	5	6	GND		
226	GPIO7_02	UTXD7	7	8	UTXD0	GPIO0_14	14
		GND	9	10	URXD0	GPIO0_15	15
82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

启动 Linux 系统后，先确认下**/dev** 下存在 i2c6 和 i2c7 的设备节点。

```
[openEuler@openEuler ~]$ ls /dev/i2c-6
/dev/i2c-6
[openEuler@openEuler ~]$ ls /dev/i2c-7
/dev/i2c-7
```

然后在 40 pin 接口的 i2c6 或者 i2c7 引脚上接一个 i2c 设备。

	i2c6	i2c7
sda 引脚	对应 40 pin 中 27 号引脚	对应 40 pin 中 3 号引脚
scl 引脚	对应 40 pin 中 28 号引脚	对应 40 pin 中 5 号引脚
3.3v 引脚	对应 40 pin 中 1 号引脚	对应 40 pin 中 1 号引脚
gnd 引脚	对应 40 pin 中 6 号引脚	对应 40 pin 中 6 号引脚

然后使用 **i2cdetect** 命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 能正常使用。

1) i2c6 使用的命令如下所示：

```
[openEuler@openEuler ~]$ sudo i2cdetect -y -r 6
```

2) i2c7 使用的命令如下所示：

```
[openEuler@openEuler ~]$ sudo i2cdetect -y -r 7
```

不同的 i2c 设备地址是不同的，下图 0x68 地址只是一个示例。请以实际看到的为准。

```
[openEuler@openEuler ~]$ sudo i2cdetect -y -r 7
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
[openEuler@openEuler ~]$
```

3.10.4. 40 pin UART 测试

开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 uart 功能，并且 Linux 系统默认配置为了 uart 功能，可以直接使用。另外请注意 uart0 默认设

置为调试串口功能，请不要将其当成普通串口使用。

GPIO序号	GPIO	功能	引脚	引脚	功能	GPIO	GPIO序号
		3.3V	1	2	5V		
76	GPIO2_12	SDA7	3	4	5V		
75	GPIO2_11	SCL7	5	6	GND		
226	GPIO7_02	UTXD7	7	8	UTXD0	GPIO0_14	14
		GND	9	10	URXD0	GPIO0_15	15
82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

启动 Linux 系统后，先确认下**/dev** 下存在 uart 的设备节点。

```
[openEuler@openEuler ~]$ ls /dev/ttyAMA*
/dev/ttyAMA0  /dev/ttyAMA1  /dev/ttyAMA2
```

uart 设备节点和 uart 对应关系如下所示：

uart 设备节点	uart 接口
/dev/ttyAMA1	uart2
/dev/ttyAMA2	uart7

然后开始测试 uart 接口，先使用杜邦线短接要测试的 uart 接口的 rx 和 tx 引脚。不同的 uart 的 rx 和 tx 引脚对应的 40 pin 接口中的引脚如下所示：

uart 接口	rx 引脚	tx 引脚

uart2	40pin 的 11 号引脚	40pin 的 36 号引脚
uart7	40pin 的 29 号引脚	40pin 的 7 号引脚

然后进入串口测试程序的路径。

```
[openEuler@openEuler ~]$ sudo -i
[root@openEuler ~]# cd /opt/opi_test/uart
[root@openEuler uart]# ls
serial  serial.c
```

串口测试程序 serial 的使用方法如下所示：

```
[root@openEuler uart]# ./serial
Usage: ./serial <serialport>
```

使用 serial 测试程序可以测试下串口的自收自发。serial 程序会打开对应的串口发送一个字符串——**Hello, Serial Port!**，然后打印接收到的字符串。如果自发自收的字符串相同，说明测试成功。

1) uart2 测试命令如下所示：

```
[root@openEuler uart]# ./serial /dev/ttyAMA1
W: Hello, Serial Port!
R: Hello, Serial Port!
```

2) uart7 测试命令如下所示：

```
[root@openEuler uart]# ./serial /dev/ttyAMA2
W: Hello, Serial Port!
R: Hello, Serial Port!
```

3.10.5. 40 pin PWM 测试

开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 pwm 功能。

GPIO序号	GPIO	功能	引脚
		3.3V	1
76	GPIO2_12	SDA7	3
75	GPIO2_11	SCL7	5
226	GPIO7_02	UTXD7	7
		GND	9
82	GPIO2_18	URXD2	11

引脚	功能	GPIO	GPIO序号
2	5V		
4	5V		
6	GND		
8	UTXD0	GPIO0_14	14
10	URXD0	GPIO0_15	15
12		GPIO7_03	227

38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

目前只能通过操作寄存器的方式来测试 PWM3 引脚输出一个波形。测试步骤如下所示：

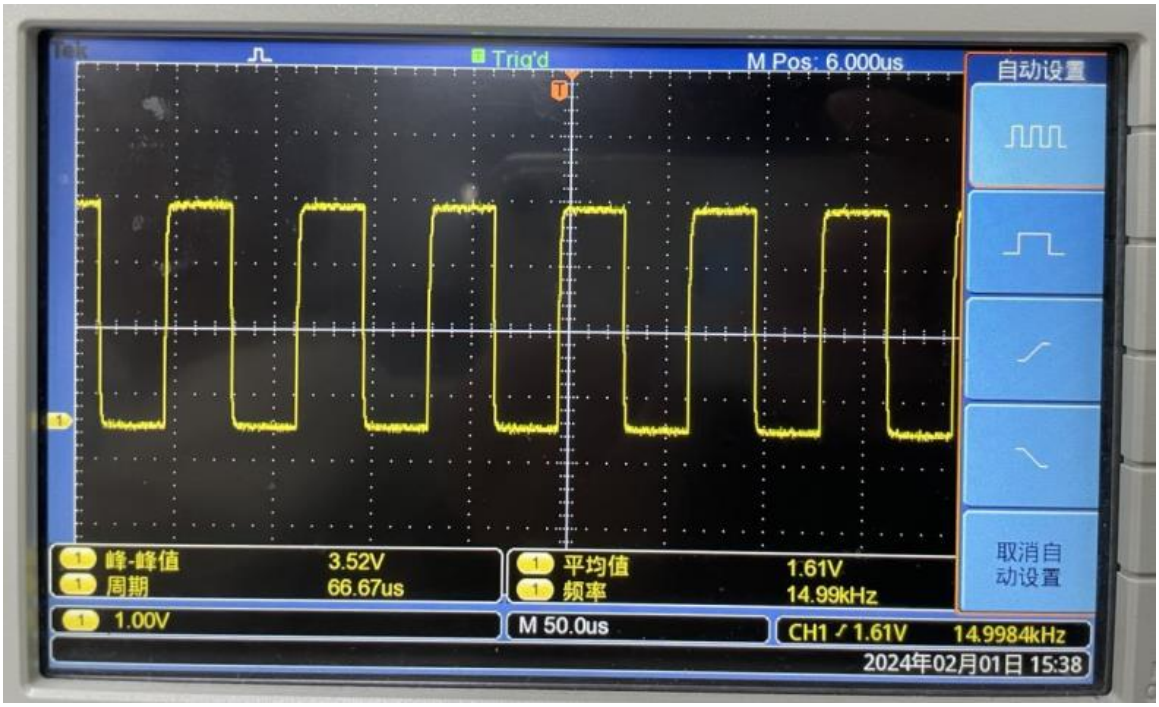
1) 首先进入 pwm 的测试代码的路径。

```
[openEuler@openEuler ~]$ sudo -i
[sudo] password for openEuler:
[root@openEuler ~]# cd /opt/opi_test/pwm
[root@openEuler pwm]# ls
test.sh
```

2) 然后运行 test.sh 脚本即可输出一个 50%占空比的方波。

```
[root@openEuler pwm]# ./test.sh
```

3) 然后用示波器测量 40 pin 中的第 32 号引脚就可以查看 PWM 的输出波形，如下所示：



3.10.6. 40 pin CAN 的测试方法

3.10.6.1. 打开 CAN 的方法

1) 开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 CAN 功能。

GPIO序号	GPIO	功能	引脚
		3.3V	1
76	GPIO2_12	SDA7	3
75	GPIO2_11	SCL7	5
226	GPIO7_02	UTXD7	7
		GND	9
82	GPIO2_18	CAN_RX3	11
38	GPIO1_06		13
79	GPIO2_15		15
		3.3V	17
91	GPIO2_27	SPI0_SDO	19
92	GPIO2_28	SPI0_SDI	21
89	GPIO2_25	SPI0_SCLK	23

引脚	功能	GPIO	GPIO序号
2	5V		
4	5V		
6	GND		
8	UTXD0	GPIO0_14	14
10	URXD0	GPIO0_15	15
12		GPIO7_03	227
14	GND		
16		GPIO2_16	80
18		GPIO0_25	25
20	GND		
22		GPIO0_02	2
24	SPI0_CS	GPIO2_26	90

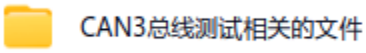
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	CAN_TX3	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

2) CAN3 对应的引脚为：

	CAN3
RX 引脚	**对应 40pin 的 11 号引脚**
TX 引脚	**对应 40pin 的 36 号引脚**

3) Linux 系统中 40pin 的 11 号和 36 号引脚在设备树中默认设置为 UART2 功能。需要更新下 Linux 系统的 dt.img 才能打开 CAN3 功能（**打开 CAN3 后 UART2 就无法使用了，这两个功能二选一**）。具体步骤如下所示：

a. 首先在开发板的**官方工具**中下载打开了 CAN3 功能的 dt.img 文件。



b. 然后将 dt.img 文件上传到开发板的 Linux 系统中。上传文件到开发板 Linux 中的方法请参考**上传文件到开发板 Linux 系统中的方法**一小节的说明。

c. 然后使用下面的命令更新 dt.img 文件。

a) TF 卡启动：

```
sudo dd if=dt.img of=/dev/mmcblk1 count=4096 seek=114688 bs=512
sudo dd if=dt.img of=/dev/mmcblk1 count=4096 seek=376832 bs=512
```

b) NVMe SSD 启动：

```
sudo dd if=dt.img of=/dev/nvme0n1 count=4096 seek=114688 bs=512
sudo dd if=dt.img of=/dev/nvme0n1 count=4096 seek=376832 bs=512
```

c) eMMC 启动：

```
sudo dd if=dt.img of=/dev/mmcblk0 count=4096 seek=114688 bs=512
sudo dd if=dt.img of=/dev/mmcblk0 count=4096 seek=376832 bs=512
```

d. 然后重启 Linux 系统。

4) 重启进入 Linux 系统后，使用 **sudo ifconfig -a** 命令如果能看到 CAN3 的设备节

点，就说明 CAN3 已正确打开了。

```
[openEuler@openEuler ~]$ sudo ifconfig -a
.....
can3: flags=128<NOARP>  mtu 16
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00  txqueuelen 10  (UNSPEC)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
.....
```

3.10.6.2. 使用 CANalyst-II 分析仪测试 CAN 总线收发消息

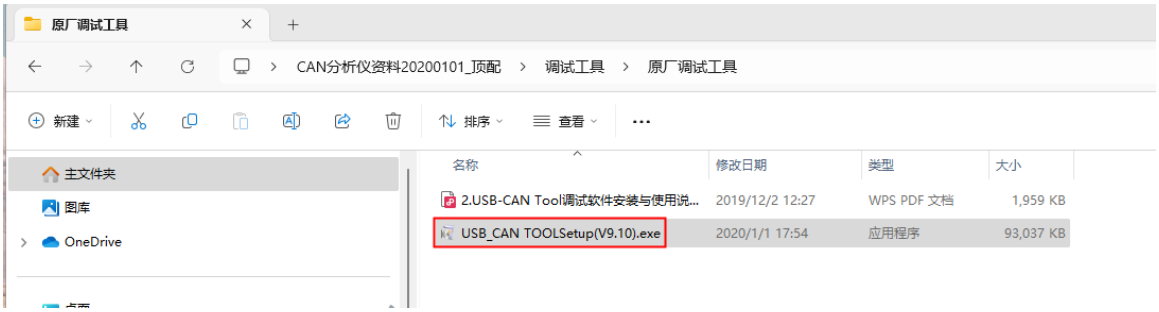
1) 测试使用的 CANalyst-II 分析仪如下图所示：



2) CANalyst-II 分析仪资料下载链接。

https://www.zhcxgd.com/3.html

3) 首先要安装 USB_CAN ToolSetup 这个软件。



4) USB_CAN ToolSetup 安装后的快捷方式为：



5) 另外还需要安装一下 USB 驱动程序。



6) CANalyst-II 分析仪的 USB 接口那端需要接到电脑的 USB 接口中。



7) 测试 CAN 功能还需要准备一个下图所示的 CAN 收发器，CAN 收发器主要功能是将 CAN 控制器的 TTL 信号转换成 CAN 总线的差分信号。

a. CAN 收发器的 3.3V 引脚需要接开发板 40 pin 中的 3.3V 引脚。
b. CAN 收发器的 GND 引脚需要接开发板 40 pin 中的 GND 引脚。
c. CAN 收发器的 CAN TX 引脚需要接开发板 40 pin 中 CAN 总线的 TX 引脚。
d. CAN 收发器的 CAN RX 引脚需要接开发板 40 pin 中 CAN 总线的 RX 引脚。
e. CAN 收发器的 CANL 引脚需要接分析仪的 H 接口。
f. CAN 收发器的 CANL 引脚需要接分析仪的 L 接口。



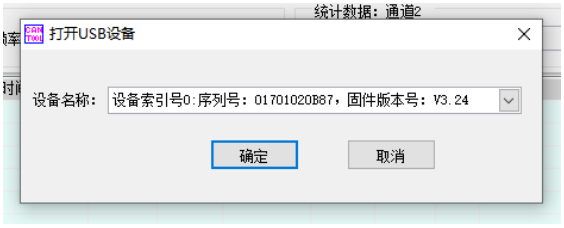
8) 然后可以打开 USB-CAN 软件。



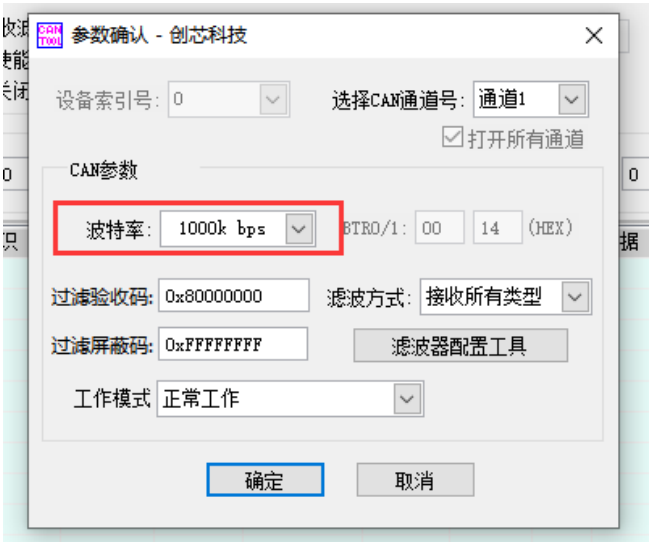
9) 然后点击启动设备。



10) 然后点击确定。



11) 再设置波特率为 1000k bps。



12) 成功打开后 USB-CAN 软件会显示序列号等信息。



13) 然后编译安装下 can-utils 软件包。

```
[openEuler@openEuler ~]$ git clone https://github.com/linux-can/can-utils
[openEuler@openEuler ~]$ cd can-utils/
[openEuler@openEuler can-utils]$ sudo make && sudo make install
```

14) 开发板接收 CAN 消息测试。

a. 首先在开发板的 Linux 系统中设置下 CAN 总线的波特率为 **1000kbps**。

```
[openEuler@openEuler ~]$ sudo ip link set can3 down
[openEuler@openEuler ~]$ sudo ip link set can3 type can bitrate 1000000
[openEuler@openEuler ~]$ sudo ip link set can3 up
```

b. 然后运行 **candump can3** 命令准备接收消息。

```
[openEuler@openEuler ~]$ sudo candump can3
```

c. 然后在 USB-CAN 软件中发送一个消息给开发板。



d. 如果开发板中可以接收到分析仪发送的消息说明 CAN 总线能正常使用。

```
[openEuler@openEuler ~]$ sudo candump can3
    can3  001   [8]  01 02 03 04 05 06 07 08
```

15) 开发板发送 CAN 消息测试。

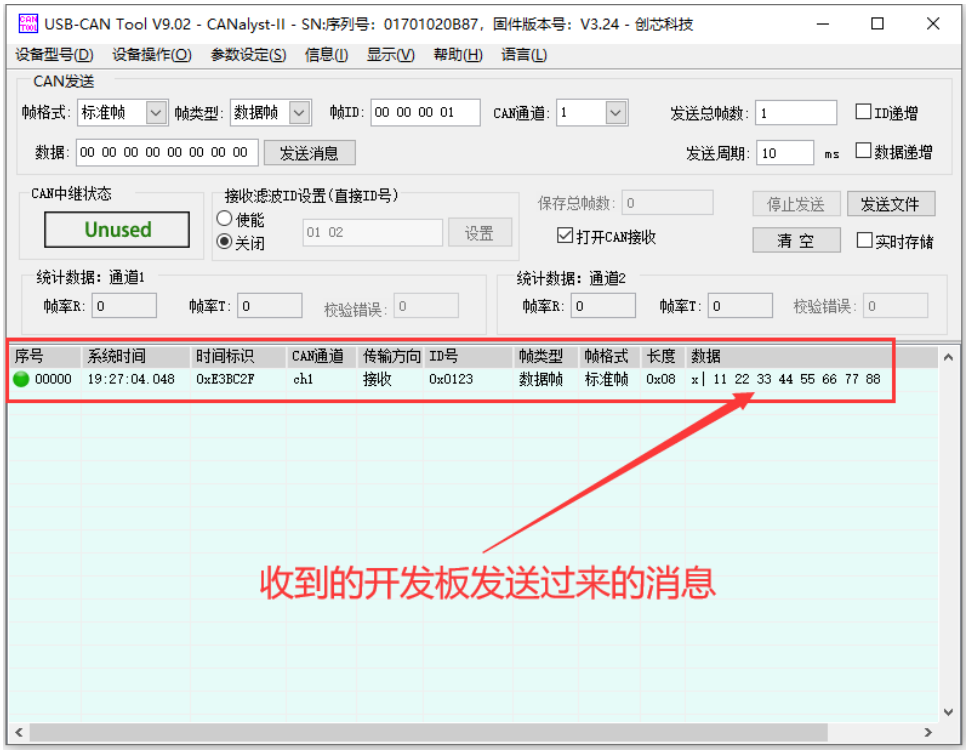
a. 首先在 Linux 系统中设置下 CAN 的波特率为 **1000kbps**。

```
[openEuler@openEuler ~]$ sudo ip link set can3 down
[openEuler@openEuler ~]$ sudo ip link set can3 type can bitrate 1000000
[openEuler@openEuler ~]$ sudo ip link set can3 up
```

b. 再在开发板中执行 **cansend** 命令，发送一个消息。

```
[openEuler@openEuler ~]$ sudo cansend can3 123#1122334455667788
```

c. 如果 USB-CAN 软件可以接收到开发板发过来的消息说明通信成功。



3.11. wiringOP 的安装使用方法

3.11.1. 安装 wiringOP 的方法

1) 安装 wiringOP 前，请先确保 Linux 系统中存在**/etc/orangepi-release** 这个配置文件，里面的内容为：**BOARD=orangepikunpengpro**。

```
[openEuler@openEuler ~]$ cat /etc/orangepi-release
BOARD=orangepikunpengpro
```

2) 如果 Linux 中没有**/etc/orangepi-release** 这个配置文件，可以使用下面的命令创建一个。

```
[openEuler@openEuler ~]$ echo "BOARD=orangepikunpengpro" | sudo tee /etc/orangepi-release
```

3) 下载 wiringOP 的代码。

```
[openEuler@openEuler ~]$ sudo dnf install -y git
[openEuler@openEuler ~]$ git clone https://github.com/orangepi-xunlong/wiringOP.git -b next
```

注意，源码需要下载 wiringOP next 分支的代码，请别漏了-b next 这个参数。

4) 然后编译安装 wiringOP。

```
[openEuler@openEuler ~]$ cd wiringOP
[openEuler@openEuler wiringOP]$ sudo ./build clean
[openEuler@openEuler wiringOP]$ sudo ./build
```

5) 测试 gpio readall 命令的输出如下：

```
+------+-----+----------+--------+---+  AI PRO  +---+--------+----------+-----+------+
| GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
+------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
|      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
|   76 |   0 |     SDA7 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
|   75 |   1 |     SCL7 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
|  226 |   2 | GPIO7_02 |    OFF | 0 |  7 || 8  | 0 | OFF    | UTXD0    | 3   | 14   |
|      |     |      GND |        |   |  9 || 10 | 0 | OFF    | URXD0    | 4   | 15   |
|   82 |   5 | GPIO2_18 |    OFF | 0 | 11 || 12 | 0 | OFF    | GPIO7_03 | 6   | 227  |
|   38 |   7 | GPIO1_06 |     IN | 1 | 13 || 14 |   |        | GND      |     |      |
|   79 |   8 | GPIO2_15 |     IN | 1 | 15 || 16 | 1 | IN     | GPIO2_16 | 9   | 80   |
|      |     |     3.3V |        |   | 17 || 18 | 1 | IN     | GPIO0_25 | 10  | 25   |
|   91 |  11 | SPI0_SD0 |    OFF | 0 | 19 || 20 |   |        | GND      |     |      |
|   92 |  12 | SPI0_SDI |    OFF | 0 | 21 || 22 | 1 | IN     | GPIO0_02 | 13  | 2    |
|   89 |  14 | SPI0_CLK |    OFF | 0 | 23 || 24 | 0 | OFF    | SPI0_CS  | 15  | 90   |
|      |     |      GND |        |   | 25 || 26 | 1 | IN     | GPIO2_19 | 16  | 83   |
|      |     |     SDA6 |        |   | 27 || 28 |   |        | SCL6     |     |      |
|  231 |  17 |    URXD7 |    OFF | 0 | 29 || 30 |   |        | GND      |     |      |
|   84 |  18 | GPIO2_20 |     IN | 1 | 31 || 32 | 1 | IN     | PWM3     | 19  | 33   |
|  128 |  20 | GPIO4_00 |     IN | 1 | 33 || 34 |   |        | GND      |     |      |
|  228 |  21 | GPIO7_04 |    OFF | 0 | 35 || 36 | 0 | OFF    | GPIO2_17 | 22  | 81   |
|    3 |  23 | GPIO0_03 |     IN | 1 | 37 || 38 | 1 | IN     | GPIO7_06 | 24  | 230  |
|      |     |      GND |        |   | 39 || 40 | 0 | OFF    | GPIO7_05 | 25  | 229  |
+------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
| GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
+------+-----+----------+--------+---+  AI PRO  +---+--------+----------+-----+------+
```

3.11.2. 使用 wiringOP 控制 40pin GPIO 的方法

1) 下面以 7 号引脚——对应 GPIO 为 GPIO7_02——对应 wPi 序号为 2——为例演示如何设置 GPIO 口的方向和高低电平。

```
+------+-----+----------+--------+---+  AI PRO  +---+--------+----------+-----+------+
| GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
+------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
|      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
|   76 |   0 |     SDA7 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
|   75 |   1 |     SCL7 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
|  226 |   2 | GPIO7_02 |    OFF | 0 |  7 || 8  | 0 | OFF    | UTXD0    | 3   | 14   |
|      |     |      GND |        |   |  9 || 10 | 0 | OFF    | URXD0    | 4   | 15   |
|   82 |   5 | GPIO2_18 |    OFF | 0 | 11 || 12 | 0 | OFF    | GPIO7_03 | 6   | 227  |
|   38 |   7 | GPIO1_06 |     IN | 1 | 13 || 14 |   |        | GND      |     |      |
|   79 |   8 | GPIO2_15 |     IN | 1 | 15 || 16 | 1 | IN     | GPIO2_16 | 9   | 80   |
```

2) 首先设置 GPIO 口为输出模式，其中第三个参数需要输入引脚对应的 wPi 的序号。

```
[openEuler@openEuler ~]$ gpio mode 2 out
```

3) 然后使用下面的命令可以查看下 GPIO 当前的模式，可以看到当前的模式为输出——**OUT**。命令中第二个参数需要输入引脚对应的 wPi 的序号。

```
[openEuler@openEuler ~]$ gpio qmode 2
OUT
```

4) 然后就可以开始设置引脚输出高低电平了。比如使用下面的命令可以设置 GPIO 口输出低电平，设置完后可以使用万用表测量下引脚的电压的数值，如果为 0v，说明设置低电平成功。

```
[openEuler@openEuler ~]$ gpio write 2 0
```

5) 除了使用万用表测量引脚的电压外，还可以使用 **gpio read** 命令查看引脚的高低电平状态。当前命令输出为 0，说明引脚目前为低电平。

```
[openEuler@openEuler ~]$ gpio read 2
0
```

6) 然后可以设置 GPIO 口输出高电平，设置完后可以使用万用表测量引脚的电压的数值，如果为 3.3v，说明设置高电平成功。

```
[openEuler@openEuler ~]$ gpio write 2 1
```

7) 除了使用万用表测量引脚的电压外，还可以使用 **gpio read** 命令查看引脚的高低电平状态。当前命令输出为 1，说明引脚目前为高电平。

```
[openEuler@openEuler ~]$ gpio read 2
1
```

8) 另外使用 **gpio readall** 命令也可以查看 GPIO 当前的设置情况。

```
+------+-----+----------+--------+---+  AI PRO  +---+--------+----------+-----+------+
| GPIO | wPi |   Name   |  Mode  | V | Physical | V |  Mode  | Name     | wPi | GPIO |
+------+-----+----------+--------+---+----++----+---+--------+----------+-----+------+
|      |     |     3.3V |        |   |  1 || 2  |   |        | 5V       |     |      |
|   76 |   0 |     SDA7 |    OFF | 0 |  3 || 4  |   |        | 5V       |     |      |
|   75 |   1 |     SCL7 |    OFF | 0 |  5 || 6  |   |        | GND      |     |      |
|  226 |   2 | GPIO7_02 |    OUT | 1 |  7 || 8  | 0 | OFF    | UTXD0    | 3   | 14   |
|      |     |      GND |        |   |  9 || 10 | 0 | OFF    | URXD0    | 4   | 15   |
|   82 |   5 | GPIO2_18 |    OFF | 0 | 11 || 12 | 0 | OFF    | GPIO7_03 | 6   | 227  |
|   38 |   7 | GPIO1_06 |     IN | 1 | 13 || 14 |   |        | GND      |     |      |
|   79 |   8 | GPIO2_15 |     IN | 1 | 15 || 16 | 1 | IN     | GPIO2_16 | 9   | 80   |
```

9) 使用下面的命令可以将 GPIO 口设置为输入模式，其中第三个参数需要输入引脚对应的 wPi 的序号。

```
[openEuler@openEuler ~]$ gpio mode 2 in
```

10) 将 GPIO 口设置为输入模式后，当将 GPIO 口用杜邦线连接到 GND 引脚后，使用 **gpio read** 命令可以看到 GPIO 口的输入值会为 0。

```
[openEuler@openEuler ~]$ gpio read 2
0
```

11) 将 GPIO 口设置为输入模式后，当将 GPIO 口用杜邦线连接到 3.3v 引脚后，使用 **gpio read** 命令可以看到 GPIO 口的输入值会为 1。

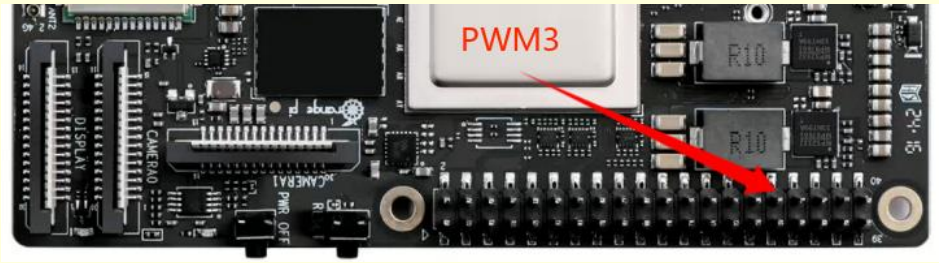
```
[openEuler@openEuler ~]$ gpio read 2
1
```

12) 其他引脚的设置方法类似，只需修改 wPi 的序号为引脚对应的序号即可。

3.12. wiringOP 硬件 PWM 的使用方法

使用 wiringOP 操作 PWM 前，请确保 Linux 系统已经安装了 wiringOP。如果 gpio readall 命令能正常使用，说明 wiringOP 已经安装了。如果提示找不到命令，请参考 wiringOP 的安装使用方法一小节的说明先安装下 wiringOP。

开发板 40pin 接口中可以使用 PWM3 这路 PWM，引脚的位置如下图所示：



3.12.1. 使用 wiringOP 的 gpio 命令设置 PWM 的方法

3.12.1.1. 设置对应引脚为 PWM 模式的方法

1) 开发板 40 pin 接口引脚的功能如下表所示，其中标红部分的引脚具有 pwm 功能。

GPIO序号	GPIO	功能	引脚	引脚	功能	GPIO	GPIO序号
		3.3V	1	2	5V		
76	GPIO2_12	SDA7	3	4	5V		
75	GPIO2_11	SCL7	5	6	GND		
226	GPIO7_02	UTXD7	7	8	UTXD0	GPIO0_14	14
		GND	9	10	URXD0	GPIO0_15	15
82	GPIO2_18	URXD2	11	12		GPIO7_03	227
38	GPIO1_06		13	14	GND		
79	GPIO2_15		15	16		GPIO2_16	80
		3.3V	17	18		GPIO0_25	25
91	GPIO2_27	SPI0_SDO	19	20	GND		
92	GPIO2_28	SPI0_SDI	21	22		GPIO0_02	2
89	GPIO2_25	SPI0_SCLK	23	24	SPI0_CS	GPIO2_26	90
		GND	25	26		GPIO2_19	83
		SDA6	27	28	SCL6		
231	GPIO7_07	URXD7	29	30	GND		
84	GPIO2_20		31	32	PWM3	GPIO1_01	33
128	GPIO4_00		33	34	GND		
228	GPIO7_04		35	36	UTXD2	GPIO2_17	81
3	GPIO0_03		37	38		GPIO7_06	230
		GND	39	40		GPIO7_05	229

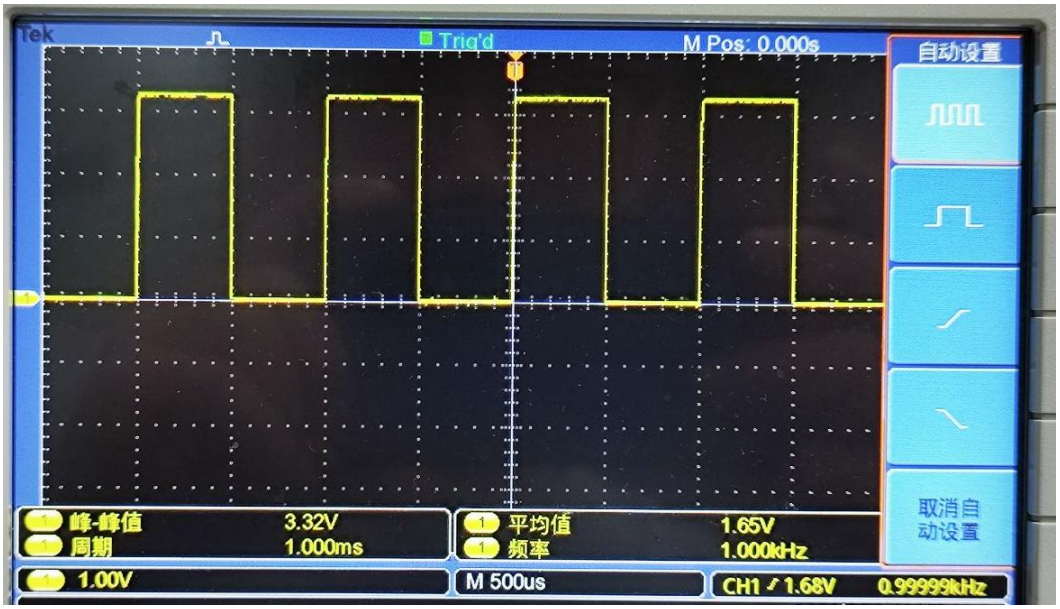
2) 开发板 40Pin 中 PWM 引脚与 wPi 序号对应关系如下表所示：

PWM 引脚	wPi 序号
PWM3	19

3) 设置引脚为 PWM 模式的命令如下，其中第三个参数需要输入 PWM 引脚对应的 wPi 的序号。

```
[openEuler@openEuler ~]$ gpio mode 19 pwm
```

4) 引脚设置为 PWM 模式后，默认会输出一个频率为 1kHz，周期为 1ms，占空比为 50%的方波，此时，我们使用示波器测量 PWM3 引脚，就可以看到下面的波形。



5) 然后使用 **gpio qmode 19** 命令可以看到 PWM3 引脚的模式设置为了 PWM。

```
[openEuler@openEuler ~]$ gpio qmode 19
PWM
```

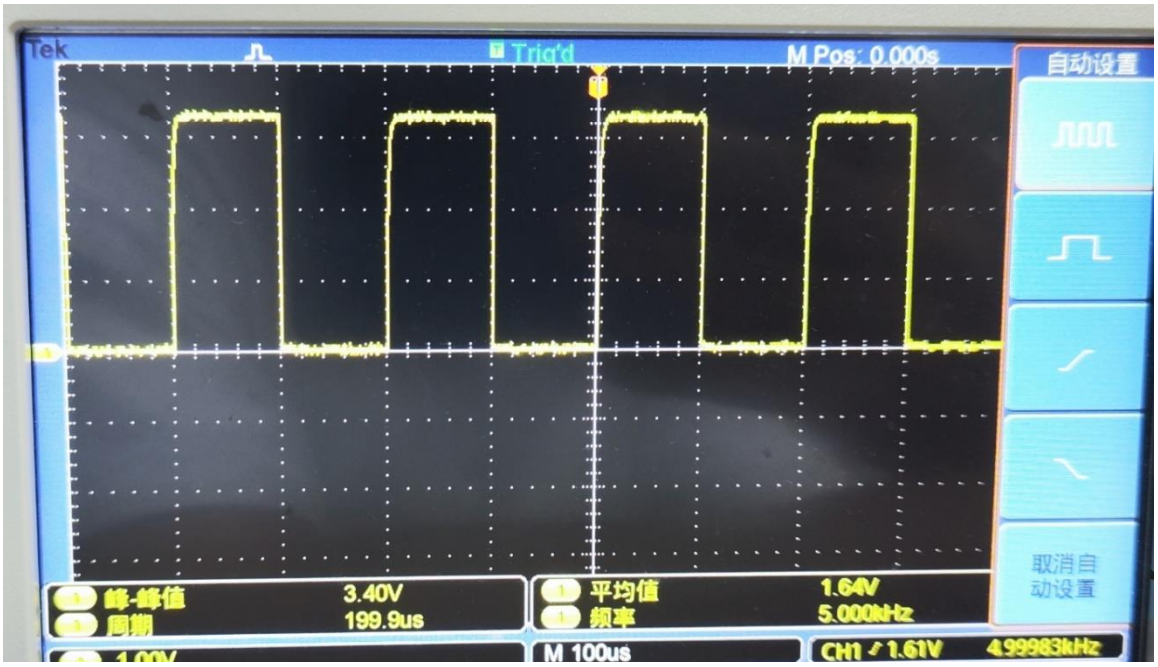
3.12.1.2. 直接设置 PWM 频率的方法

设置 PWM 的频率前，请先确保对应的引脚已设置为了 PWM 模式。

1) 我们可以使用 **gpio pwmTone** 命令来设置 PWM 引脚的频率，比如使用下面的命令可以设置 PWM 的频率为 5000Hz。设置频率的同时，**gpio pwmTone** 命令还会将占空比设置为 **50%**。另外需要注意的是，PWM 频率可以设置的范围为：**1 ~ 32000hz**，不在这个范围内的频率值会报错。

```
[openEuler@openEuler ~]$ gpio pwmTone 19 5000
```

2) 然后通过示波器可以观察到 PWM 频率变为 5000Hz，并且占空比为 50%。



3.12.1.3. 通过脉冲周期寄存器设置 PWM 周期的方法

1) PWM3 脉冲周期寄存器——PWM_PRD3 的说明如下所示：

PWM_PRD3	3通道脉冲周期寄存器	0x2C	32	0x0FFFFFFF	reserved	31:28	-	0x0	保留
					pwm_prd3	27:0	RW	0xFFFFFFF	通道3输出PWM信号的周期

2) PWM 使用的 APB 总线时钟为 150MHz，并且此时钟不能分频（**所以不支持通过 pwmc 命令来修改时钟频率**）。PWM 输出波形周期的计算公式如下所示：

PWM 周期 = PWM_PRD3 寄存器的值 / 150 (PWM 周期单位为 us)

PWM_PRD3 寄存器的值 = PWM 周期 x 150 (PWM 周期单位为 us)

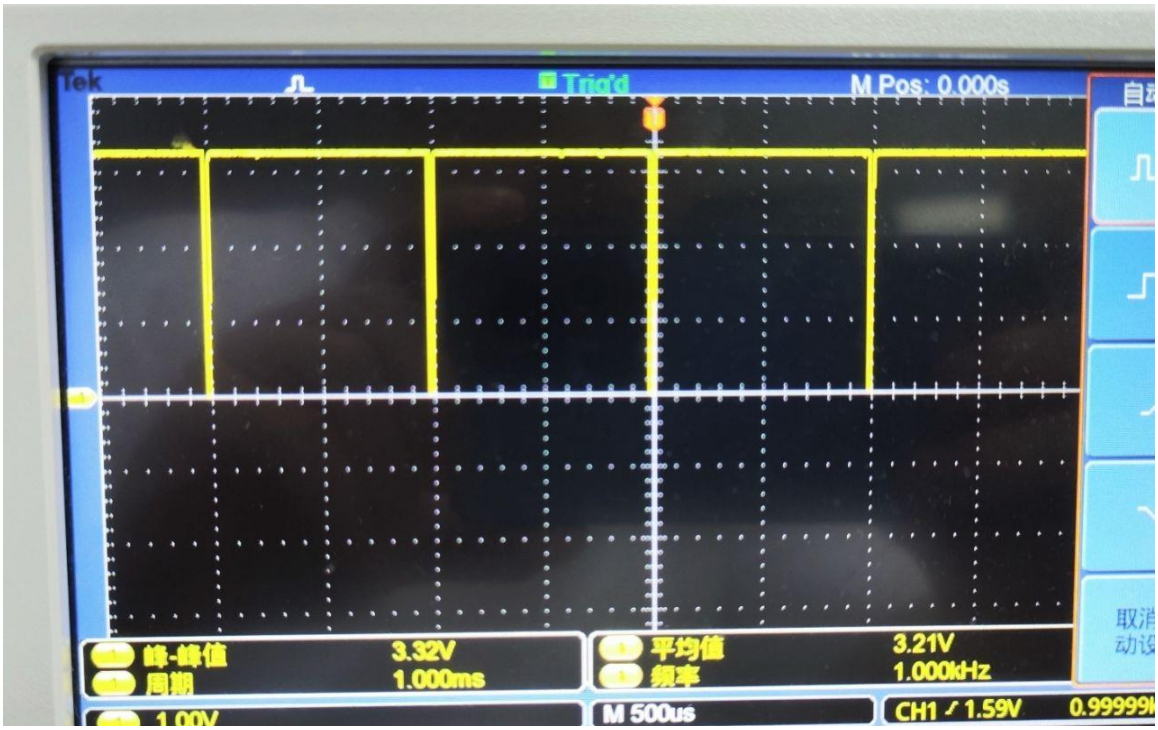
3) **gpio pwmr** 命令可以用来设置脉冲周期寄存器的值，比如要设置 PWM 输出波形的周期为 1000us，通过上面的公式可以知道 PWM_PRD3 寄存器的值需要设置为 150000。

```
[openEuler@openEuler ~]$ gpio pwmr 19 150000
```

由于 PWM 频率的值建议在 1 ~ 32000hz 之间，所以 PWM_PRD3 寄存器的取

值范围为：4688 ~ 150000000。

4) 设置完后可以通过示波器看到 PWM 的波形如下所示，显示周期为 1ms，也就是 1000us。



5) 已知周期就可以算出频率，所以此命令也可以用来设置 PWM 输出波形的频率。但此命令只会修改脉冲周期寄存器，不会将占空比设置为 50%。

3.12.1.4. 调节 PWM 占空比的方法

1) **gpio pwm** 命令可以设置 PWM 的占空比。**gpio pwm** 命令的使用方法如下所示：

a. **pin** 需要填写 pwm 引脚对应的 wPi 序号。

b. **value** 需要填写高电平占用的周期值。**占空比 = value / 脉冲周期寄存器的值**。比如脉冲周期寄存器设置为 150000，如果需要设置占空比为 50%，则 value 需要设置为 75000。另外请注意，value 值最大为：**脉冲周期寄存器的值 - 1**。

```
Usage: gpio pwm <pin> <value>
```

2) 设置 PWM 占空比的示例。

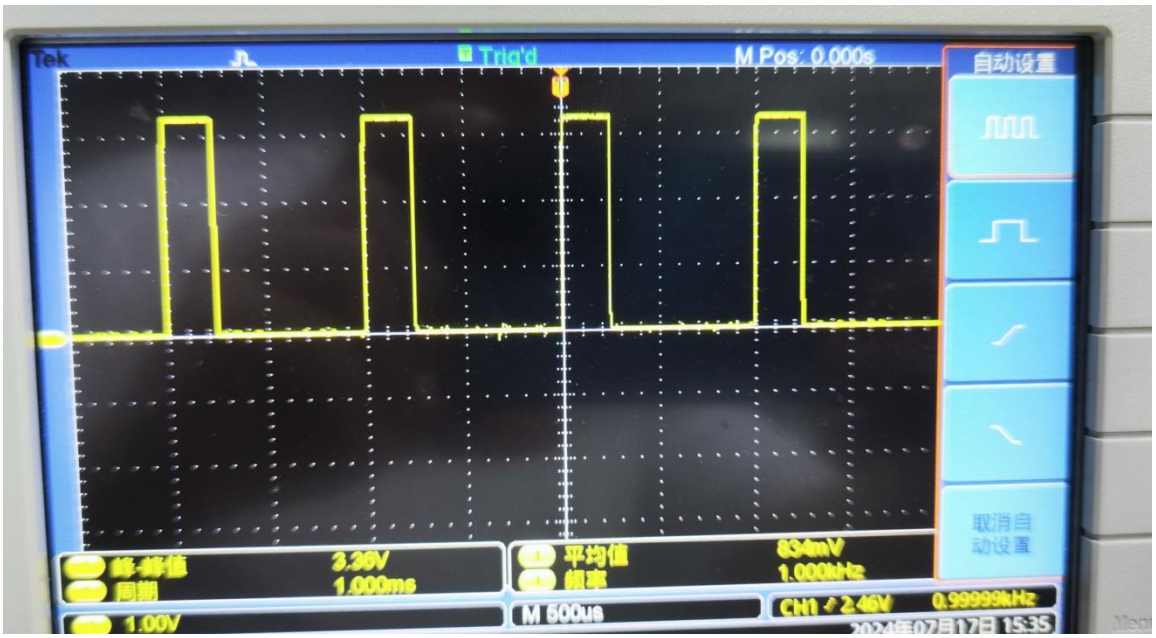
a. 首先设置脉冲周期寄存器，比如先使用下面的命令将其设置为 150000。

```
[openEuler@openEuler ~]$ gpio pwmr 19 150000
```

b. 然后使用下面的命令可以将占空比设置为 25%。

```
[openEuler@openEuler ~]$ gpio pwm 19 37500
```

3) 运行上面的命令后，通过示波器可以观察到 PWM 占空比会变为 25%。



3.12.2. PWM 测试程序的使用方法

1) 除了使用 gpio 命令来控制 PWM 外，还可以在 C 语言程序中调用 PWM 相关的 API 来控制 PWM 的波形。在 wiringOP 的 example 目录下，有一个名为 pwm.c 的程序，此程序演示了使用 wiringOP 中 PWM 相关的 API 来操作 PWM 的方法。

```
[openEuler@openEuler ~]$ cd wiringOP
[openEuler@openEuler wiringOP]$ cd examples
[openEuler@openEuler examples]$ ls pwm.c
pwm.c
```

2) 编译 **pwm.c** 为可执行程序的命令如下所示：

```
[openEuler@openEuler examples]$ gcc -o pwm pwm.c -lwiringPi
```

3) 然后就可以执行 PWM 测试程序了，在执行 PWM 测试程序时，需要指定 PWM 引脚对应的 wPi 序号，比如可以使用下面的命令对 PWM3 引脚进行测试：

```
[openEuler@openEuler examples]$ sudo ./pwm 19
```

4) pwm 程序执行后会对以下内容依次进行测试：

a. 通过设置 ARR，即脉冲周期寄存器，来调节 PWM 占空比。
b. 通过设置 CCR，即高电平占用的周期数，来调节 PWM 占空比。
c. 直接设置 PWM 频率。

5) 每完成一项测试后，pwm 测试程序会保持最后的 pwm 波形 5 秒钟，在完成所有测试内容后，会重新开始新一轮测试。

6) PWM 测试程序的详细执行过程如下所示：

a. 通过设置 ARR 调节 PWM 占空比：通过示波器可以观察到 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 50%变为 75%，保持 5 秒，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 75%变为 50%，保持 5 秒。
b. 通过设置 CCR 调节 PWM 占空比：通过示波器可以观察到 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 50%变为 100%，保持 5 秒，然后 PWM 波形每隔 0.5 秒产生变化，在变化了 8 次后，PWM 占空比从 100%变为 50%，保持 5 秒。
c. 直接设置PWM频率：通过示波器可以观察到PWM频率首先变为2000Hz，然后每隔两秒 PWM 频率增加 2000Hz，在变化了 9 次后，PWM 频率变为 20000Hz，保持 5 秒。

3.13. 上传文件到开发板 Linux 系统中的方法

3.13.1. 在 Ubuntu PC 中上传文件到开发板 Linux 系统中的方法

3.13.1.1. 使用 scp 命令上传文件的方法

1) 使用 scp 命令可以在 Ubuntu PC 中上传文件到开发板的 Linux 系统中，具体命令如下所示：

a. **file_path**：需要替换为要上传文件的路径。
b. **root**：为开发板 Linux 系统的用户名，也可以替换成其他的，比如

openEuler。

c. **192.168.xx.xx:** 为开发板的 IP 地址，请根据实际情况进行修改。

d. **/root:** 开发板 Linux 系统中的路径，也可以修改为其他的路径。

```
test@test:~$ scp  file_path  root@192.168.xx.xx:/root/
```

2) 如果要上传文件夹，需要加上**-r** 参数。

```
test@test:~$ scp  -r  dir_path  root@192.168.xx.xx:/root/
```

3) scp 还有更多的用法，请使用下面的命令查看 man 手册。

```
test@test:~$ man scp
```

3.13.1.2. 使用 filezilla 上传文件的方法

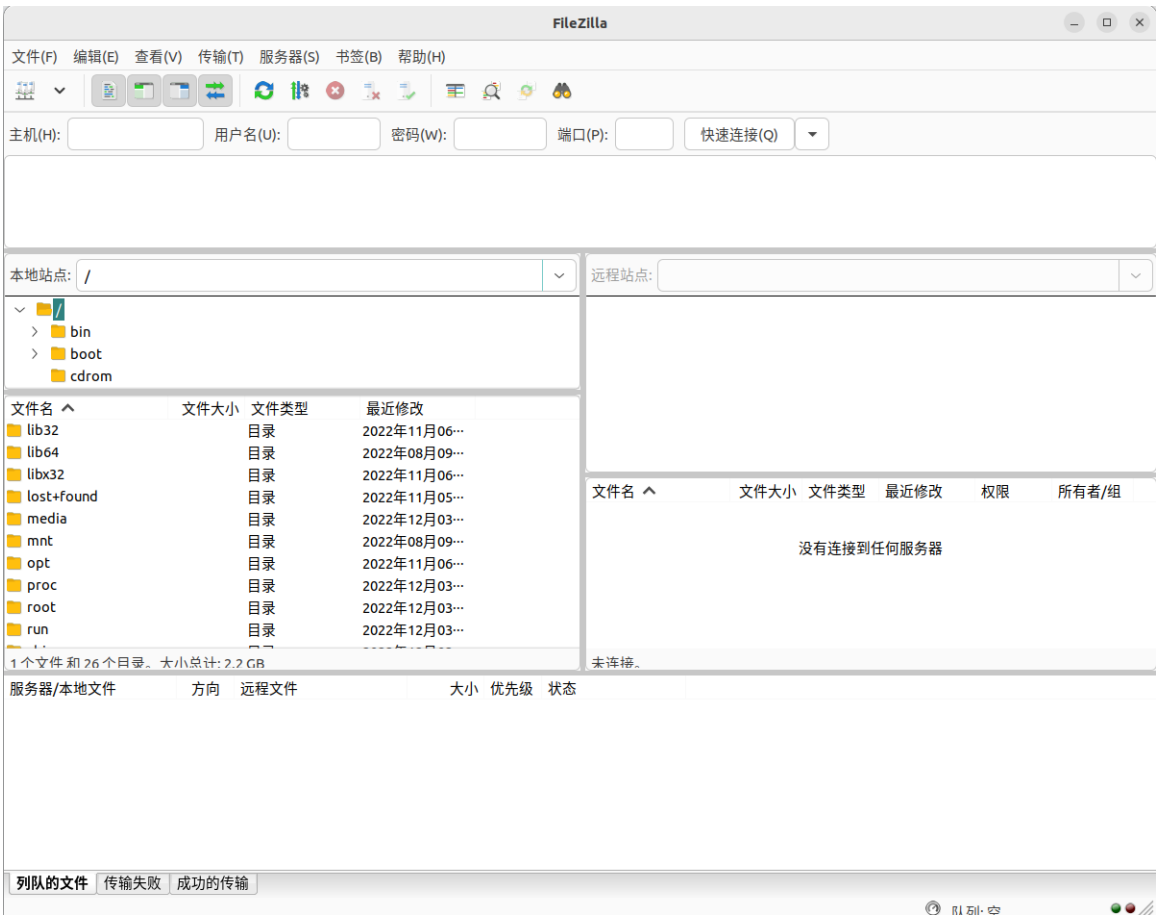
1) 首先在 Ubuntu PC 中安装 filezilla。

```
test@test:~$ sudo apt-get install -y filezilla
```

2) 然后使用下面的命令打开 filezilla。

```
test@test:~$ filezilla
```

3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的。



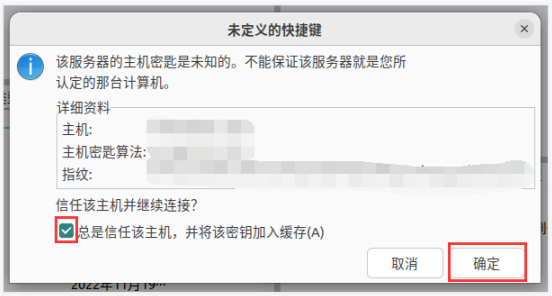
4) 连接开发板的方法如下图所示：



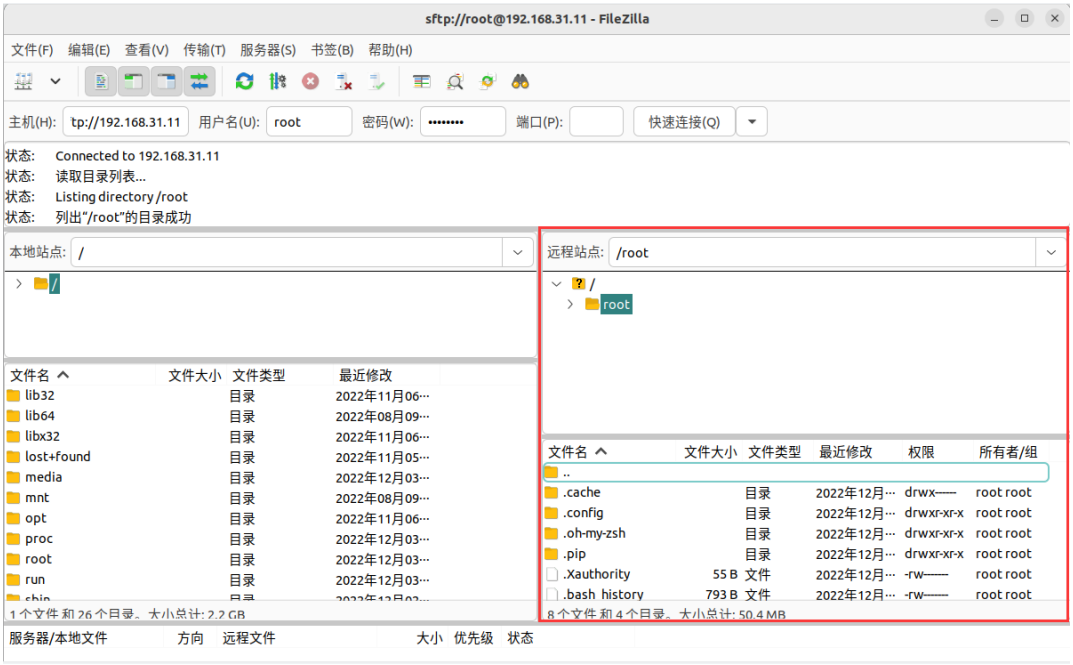
5) 然后选择**保存密码**，再点击**确定**。



6) 然后选择**总是信任该主机**，再点击**确定**。

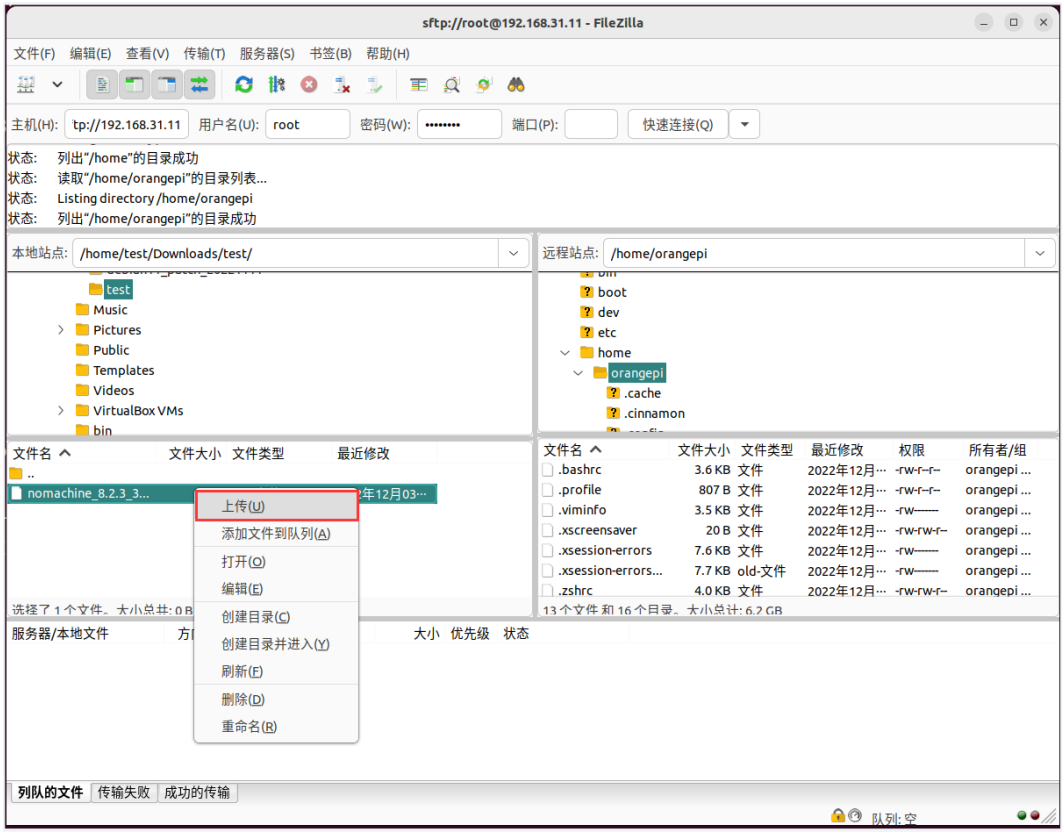


7) 连接成功后在 filezilla 软件的右边就可以看到开发板 linux 文件系统的目录结构了。



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左

边选中 Ubuntu PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始上传文件到开发板中了。



9) 上传完成后就可以去开发板 Linux 系统中的对应路径中查看上传的文件了。

10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了。

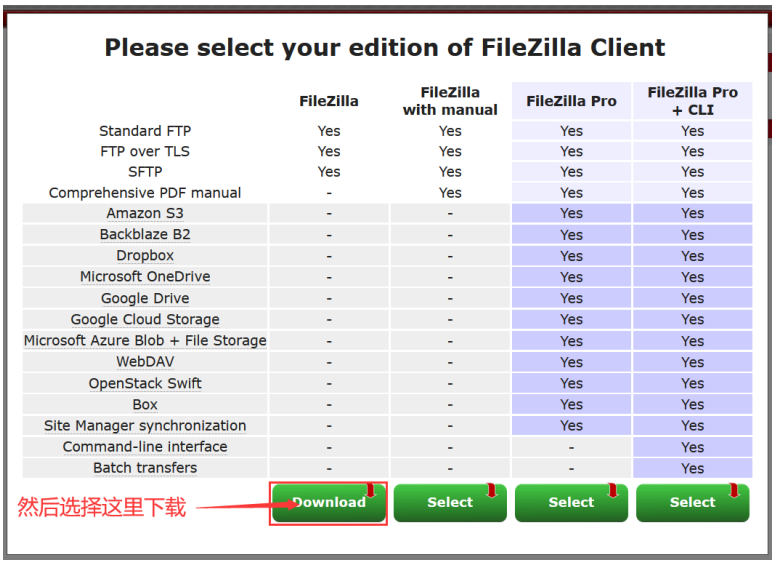
3.13.2. 在 Windows PC 中上传文件到开发板 Linux 系统中的方法

3.13.2.1. 使用 filezilla 上传文件的方法

1) 首先下载 filezilla 软件 Windows 版本的安装文件，下载链接如下所示：

https://filezilla-project.org/download.php?type=client

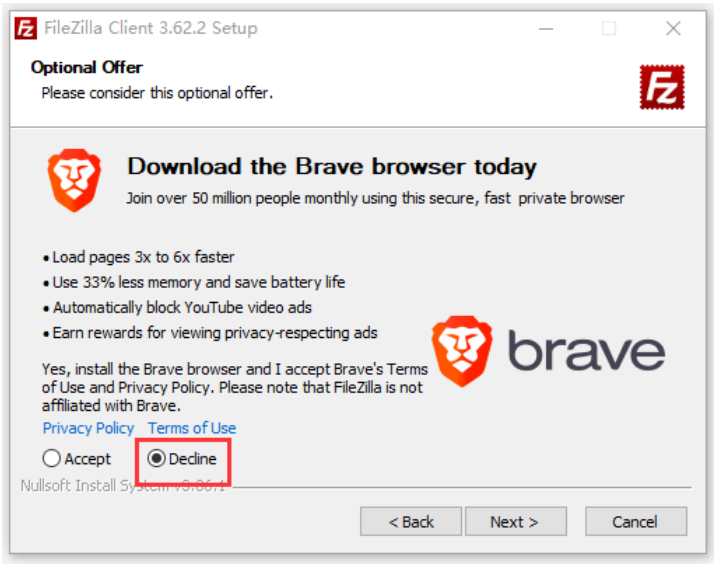




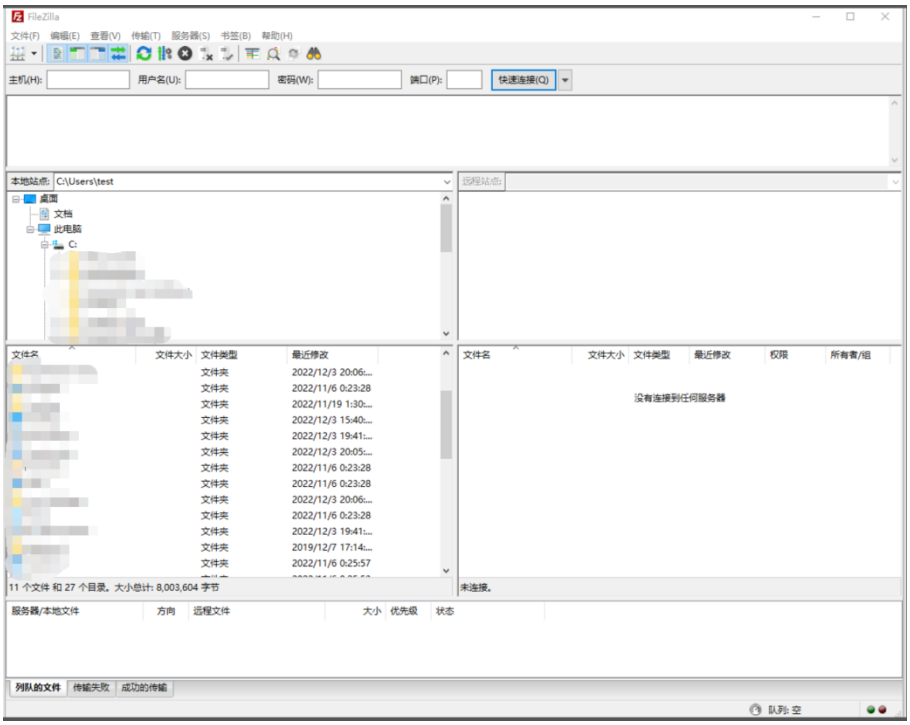
2) 下载的安装包如下所示，然后双击直接安装即可。

FileZilla_Server_x.x.x_win64-setup.exe

安装过程中，下面的安装界面请选择 `Decline`，然后再选择 `Next>`。



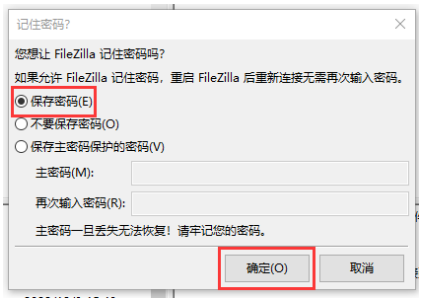
3) filezilla 打开后的界面如下所示，此时右边远程站点下面显示的是空的。



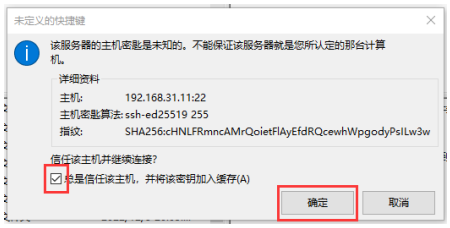
4) 连接开发板的方法如下图所示：



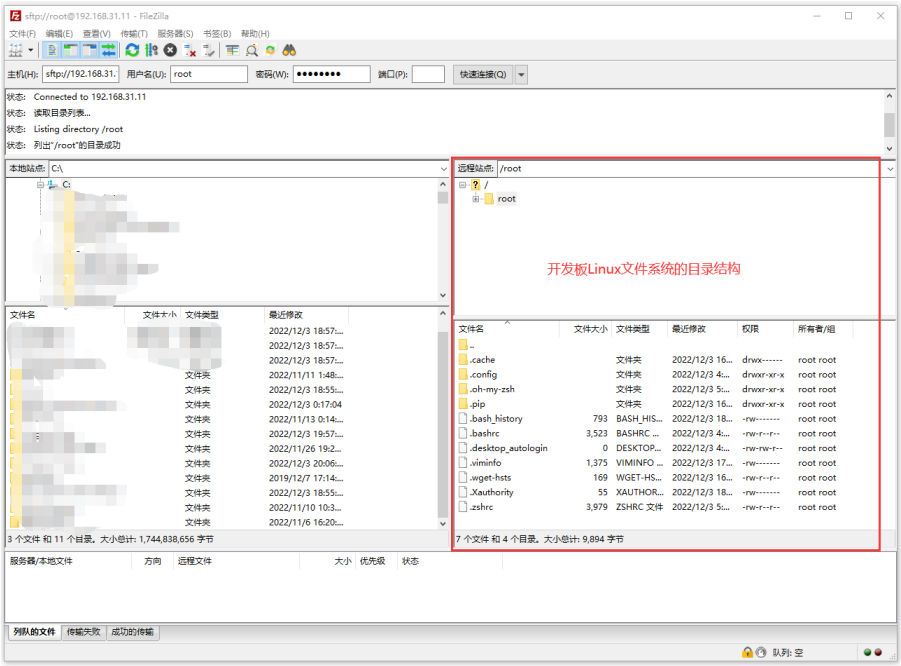
5) 然后选择**保存密码**，再点击**确定**。



6) 然后选择**总是信任该主机**，再点击**确定**。

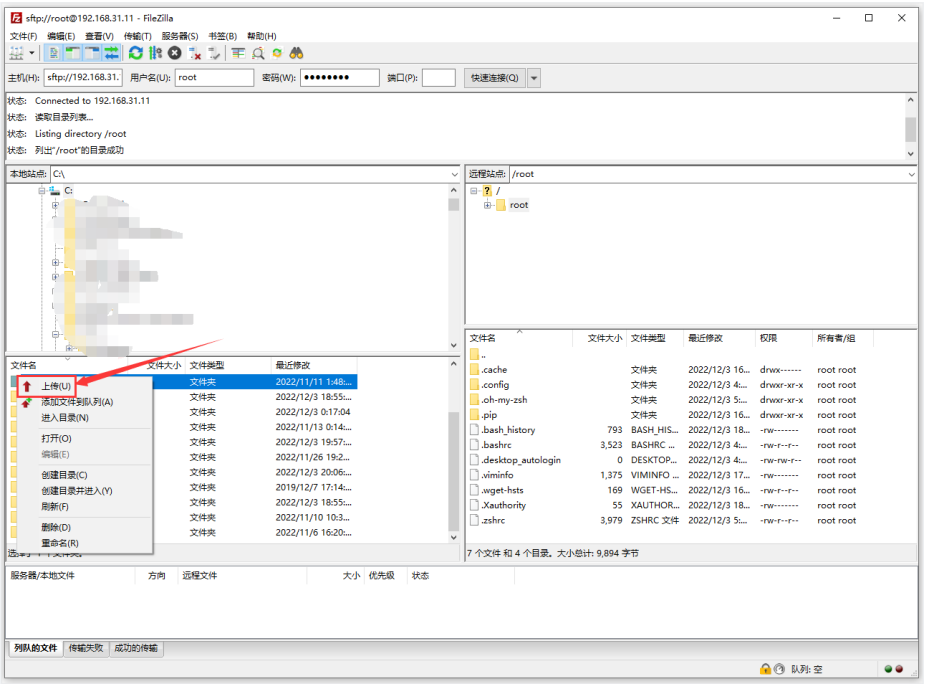


7) 连接成功后在 filezilla 软件的右边就可以看到开发板 linux 文件系统的目录结构了。



8) 然后在 filezilla 软件的右边选择要上传到开发板中的路径，再在 filezilla 软件的左边选中 Windows PC 中要上传的文件，再点击鼠标右键，再点击上传选项就会开始

上传文件到开发板中了。

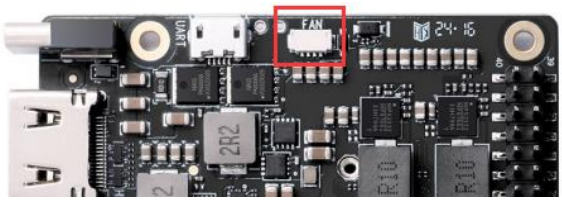


9) 上传完成后就可以去开发板 Linux 系统中的对应路径中查看上传的文件了。

10) 上传文件夹的方法和上传文件的方法是一样的，这里就不再赘述了。

3.14. 散热风扇的使用方法

开发板散热风扇的接口所在的位置如下所示：



开发板使用的散热风扇为 12V 的，接口为 4pin，0.8mm 间距规格。可以通过 PWM 来控制风扇的转速。

使用 npu-smi 命令可以查询和控制 PWM 风扇，详细用法如下所示：

1) 查询风扇模式的命令如下所示：

```
[openEuler@openEuler ~]$ sudo npu-smi info -t pwm-mode
        pwm-mode                    : auto
```

字段	说明
pwm-mode	风扇模式。 有如下两种模式： • manual：手动模式 • auto：自动模式 默认为自动模式。

2) 查询风扇调速比的命令如下所示：

```
[openEuler@openEuler ~]$ sudo npu-smi info -t pwm-duty-ratio
        pwm-duty-ratio(%)           : 15
```

3) 设置风扇模式为手动模式的命令如下所示：

```
[openEuler@openEuler ~]$ sudo npu-smi set -t pwm-mode -d 0
```

描述
风扇使能模式。分为手动模式、自动模式。默认为自动模式。 • 0：手动模式 • 1：自动模式

4) 将风扇设置为手动模式后就可以通过下面的命令来设置风扇的调速比了。比如下面的命令会将风扇调速比设置为 100，设置完后风扇会用最大的转速运行。

```
[openEuler@openEuler ~]$ sudo npu-smi set -t pwm-duty-ratio -d 100
```

描述
风扇调速比 取值范围：[0-100]

3.15. 设置 Swap 内存的方法

虽然开发板有 8GB 或 16GB 的大内存，但有些应用需要的内存大于 8GB 或

16GB，此时我们可以通过 Swap 内存来扩展系统能使用的最大内存容量。方法如下所示：

1) 首先创建一个 swap 文件，下面的命令会创建一个 16GB 大小的 swap 文件，容量大小请根据自己的需求进行修改。

```
[openEuler@openEuler ~]$ sudo fallocate -l 16G /swapfile
```

如果已经启用了Swap分区再运行fallocate 命令会报下面的错误：

```
fallocate: fallocate failed: Text file busy
```

需要先运行swapoff /swapfile命令关闭系统上的swap分区才行。

注意，添加Swap内存前，请确保TF卡、eMMC或者SSD的剩余空间大于需要添加的Swap内存容量。

2) 然后修改文件权限，确保只有 root 用户可以读写。

```
[openEuler@openEuler ~]$ sudo chmod 600 /swapfile
```

3) 然后把这个文件设置成 swap 空间。

```
[openEuler@openEuler ~]$ sudo mkswap /swapfile
```

4) 然后启用 swap。

```
[openEuler@openEuler ~]$ sudo swapon /swapfile
```

5) 完成以上步骤后，可以通过下面的命令可以检查 swap 内存是否已经添加成功。

```
[openEuler@openEuler ~]$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.4Gi       1.1Gi       5.5Gi        27Mi       835Mi       6.1Gi
Swap:         15Gi          0B       15Gi
```

6) 如果需要 swap 设置在重启之后依然有效，请运行下面命令将对应的配置添加到**/etc/fstab** 文件中。

```
[openEuler@openEuler ~]$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

3.16. 关机和重启开发板的方法

1) 在 Linux 系统运行的过程中，如果直接拔掉电源断电，可能会导致文件系统丢失某些数据，建议断电前先使用 **poweroff** 命令关闭开发板的 Linux 系统，然后再拔掉电源。

```
[openEuler@openEuler ~]$ sudo poweroff
```

2) 除了 **poweroff** 命令可以关闭 Linux 系统外，还可以使用开发板上的关机按键来关闭开发板的 Linux 系统，然后再拔掉电源。请注意，关机按键是没有开机功能的。



3) 使用 **reboot** 命令即可重启开发板中的 Linux 系统。

```
[openEuler@openEuler ~]$ sudo reboot
```

4. Linux 内核源码包的使用说明

4.1. 编译主机系统的需求

目前的 Linux 内核源码包只在 **Ubuntu 22.04** 的 X64 电脑上测试过，所以首先请确保自己电脑安装的 Ubuntu 版本是 Ubuntu 22.04。查看电脑已安装的 Ubuntu 版本的命令如下所示，如果 Release 字段显示的不是 **22.04**，说明当前使用的 Ubuntu 版本不符合要求，请更换系统后再进行下面的操作。

```
test@test:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04 LTS
Release:        22.04
Codename:       jammy
```

如果电脑安装的是 Windows 系统，没有安装有 Ubuntu 22.04 的电脑，可以考虑使用 **VirtualBox** 或者 **VMware** 来在 Windows 系统中安装一个 Ubuntu 22.04 虚拟机。但是请注意，Linux 内核源码包没有在 WSL 虚拟机中测试过，所以无法确保能在 WSL 中正常运行。Ubuntu 22.04 **amd64** 版本的安装镜像下载地址为：

```
https://mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/22.04/ubuntu-22.04-desktop-amd64.iso
```

在电脑中或者虚拟机中安装完 Ubuntu 22.04 后，请先设置 Ubuntu 22.04 的软件源为清华源（或者其他速度快的国内源），不然后面安装软件的时候很容易由于网络原因而出错。替换清华源的步骤如下所示：

1) 替换清华源的方法参考这个网页的说明即可。

```
https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/
```

2) 注意 Ubuntu 版本需要切换到 22.04。



3) 需要替换的**/etc/apt/sources.list** 文件的内容为：

test@test:~$ **sudo mv /etc/apt/sources.list cat /etc/apt/sources.list.bak**

test@test:~$ **sudo vim /etc/apt/sources.list**

```
# 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-backports main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-security main restricted universe multiverse


# 预发布软件源，不建议启用
# deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ jammy-proposed main restricted universe multiverse
```

4) 替换完后需要更新下软件包列表，并确保没有报错。

test@test:~$ **sudo apt-get update**

4.2. 安装交叉编译工具链和依赖包

1) 交叉编译工具链的压缩包可以从开发板的资料下载页面下载到。步骤为：

a. 打开下面的链接：

http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-kunpeng.html

b. 然后选择官方工具。



c. 然后下载交叉编译工具链文件夹中的 **toolchain.tar.gz** 压缩包。



2) 然后在 Ubuntu 22.04 电脑中执行如下命令，切换至 root 用户。

```
test@test:~$ sudo -i
```

3) 然后安装下面的依赖包

```
root@test:~# apt  install -y python3 make gcc unzip pigz bison flex libncurses-dev cmake
root@test:~# apt  install -y squashfs-tools bc device-tree-compiler libssl-dev  rpm2cpio g++
```

4) 然后执行如下命令，创建**/opt/compiler** 目录，并进入到/opt/compiler 目录。

```
root@test:~# mkdir /opt/compiler
root@test:~# cd /opt/compiler
```

5) 然后将下载的 **toolchain.tar.gz** 复制到**/opt/compiler** 中，再使用下面的命令将 **toolchain.tar.gz** 解压到 **/opt/compiler** 中。

```
root@test:/opt/compiler# tar -xvf toolchain.tar.gz
```

6) 解压后的交叉编译工具链如下所示：

```
root@test:/opt/compiler# ls toolchain
aarch64-target-linux-gnu  bin  include  lib  lib64  libexec  sysroot
```

7) 然后在配置文件中增加交叉编译工具链路径。

```
root@test:~# echo "export PATH=/opt/compiler/toolchain/bin:\$PATH: " >> /etc/profile
```

8) 然后执行如下命令，使环境变量生效。

```
root@test:~# source /etc/profile
```

9) 然后可以执行如下命令，查看交叉编译工具链版本。如果显示有版本信息，则表明安装工具链成功。

```
root@test:~# aarch64-target-linux-gnu-gcc -v
Using built-in specs.
COLLECT_GCC=aarch64-target-linux-gnu-gcc
COLLECT_LTO_WRAPPER=/opt/compiler/toolchain/bin/../libexec/gcc/aarch64-target-linux-gnu/7.3.0/lto-wrapper
Target: aarch64-target-linux-gnu
.....
Thread model: posix
gcc version 7.3.0 (Do-Compiler V100R001C13B001)
```

4.3. 下载解压 Linux 内核源码包

1) Linux 内核源码压缩包可以从开发板的资料下载页面下载到。步骤为：

a. 打开下面的链接：

```
http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/service-and-support/Orange-Pi-kunpeng.html
```

b. 然后选择 Linux 源码。



c. 然后下载 Linux 内核源码压缩包 **Ascend310B-source-opi.tar.gz**。



2) 然后在 Ubuntu 22.04 电脑中，然后执行如下命令，切换至 root 用户。

```
test@test:~$ sudo -i
```

3) 然后将下载好的 Linux 内核源码压缩包 **Ascend310B-source-opi.tar.gz** 拷贝到 Ubuntu 22.04 电脑的**/opt**目录下，然后使用下面的命令解压 Linux 内核源码压缩包。

```
root@test:/opt # tar zxf Ascend310B-source-opi.tar.gz
```

4) 解压后的 Linux 内核源码包的内容如下所示：

```
root@test:/opt # cd Ascend310B-source-opi
root@test:/opt/Ascend310B-source-opi# ls
abl  build.sh  config  driver  dtb  kernel  scripts  tools
```

4. 4. 编译并生效内核 Image 文件的方法

1) 首先进入 Ubuntu 22.04 电脑中，然后执行如下命令，切换至 root 用户。

```
test@test:~$ sudo -i
```

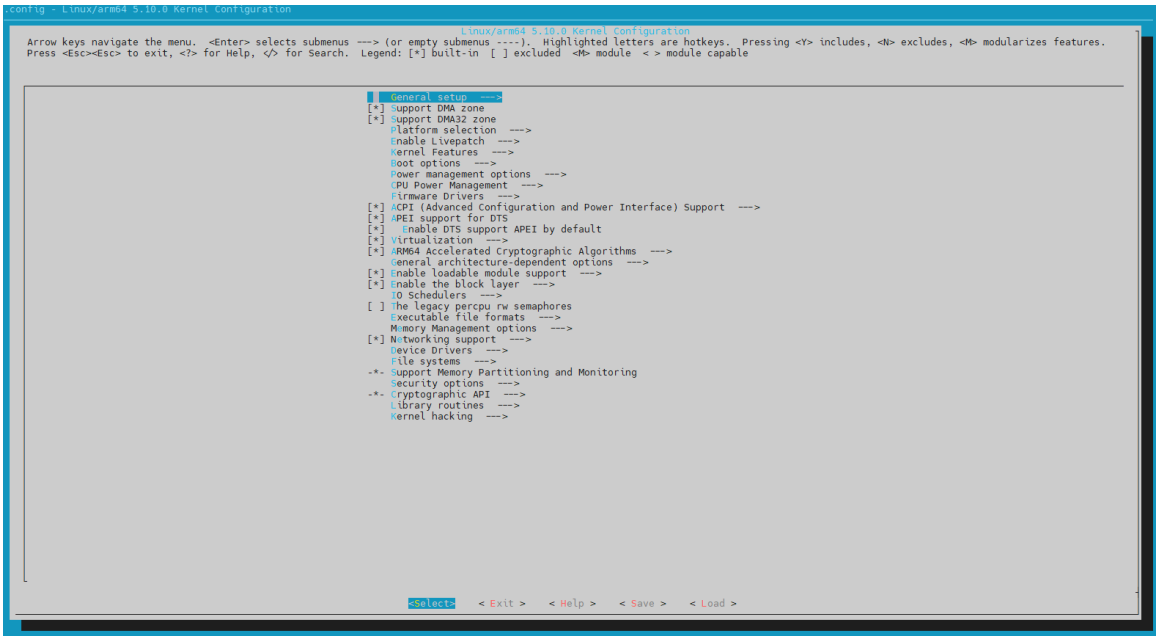
2) 然后执行如下命令，进入“**Ascend310B-source-opi**”目录。

```
root@test:~# cd /opt/Ascend310B-source-opi
```

3) 然后执行如下命令，即可开始编译内核。

```
root@test:/opt/Ascend310B-source-opi# bash build.sh kernel
```

4) 执行过程会弹出内核配置选项的图形界面，如果不需要修改，直接选择 **Exit** 退出即可。



5) 编译完成后会打印下面的信息：

```
generate /opt/Ascend310B-source-opi/output/kernel_modules success!
generate /opt/Ascend310B-source-opi/output/Image success!
sign /opt/Ascend310B-source-opi/output/Image success!
```

6) 编译后的 Image 文件会存放于 **Ascend310B-source/output** 目录下。

```
root@test:/opt/Ascend310B-source-opi# ls output/Image
output/Image
```

7) 编译后更新内核 Image 文件的方法如下所示：

a. 首先登录开发板的 Linux 系统。

b. 然后将编译好的 **Image** 文件上传至开发板 Linux 系统的任意目录下，例如 **/root** 目录下。

c. 然后进入**/root** 目录。

d. 然后执行如下命令，更新 Image 文件。

a) NVMe SSD 启动：

```
dd if=Image of=/dev/nvme0n1 count=61440 seek=32768 bs=512
```

b) SATA SSD 启动：

```
dd if=Image of=/dev/sda count=61440 seek=32768 bs=512
```

c) eMMC 启动：

```
dd if=Image of=/dev/mmcblk0 count=61440 seek=32768 bs=512
```

d) TF 卡启动：

```
dd if=Image of=/dev/mmcblk1 count=61440 seek=32768 bs=512
```

4.5. 编译并生效内核 DTB 文件的方法

2) 首先进入 Ubuntu 22.04 电脑中，然后执行如下命令，切换至 root 用户。

```
test@test:~$ sudo -i
```

3) 然后执行如下命令，进入 **Ascend310B-source-opi** 目录。

```
root@test:~# cd /opt/Ascend310B-source-opi
```

4) 开发板使用的 DTS 文件如下所示，

```
Ascend310B-source-opi/dtb/dts/hi1910b/hi1910BL/hi1910B-default.dts
```

5) 然后执行如下命令，即可开始编译 DTB。

```
root@test:/opt/Ascend310B-source-opi# bash build.sh dtb
```

6) 如果最后打印如下信息表示编译内核 DTB 文件成功。

```
generate /opt/Ascend310B-source-opi/output/dt.img success!
sign /opt/Ascend310B-source-opi/output/dt.img success!
```

7) 生成的 DTB 文件为 **dt.img**，存放在 **output** 目录下：

```
root@orangepi-M600:/opt/Ascend310B-source-opi# ls output/dt.img
output/dt.img
```

8) 编译后更新内核 DTB 文件的方法如下所示：

a. 首先登录开发板的 Linux 系统。
b. 然后将编译好的 **dt.img** 文件上传至开发板 Linux 系统的任意目录下，例如 **/root** 目录下。
c. 然后进入**/root** 目录。
d. 然后执行如下命令，更新 **dt.img** 文件。DTB 文件有主备两份，下面的两条命令中第一条是更新主区的 DTB 文件，第二条是更新备区的 DTB 文件。测试时，一般只需更新主区的 DTB 文件，等测试没问题后再更新备区的 DTB 文件。

a) NVMe SSD 启动：

```
dd if=dt.img of=/dev/nvme0n1 count=4096 seek=114688 bs=512
dd if=dt.img of=/dev/nvme0n1 count=4096 seek=376832 bs=512
```

b) SATA SSD 启动：

```
dd if=dt.img of=/dev/sda count=4096 seek=114688 bs=512
dd if=dt.img of=/dev/sda count=4096 seek=376832 bs=512
```

c) eMMC 启动：

```
dd if=dt.img of=/dev/mmcblk0 count=4096 seek=114688 bs=512
dd if=dt.img of=/dev/mmcblk0 count=4096 seek=376832 bs=512
```

d) TF 卡启动：

```
dd if=dt.img of=/dev/mmcblk1 count=4096 seek=114688 bs=512
dd if=dt.img of=/dev/mmcblk1 count=4096 seek=376832 bs=512
```

5. 附录

5.1. 用户手册更新历史

版本	日期	更新说明
v0.1	2024-02-03	初始版本
v0.2	2024-07-18	1. WIFI 蓝牙天线使用注意事项 2. 安装 wiringOP 的方法 3. 使用 wiringOP 控制 40pin GPIO 的方法 4. 40 pin CAN 的测试方法 5. wiringOP 硬件 PWM 的使用方法 6. Linux 内核源码包的使用说明
v0.3	2024-09-24	1. 添加 v1.2 版本 Kunpeng Pro 开发板的说明 2. v1.2 版本开发板搭配 OPi MSOC 的使用说明 3. 更新网口的使用说明

5.2. 镜像更新历史

日期	更新说明
2024-02-03	Kunpeng-Develop-openEuler-22.03-LTS-SP3-20240506-0416.img.xz * 初始版本
2024-09-24	Kunpeng-Develop-openEuler-22.03-LTS-SP4-20240924-1607.img.xz * 网口默认设置静态 IP 地址 * 预装 Gowin IDE